Provable Cryptography for Bitcoin: An Introduction

Jonas Nick

Blockstream Research
jonas@n-ck.net

August 27, 2025

Table of Contents

1 Introduction

This workbook has two primary goals. First, it aims to provide sufficient background to understand state-of-the-art papers on cryptographic signatures, with a focus on discrete-logarithm-based multi-party signatures, including their security proofs. Second, it seeks to develop the skills needed to formalize security notions for cryptographic primitives. This skill is crucial for selecting appropriate primitives when proposing and reviewing cryptographic protocols, and for defining precisely what a protocol aims to achieve.

The concepts introduced in this section may at first seem abstract, but this level of abstraction is important: it provides the rigorous mathematical framework on which modern provable cryptography is built. Throughout this workbook we will study algorithms, analyse their running time and success probability, both for explicit algorithms and for hypothetical adversarial algorithms. In later sections we will sometimes study these algorithms within idealised computational models.

This workbook primarily contains definitions, propositions, and lemmas, with limited intuitive explanations. Good cryptography papers should include intuition, but it's never complete: intuition is inherently subjective and what makes sense to one reader may not resonate with another. What you need to do is develop your own explanations while reading the mathematics very precisely. The goal of this workbook is to build intuition step-by-step through the exercises, which provide hands-on experience with the theoretical concepts.

Note that there is rarely a single standard definition for a concept in cryptography. Sometimes we encounter equivalent definitions in the literature, while other times we find definitions that differ in subtle but important ways. This is why contemporary papers in cryptography include a preliminaries section that rigorously defines the concepts they use.

1.1 Algorithms

"In mathematics, *algorithm* is commonly understood to be an exact prescription, defining a computational process, leading from various initial data to the desired result." (A. Markov, 1954)

Definition 1.1. *An algorithm $\mathcal{A}$ is*

- probabilistic *if it gets random bits as input (in addition to its regular input). We write $\mathcal{A}(x;\rho)$ to denote running algorithm $\mathcal{A}$ on input x with randomness ρ. When the randomness is clear from context or unnecessary, we simply write $\mathcal{A}(x)$.*
- polynomial-time *if there exists a polynomial p such that for every $x \in \{0,1\}^\lambda$, $\mathcal{A}(x)$ terminates in time $p(\lambda)$.*
- probabilistic polynomial-time (p.p.t.) *if it is probabilistic and polynomial-time.*

Note that the runtime of an algorithm is characterized relative to the size of the input. All adversaries considered in cryptography are algorithms.

1.2 Negligible Functions

Definition 1.2. *A function $f : \mathbb{Z} \to \mathbb{R}$ is* negligible *if for every constant $c > 0$ there exists $N \in \mathbb{N}$ such that for all integers $\lambda > N$, $f(\lambda) < \lambda^{-c}$.*

Example 1. $2^{-\lambda}$ is negligible. $1/\lambda$ is not negligible.

If a function f is negligible in λ, we write $f(\lambda) = negl(\lambda)$.

Lemma 1.1 (Properties of negligible functions).

1. *Let f be a negligible function and p be a polynomial. Then $f(\lambda) \cdot p(\lambda)$ is negligible.*
2. *Let f_1, f_2 be negligible functions. Then $f_1 + f_2$ is negligible.*
3. *Let f be a negligible function $f(\lambda) \geq 0$ and $k > 0$ be a constant. Then $f + k$ is not negligible.*
4. *Let f be a non-negligible function and g be a negligible function. Then $f - g$ is not negligible.*

The proof of property (1) is left as an exercise (see Exercise 1.6). The proofs of properties (2)-(4) can be found in Appendix B.

1.3 Games & Asymptotic Security

In this section we will introduce the concept of security games (sometimes called "experiments") and asymptotic security through a series of toy examples.

The $\mathsf{Game\ Guess}^{\mathcal{A}}(\lambda)$ defined in Figure 1.1 takes an algorithm $\mathcal{A}$ and the security parameter λ as input and outputs 1 or 0. The game first sets $n = 2^\lambda$ and samples x uniformly at random from $\mathbb{Z}_n$. Then it runs algorithm $\mathcal{A}$ without input, sets its output to x' and returns 1 if $x = x'$.

$\mathsf{Game\ Guess}^{\mathcal{A}}(\lambda)$

$n \coloneqq 2^\lambda$

$x \leftarrow\!\!\$\ \mathbb{Z}_n$

$x' \coloneqq \mathcal{A}()$

return $x = x'$

Fig. 1.1. Guessing game

Definition 1.3 (Guessing game advantage). *For* $\mathsf{Game\ Guess}^{\mathcal{A}}(\lambda)$ *as defined in Figure 1.1 we define the advantage of algorithm* $\mathcal{A}$ *as*

$$\mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) \coloneqq \Pr\left[\mathsf{Guess}^{\mathcal{A}}(\lambda) = 1\right].$$

Proposition 1.1. *For any algorithm* $\mathcal{A}$, $\mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) = \mathit{negl}(\lambda)$.

Proof. Let x' be the value returned by $\mathcal{A}$. The probability that x sampled independently and uniformly at random from $[1, 2^\lambda]$ is equal to x' is $2^{-\lambda} = \mathit{negl}(\lambda)$. □

While the guessing game above provides a clean security definition, formalizing the security of concrete hash functions like SHA256 requires a fundamentally different approach. To illustrate this, consider the $\mathsf{Game\ SHAPreimgZ}^{\mathcal{A}}(\lambda)$ defined in Figure 1.2, which outputs 1 if the algorithm $\mathcal{A}$ outputs a preimage of 0 under the hash function $\mathsf{SHA256} : \{0,1\}^* \to \{0,1\}^{256}$. Note that $\mathcal{A}$ gets 1^λ (a string of 1s of length λ) as input in order to characterise the running time of $\mathcal{A}$ as a function of the security parameter λ.

$\mathsf{Game\ SHAPreimgZ}^{\mathcal{A}}(\lambda)$

$y \coloneqq 0$

$x \coloneqq \mathcal{A}(1^\lambda)$

return $\mathsf{SHA256}(x) = y$

Fig. 1.2. Game for finding SHA256 preimage of zero

Proposition 1.2. *For* $\mathsf{Game\ SHAPreimgZ}^{\mathcal{A}}(\lambda)$ *as defined in Figure 1.2, let the advantage of* $\mathcal{A}$ *be defined as*

$$\mathsf{Adv}^{\mathrm{shapreiz}}_{\mathcal{A}}(\lambda) \coloneqq \Pr\left[\mathsf{SHAPreimgZ}^{\mathcal{A}}(\lambda) = 1\right].$$

If there exists x *such that* $\mathsf{SHA256}(x) = 0$*, there exists a p.p.t. algorithm* $\mathcal{A}$ *such that*

$$\mathsf{Adv}^{\mathrm{shapreiz}}_{\mathcal{A}}(\lambda) = 1.$$

Proof. Let $\mathcal{A}$ be an algorithm

return x

where $\mathsf{SHA256}(x) = 0$. □

Next, we consider a variant of preimage resistance for SHA256 where the target value is chosen uniformly at random, preventing the existence of trivial algorithms like the one in Proposition 1.2. In $\mathsf{Game\ SHAPreimg}^{\mathcal{A}}(\lambda)$ defined in Figure 1.3 the value x is a uniformly random bitstring of length 256.

$\mathsf{Game\ SHAPreimg}^{\mathcal{A}}(\lambda)$

$x \leftarrow\!\!\$\ \{0,1\}^{256}$
$y \coloneqq \mathsf{SHA256}(x)$
$x' \coloneqq \mathcal{A}(y)$
return $\mathsf{SHA256}(x') = y$

Fig. 1.3. Game for finding the SHA256 preimage of a random value

Proposition 1.3. *For* $\mathsf{Game\ SHAPreimg}^{\mathcal{A}}(\lambda)$ *as defined in Figure 1.3, let the advantage of* $\mathcal{A}$ *be defined as*

$$\mathsf{Adv}^{\mathrm{shaprei}}_{\mathcal{A}}(\lambda) \coloneqq \Pr\left[\mathsf{SHAPreimg}^{\mathcal{A}}(\lambda) = 1\right].$$

There exists a p.p.t. algorithm $\mathcal{A}$ *such that*

$$\mathsf{Adv}^{\mathrm{shaprei}}_{\mathcal{A}}(\lambda) = 1.$$

Proof. Let $\mathcal{A}$ be an algorithm that queries SHA256 until it finds a solution as shown in Figure 1.4. $\mathcal{A}$ will return the correct solution with probability 1. The running time of $\mathcal{A}$ is bounded by a

$\mathcal{A}(y)$

$x \coloneqq 0$
while $\mathsf{SHA256}(x) \neq y$ **do**
  $x \coloneqq x + 1$
endwhile
return x

Fig. 1.4. Algorithm for finding the SHA256 preimage of a given value

constant independent of the security parameter λ and, therefore, $\mathcal{A}$ is p.p.t. □

We observe that if the output space of the hash function is independent of the security parameter, then the hash function is not preimage-resistant.

1.4 Hash Functions

Definition 1.4 (Hash function). *A* hash function H *is a pair of polynomial-time algorithms* $(\mathsf{Gen}, \mathsf{Eval})$ *where:*

– $\mathsf{Gen}(1^\lambda) \to \kappa$ *is a probabilistic algorithm that takes the security parameter as input and outputs a hashing key* κ[1].
– $\mathsf{Eval}(\kappa, x) \to y$ *is a deterministic algorithm that takes a hashing key* κ *and an input* $x \in \{0,1\}^*$ *as input and outputs a hash value* $y \in S$. *When not specified otherwise, we assume* $S = \{0,1\}^\lambda$.[2]

Note that the hashing key κ is not secret because it is required to evaluate the hash function.

Remark 1.1 (Encoding of Arbitrary Inputs). We extend the notation to arbitrary inputs x (such as group elements, integers, or tuples) and simply write $\mathsf{H.Eval}(\kappa, x)$, implicitly assuming the existence of a fixed injective encoding function that maps the input x into $\{0,1\}^*$. We omit explicit reference to the encoding function in our notation.

$\mathsf{Game}\ \mathsf{Preimage}^{\mathcal{A}}_{\mathsf{H}}(\lambda)$

$\kappa \coloneqq \mathsf{H.Gen}(\lambda)$
$x \xleftarrow{\$} \{0,1\}^\lambda$
$y \coloneqq \mathsf{H.Eval}(\kappa, x)$
$x' \coloneqq \mathcal{A}(\kappa, y)$
return $\mathsf{H.Eval}(\kappa, x') = y$

Fig. 1.5. Game for finding the preimage of a given value under the hash function

Similar to the algorithm in Figure 1.4, there exists an algorithm that wins $\mathsf{Game}\ \mathsf{Preimage}^{\mathcal{A}}_{\mathsf{H}}(\lambda)$ with probability 1 by just evaluating $\mathsf{H.Eval}$ on fresh inputs until $\mathsf{H.Eval}(x) = y$. However, for adequate hash functions this algorithm runs in time exponential in the security parameter. Hence, the definition of preimage-resistance only considers p.p.t. adversaries.

Definition 1.5 (Preimage-resistance). *A hash function* H *is* preimage-resistant *if for any p.p.t. algorithm* $\mathcal{A}$,

$$\mathsf{Adv}^{\mathrm{prei}}_{\mathcal{A},\mathsf{H}}(\lambda) \coloneqq \Pr\left[\mathsf{Preimage}^{\mathcal{A}}_{\mathsf{H}}(\lambda) = 1\right] = \mathit{negl}(\lambda).$$

Remark 1.2 (Asymptotic Security Framework). Definition 1.5 is an example of a definition in the asymptotic security framework. The hash function H consists of algorithms that are polynomial-time in λ, and preimage resistance requires that any p.p.t. adversary's success probability is negligible in λ. Hence, by increasing λ we can make the success probability of any p.p.t. adversary arbitrarily small.

1.5 Exercises

Exercise 1.1. Determine which of the following functions are negligible in λ:

1. $f(\lambda) = 0$
2. $f(\lambda) = \frac{1}{2}$
3. $f(\lambda) = \frac{1}{\lambda^2}$
4. $f(\lambda) = \lambda^{4096} \cdot 2^{-\lambda}$
5. $f(\lambda) = \sqrt{2^{-\lambda}}$

Solution. We analyze each function using the definition of negligible functions and Lemma 1.1:

1. $f(\lambda) = 0$ is **negligible**. For any constant $c > 0$, we have $0 < \lambda^{-c}$ for all $\lambda > 0$.

[1] We assume κ includes 1^λ so that Eval can run in time polynomial in λ
[2] For notational simplicity, $\mathsf{H.Eval}(\kappa, x)$ is often written as $\mathsf{H}(x)$, leaving the hashing key implicit.

2. $f(\lambda) = \frac{1}{2}$ is **not negligible**. By Lemma 1.1(3), since 0 is negligible and $\frac{1}{2}$ is a positive constant, $0 + \frac{1}{2} = \frac{1}{2}$ is not negligible.
3. $f(\lambda) = \frac{1}{\lambda^2}$ is **not negligible**. For $c = 3$, we have $\frac{1}{\lambda^2} > \lambda^{-3}$ for all $\lambda > 1$, so the definition is not satisfied.
4. $f(\lambda) = \lambda^{4096} \cdot 2^{-\lambda}$ is **negligible**. First, $2^{-\lambda}$ is negligible: for any $c > 0$, we need $2^{-\lambda} < \lambda^{-c}$, which is equivalent to $\lambda^c < 2^\lambda$. Taking logarithms: $c\ln(\lambda) < \lambda\ln(2)$. We have $\lim_{\lambda\to\infty} \frac{c\ln(\lambda)}{\lambda\ln(2)} = \frac{c}{\ln(2)} \cdot \lim_{\lambda\to\infty} \frac{\ln(\lambda)}{\lambda}$ (constant multiple rule). Since $\lim_{\lambda\to\infty} \frac{\ln(\lambda)}{\lambda} = 0$ (by L'Hôpital's rule: $\lim_{\lambda\to\infty} \frac{1/\lambda}{1} = 0$), we get $\frac{c}{\ln(2)} \cdot 0 = 0$. Therefore, there exists N such that the inequality holds for all $\lambda > N$. By Lemma 1.1(1), the product of a negligible function with a polynomial is negligible.
5. $f(\lambda) = \sqrt{2^{-\lambda}}$ is **negligible**. Since $2^{-\lambda}$ is negligible, for any $c > 0$ there exists N such that for all $\lambda > N$, $2^{-\lambda} < \lambda^{-2c}$. Taking square roots preserves the inequality, so for all $\lambda > N$, $\sqrt{2^{-\lambda}} < \sqrt{\lambda^{-2c}} = \lambda^{-c}$. Hence $f(\lambda)$ is negligible.

□

Exercise 1.2. Consider the $\mathsf{Game\ Guess2}^{\mathcal{A}}(\lambda)$ which is identical to $\mathsf{Game\ Guess}^{\mathcal{A}}(\lambda)$ except that x is sampled *after* $\mathcal{A}$ is run. That is, the game first runs $x' \coloneqq \mathcal{A}()$, then samples $x \xleftarrow{\$} \mathbb{Z}_n$, and returns 1 if $x = x'$.

1. Are these two games equivalent? That is, does $\mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) = \mathsf{Adv}^{\mathrm{guess2}}_{\mathcal{A}}(\lambda)$ for all algorithms $\mathcal{A}$?
2. Does this change affect the proof of the proposition above?

This exercise illustrates an important principle in game-based proofs: the order of operations does not necessarily affect game outcomes when the operations are independent.

Solution. 1. Yes, the games are equivalent. Since $\mathcal{A}$ receives no input and x is sampled independently of $\mathcal{A}$'s execution, the order doesn't matter. In both cases, $\mathcal{A}$ outputs some fixed value x' (determined by its internal randomness), and then we check whether $x = x'$ for a uniformly random x. The probability is $2^{-\lambda}$ in both cases.
2. No, the proof remains the same. The key insight is that $\mathcal{A}$'s output x' is fixed before we consider the probability over the random choice of x. Whether x is sampled before or after running $\mathcal{A}$ doesn't affect this probability since $\mathcal{A}$ has no information about x.

This illustrates that when operations are independent (here, $\mathcal{A}$'s execution and the sampling of x), their order doesn't affect the outcome, which is a fundamental principle in game-based proofs. □

Exercise 1.3. Why is the definition of $\mathsf{Adv}^{\mathrm{shapreiz}}_{\mathcal{A}}(\lambda)$ in Proposition 1.2 problematic from a cryptographic perspective? What fundamental issue does it illustrate about concrete hash functions like SHA256?

Solution. The definition is problematic because it doesn't depend on the security parameter λ in a meaningful way. The adversary $\mathcal{A}$ that returns a hardcoded preimage of 0 (if one exists) always succeeds with probability 1, regardless of λ. This illustrates that we cannot achieve asymptotic security for concrete, fixed hash functions like SHA256: their output size doesn't grow with the security parameter. This is why we define abstract hash function families where the output length grows with λ. □

Exercise 1.4. Why is the definition of $\mathsf{Adv}^{\mathrm{shaprei}}_{\mathcal{A}}(\lambda)$ following Proposition 1.2 also problematic? Consider the adversary described in the proof.

Solution. The adversary's running time is indeed polynomial in λ: it's a constant independent of λ, which is $O(1)$ and therefore polynomial. However, this constant is potentially 2^{256}, which is astronomically large. The problem is that the output size (256 bits) doesn't grow with λ, so even exhaustive search becomes "polynomial time" in λ. This shows that asymptotic security only makes sense when the problem size grows with the security parameter. □

Exercise 1.5. How does the definition of preimage resistance for hash functions (Definition 1.5) avoid the issues encountered with the SHA256 examples?

Solution. The definition avoids the issues by considering hash function families rather than fixed functions like SHA256:

1. The definition uses a hash function family where the output size can grow with λ, ensuring that exhaustive search takes time exponential in λ rather than constant time.
2. The hash function is indexed by a key κ selected by $\mathsf{Gen}(1^\lambda)$, preventing hardcoding of specific preimages since the adversary doesn't know which function from the family will be used.

These changes allow the adversary's success probability to meaningfully depend on λ and be made negligible by increasing it. Note that this doesn't make SHA256 itself secure in the asymptotic sense - it remains a fixed function with constant output size. □

Exercise 1.6 (Optional). Prove property (1) of Lemma 1.1: If f is a negligible function and p is a polynomial, then $f(\lambda) \cdot p(\lambda)$ is negligible.

Hints:

- Any polynomial $p(\lambda)$ can be bounded by λ^a for some positive integer a and sufficiently large λ.
- To show $f(\lambda) \cdot p(\lambda)$ is negligible, for any $c > 0$, use the fact that f is negligible with constant $c + a$.

Solution. We need to show that $f(\lambda) \cdot p(\lambda)$ is negligible, i.e., for any $c > 0$, we have $f(\lambda) \cdot p(\lambda) < \lambda^{-c}$ for sufficiently large λ.

First, since p is a polynomial, there exists a positive integer a and $N_p \in \mathbb{N}$ such that $p(\lambda) < \lambda^a$ for all $\lambda > N_p$.

Let $c > 0$ be arbitrary. Since f is negligible, for the constant $c + a > 0$, there exists $N_f \in \mathbb{N}$ such that:

$$f(\lambda) < \lambda^{-(c+a)} \quad \text{for all } \lambda > N_f$$

Let $N = \max(N_p, N_f)$. Then for all $\lambda > N$:

$$f(\lambda) \cdot p(\lambda) < \lambda^{-(c+a)} \cdot \lambda^a = \lambda^{-c}$$

Since $c > 0$ was arbitrary, this proves that $f(\lambda) \cdot p(\lambda)$ is negligible. □

Exercise 1.7 (Optional). An alternative to the asymptotic approach is the *concrete security* approach. In this approach, we define (t, ϵ)-hardness as follows: The $\mathsf{Game\ Guess}^{\mathcal{A}}(\lambda)$ as defined in Figure 1.1 is (t, ϵ)-hard if for any algorithm $\mathcal{A}$ running in time t, we have $\Pr\left[\mathsf{Game\ Guess}^{\mathcal{A}}(\lambda) = 1\right] \leq \epsilon$.

Give a proposition about the concrete hardness of the guessing game, analogous to Proposition 1.1.

Solution. **Proposition:** For any t and λ, the $\mathsf{Game\ Guess}^{\mathcal{A}}(\lambda)$ is $(t, 2^{-\lambda})$-hard.

Proof: The proof is identical to Proposition 1.1, but we emphasize that the bound $2^{-\lambda}$ holds for *all* t, not just polynomial time.

Note: This shows that in the concrete setting, we explicitly state the security level ($2^{-\lambda}$) rather than saying it's "negligible". The concrete approach gives precise bounds rather than asymptotic statements. □

Exercise 1.8 (Optional). This question explores problems when defining security for hash functions. Read Section 1 and Section 4 of "Formalizing Human Ignorance" [Rog06].

1. What is the "Foundations-of-Hashing dilemma"?
2. To get around this, what alternative approach to defining security is suggested? What does "explicitly given" mean in this context?
3. (Optional) See also the discussion in Appendix B.7 and the proposed approaches in Appendix B.5 of [BL13].

Solution. 1. The Foundations-of-Hashing dilemma: For any fixed, unkeyed hash function (like SHA256), there always exists an adversary that simply prints a collision. This adversary exists mathematically even if we don't know how to construct it. This makes it impossible to prove collision resistance in the standard model.

2. The alternative approach requires "explicitly given" reductions. This means the security proof must provide an explicit, constructive reduction that converts a collision-finding adversary into a solution to some other hard problem. The adversary that prints SHA256 collisions is not "explicitly given" because we cannot actually write down its code (we don't know any collisions). □

2 Reductions

We use *reductions* to relate the hardness of winning various games. To show that problem Y is at least as hard as problem X, we construct a reduction: a p.p.t. algorithm that solves X using any algorithm that solves Y as a subroutine. This reduces problem X to Y: if there were an efficient algorithm that solves Y, we would also have an efficient algorithm that solves X. By contrapositive, if X is hard, then Y must also be hard.

We illustrate the concept of reductions through a series of examples.

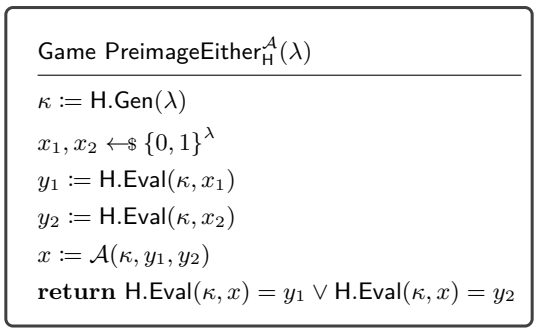

Fig. 2.1. Game for finding the preimage of either of two given values under the hash function

Definition 2.1. *For* Game $\mathsf{PreimageEither}^{\mathcal{A}}_{\mathsf{H}}(\lambda)$ *as defined in Figure 2.1 we define the advantage of* $\mathcal{A}$ *as*

$$\mathsf{Adv}^{\mathrm{preie}}_{\mathcal{A},\mathsf{H}}(\lambda) \coloneqq \Pr\left[\mathsf{PreimageEither}^{\mathcal{A}}_{\mathsf{H}}(\lambda) = 1\right].$$

Proposition 2.1. *Let* H *be a hash function. If for all p.p.t. adversaries* $\mathcal{A}$*, it holds that*

$$\mathsf{Adv}^{\mathrm{preie}}_{\mathcal{A},\mathsf{H}}(\lambda) = \mathit{negl}(\lambda)$$

then H *is preimage-resistant.*

Proof. We prove the contrapositive statement (which is equivalent to the statement in the proposition): If H is not preimage-resistant, then there exists a p.p.t. algorithm $\mathcal{B}$ that wins Game $\mathsf{PreimageEither}^{\mathcal{B}}_{\mathsf{H}}(\lambda)$ with non-negligible probability.

If H is not preimage-resistant, then there exists a p.p.t. algorithm $\mathcal{A}$ that wins Game $\mathsf{Preimage}^{\mathcal{A}}_{\mathsf{H}}(\lambda)$ with non-negligible probability. Let $\mathcal{B}^{\mathcal{A}}_{\mathsf{H}}$ be the algorithm defined in Figure 2.2. It gets both challenges y_1, y_2, runs $\mathcal{A}$ on y_1 and returns its output.

The success probability of $\mathcal{B}$ is at least that of $\mathcal{A}$. When $\mathcal{A}$ successfully finds a preimage of y_1, then $\mathcal{B}$ succeeds in Game PreimageEither. Additionally, there is a non-zero probability that $\mathsf{H.Eval}(\kappa, x) = y_2$ even when $\mathsf{H.Eval}(\kappa, x) \neq y_1$. Therefore, $\mathsf{Adv}^{\mathrm{preie}}_{\mathcal{B}^{\mathcal{A}}_{\mathsf{H}},\mathsf{H}}(\lambda) \geq \mathsf{Adv}^{\mathrm{prei}}_{\mathcal{A},\mathsf{H}}(\lambda)$. Since $\mathsf{Adv}^{\mathrm{prei}}_{\mathcal{A},\mathsf{H}}(\lambda)$ is non-negligible and $\mathcal{B}^{\mathcal{A}}_{\mathsf{H}}$ is p.p.t., we have found a p.p.t. algorithm with non-negligible advantage for Game PreimageEither. □

Proposition 2.2. *Let* H *be a preimage-resistant hash function. Then* $\mathsf{Adv}^{\mathrm{preie}}_{\mathcal{A},\mathsf{H}}(\lambda)$ *is negligible for all p.p.t. adversaries* $\mathcal{A}$*. More precisely, for any p.p.t. adversary* $\mathcal{A}$ *against* Game PreimageEither *of* H*, there exists a p.p.t. adversary* $\mathcal{B}$ *against the preimage-resistance of* H *such that*

$$\mathsf{Adv}^{\mathrm{preie}}_{\mathcal{A},\mathsf{H}}(\lambda) \leq 2\mathsf{Adv}^{\mathrm{prei}}_{\mathcal{B}^{\mathcal{A}}_{\mathsf{H}},\mathsf{H}}(\lambda).$$

The proof is left as an exercise (see Exercise 2.1).

$\underline{\mathcal{B}_{\mathsf{H}}^{\mathcal{A}}(\kappa, y_1, y_2)}$
$x \coloneqq \mathcal{A}(\kappa, y_1)$
return x

Fig. 2.2. Algorithm for finding the preimage of either of two given values under the hash function

2.1 Collision Resistance

$\underline{\mathsf{Game}\ \mathsf{Collision}_{\mathsf{H}}^{\mathcal{A}}(\lambda)}$
$\kappa \coloneqq \mathsf{H.Gen}(\lambda)$
$(x, x') \coloneqq \mathcal{A}(\kappa)$
return $(x \neq x' \wedge \mathsf{H.Eval}(\kappa, x) = \mathsf{H.Eval}(\kappa, x'))$

Fig. 2.3. Game for finding a collision under the hash function

Definition 2.2 (Collision-resistance). *Hash function* H *is collision-resistant if for any p.p.t. algorithm* $\mathcal{A}$*,*

$$\mathsf{Adv}^{\mathrm{coll}}_{\mathcal{A},\mathsf{H}}(\lambda) \coloneqq \Pr\left[\mathsf{Collision}^{\mathcal{A}}_{\mathsf{H}}(\lambda) = 1\right] = \mathit{negl}(\lambda).$$

Theorem 2.1 (Collision-resistance implies preimage-resistance). *Let* H *be a collision-resistant hash function. Then* H *is preimage-resistant. More precisely, for any p.p.t. adversary* $\mathcal{A}$ *against* Game Preimage *of* H*, there exists a p.p.t. adversary* $\mathcal{B}$ *against* Game Collision *of* H *such that*

$$\mathsf{Adv}^{\mathrm{prei}}_{\mathcal{A},\mathsf{H}}(\lambda) \leq \mathsf{Adv}^{\mathrm{coll}}_{\mathcal{B}^{\mathcal{A}}_{\mathsf{H}},\mathsf{H}}(\lambda) + 2^{-\lambda}.$$

The proof is left as an exercise (see Exercise 2.2).

2.2 Exercises

Exercise 2.1. Prove Proposition 2.2.

Hint: Construct a reduction that randomly places the challenge in either the first or second position.

Solution. We prove the contrapositive statement: If there exists a p.p.t. algorithm $\mathcal{A}$ that wins $\mathsf{Game}\ \mathsf{PreimageEither}^{\mathcal{A}}_{\mathsf{H}}(\lambda)$ with non-negligible probability, then H is not preimage-resistant. Let $\mathcal{B}^{\mathcal{A}}_{\mathsf{H}}$ be an algorithm that runs $\mathcal{A}$ and returns its output:

$\underline{\mathcal{B}_{\mathsf{H}}^{\mathcal{A}}(\kappa, y)}$
$x_2 \leftarrow\!\!\$\ \{0,1\}^{\lambda}$
$y_2 \coloneqq \mathsf{H.Eval}(\kappa, x_2)$
$b \leftarrow\!\!\$\ \{0,1\}$
if $b = 0$ **then**
  $x \coloneqq \mathcal{A}(\kappa, y, y_2)$
else
  $x \coloneqq \mathcal{A}(\kappa, y_2, y)$
return x

Let E_i denote the event that $\mathcal{A}$ outputs a preimage of its i-th input ($i \in \{1,2\}$). By definition, $\mathsf{Adv}^{\mathrm{preie}}_{\mathcal{A},\mathsf{H}}(\lambda) = \Pr[E_1 \vee E_2] \leq \Pr[E_1] + \Pr[E_2]$ (union bound; equality holds when $y_1 \neq y_2$).

Since x_1 and x_2 are chosen uniformly and independently in the PreimageEither game (and thus y_1 and y_2 are independent samples from the image of H), by symmetry: $\Pr[E_1] = \Pr[E_2]$.

Therefore: $\mathsf{Adv}^{\mathrm{preie}}_{\mathcal{A},\mathsf{H}}(\lambda) \leq 2 \cdot \Pr[E_1]$.

Now consider $\mathcal{B}$'s success probability. With probability $\frac{1}{2}$, it places the challenge y in position 1 (calling $\mathcal{A}(\kappa, y, y_2)$), and with probability $\frac{1}{2}$, it places y in position 2 (calling $\mathcal{A}(\kappa, y_2, y)$). Since in both games y comes from $\mathsf{H.Eval}(\kappa, x)$ for uniform x, and y_2 comes from $\mathsf{H.Eval}(\kappa, x_2)$ for uniform x_2, $\mathcal{B}$ perfectly simulates the PreimageEither game distribution. Thus $\mathcal{B}$ succeeds with probability: $\mathsf{Adv}^{\mathrm{prei}}_{\mathcal{B}^{\mathcal{A}}_{\mathsf{H}},\mathsf{H}}(\lambda) = \frac{1}{2} \cdot \Pr[E_1] + \frac{1}{2} \cdot \Pr[E_2] = \Pr[E_1] \geq \frac{1}{2} \cdot \mathsf{Adv}^{\mathrm{preie}}_{\mathcal{A},\mathsf{H}}(\lambda)$.

Since $\mathsf{Adv}^{\mathrm{preie}}_{\mathcal{A},\mathsf{H}}(\lambda)$ is non-negligible and $\mathcal{B}^{\mathcal{A}}_{\mathsf{H}}$ is p.p.t., H is not preimage-resistant. □

Exercise 2.2. Prove Theorem 2.1 (collision-resistance implies preimage-resistance).

Hint: Construct a reduction that samples a random x, computes $y = \mathsf{H.Eval}(\kappa, x)$, and uses the preimage-finding adversary to find x' such that $\mathsf{H.Eval}(\kappa, x') = y$. What is the probability that $x = x'$?

Solution. We prove the contrapositive statement: If H is not preimage-resistant, then there exists a p.p.t. algorithm $\mathcal{A}$ that wins $\mathsf{Game\ Preimage}^{\mathcal{A}}_{\mathsf{H}}(\lambda)$ with non-negligible probability. Let $\mathcal{B}^{\mathcal{A}}_{\mathsf{H}}$ be an algorithm that runs $\mathcal{A}$ and returns its output:

$\mathcal{B}^{\mathcal{A}}_{\mathsf{H}}(\kappa)$

1 : $x \leftarrow\!\$\ \{0,1\}^{2\lambda}$
2 : $y \coloneqq \mathsf{H.Eval}(\kappa, x)$
3 : $x' \coloneqq \mathcal{A}(\kappa, y)$
4 : **assert** $x \neq x'$
5 : **return** (x, x')

If $\mathcal{A}$ succeeds and $\mathcal{B}$ does not abort in line 4, then $\mathcal{B}$ wins game $\mathrm{Collision}^{\mathcal{A}}_{\mathsf{H}}(\lambda)$, or more precisely:

$$\begin{aligned}\Pr\left[\mathrm{Collision}^{\mathcal{A}}_{\mathsf{H}}(\lambda) = 0\right] &= \Pr\left[\mathrm{Preimage}^{\mathcal{A}}_{\mathsf{H}}(\lambda) = 0 \vee \mathcal{B} \text{ aborts at line 4}\right] \\ &\leq \Pr\left[\mathrm{Preimage}^{\mathcal{A}}_{\mathsf{H}}(\lambda) = 0\right] + \Pr\left[\mathcal{B} \text{ aborts at line 4}\right]\end{aligned}$$

using union bound. Then, by the definition of $\mathsf{Adv}^{\mathrm{coll}}_{\mathcal{B},\mathsf{H}}(\lambda)$ and $\mathsf{Adv}^{\mathrm{prei}}_{\mathcal{A},\mathsf{H}}(\lambda)$ we have

$$1 - \mathsf{Adv}^{\mathrm{coll}}_{\mathcal{B},\mathsf{H}}(\lambda) \leq 1 - \mathsf{Adv}^{\mathrm{prei}}_{\mathcal{A},\mathsf{H}}(\lambda) + \Pr\left[\mathcal{B} \text{ aborts at line 4}\right]$$

and

$$\mathsf{Adv}^{\mathrm{coll}}_{\mathcal{B},\mathsf{H}}(\lambda) \geq \mathsf{Adv}^{\mathrm{prei}}_{\mathcal{A},\mathsf{H}}(\lambda) - \Pr\left[\mathcal{B} \text{ aborts at line 4}\right].$$

We now show that $\Pr[\mathcal{B} \text{ aborts at line 4}] = \mathit{negl}(\lambda)$. Let us denote this event by A. Let B_y denote the event that $\mathsf{H.Eval}(\kappa, x) = y$ and $\operatorname{Im}\mathsf{H.Eval}(\kappa, \cdot)$ be the image of $\mathsf{H.Eval}$ for a fixed κ. Then, by the law of total probability and the definition of conditional probability we have

$$\begin{aligned}\Pr[A] &= \sum_{y \in \operatorname{Im}\mathsf{H.Eval}(\kappa,\cdot)} \Pr[A \wedge B_y] \\ &= \sum_{y \in \operatorname{Im}\mathsf{H.Eval}(\kappa,\cdot)} \Pr[B_y] \Pr[A \mid B_y]\end{aligned}$$

Let $H^{-1}(y) = \{x \in \{0,1\}^{2\lambda} : H(x) = y\}$, i.e., the preimage of y. Since x is uniformly random from a set of size $2^{2\lambda}$, the probability $\Pr[B_y]$ that $\mathsf{H.Eval}(\kappa, x) = y$ is $\frac{|\mathsf{H}^{-1}(y)|}{2^{2\lambda}}$. Also, the probability $\Pr[A \mid B_y]$ that the sampled value x matches the adversary's answer x' given that $\mathsf{H.Eval}(\kappa, x) = y$ is $\frac{1}{|\mathsf{H}^{-1}(y)|}$. Therefore, we have

$$\begin{aligned}
\Pr[A] &= \sum_{y \in \operatorname{Im} \mathsf{H.Eval}(\kappa,\cdot)} \frac{|\mathsf{H}^{-1}(y)|}{2^{2\lambda}} \frac{1}{|\mathsf{H}^{-1}(y)|} \\
&= \sum_{y \in \operatorname{Im} \mathsf{H.Eval}(\kappa,\cdot)} \frac{1}{2^{2\lambda}} \\
&= \frac{|\operatorname{Im} \mathsf{H.Eval}(\kappa,\cdot)|}{2^{2\lambda}} \\
&\leq \frac{2^{\lambda}}{2^{2\lambda}} = 2^{-\lambda}
\end{aligned}$$

which is negligible.

Since $\mathsf{Adv}^{\mathrm{prei}}_{\mathcal{A},\mathsf{H}}(\lambda)$ is not negligible and $\Pr[\mathcal{B} \text{ aborts}] = negl(\lambda)$, by lemma 1.1, $\mathsf{Adv}^{\mathrm{coll}}_{\mathcal{B},\mathsf{H}}(\lambda)$ is not negligible. Since $\mathcal{B}$ is p.p.t., H is not collision-resistant. □

Exercise 2.3 (Optional). Describe target collision resistance, extended target collision resistance, and multi-target collision resistance [HRS16] in your own words and discuss their security against classical and quantum attacks (see Table 1 in the referenced paper).

Exercise 2.4 (Optional). Drijvers et al. [Dri+19] give a *metareduction* showing that MuSig(1) [Max+19] without the first nonce-commitment round cannot be proven secure under the one-more discrete logarithm (OMDL) assumption. Interpret Theorem 1 and Figure 2 in their paper. Complete the following informal statement: If there exists an algorithm $\mathcal{B}$ that reduces n-OMDL to the EUF-CMA security of the MuSig(1) variant, then there exists a reduction $\mathcal{M}$ and a forger $\mathcal{F}$ that solve the _____ problem.

Solution. $(n+k)$-OMDL □

3 Commitments

3.1 Syntax

Definition 3.1 (Commitment Scheme). *A* commitment scheme Com *is a pair of polynomial-time algorithms* $(\mathsf{Setup}, \mathsf{Commit})$ *where:*

- $\mathsf{Setup}(1^\lambda) \to pp$[3] *is a probabilistic algorithm that takes the security parameter as input and outputs public parameters* pp *including the message space* $\mathcal{M}$ *and the randomness space* $\mathcal{R}$*.*
- $\mathsf{Commit}(pp, m, r) \to C$[4] *is a deterministic algorithm that takes public parameters* pp*, a message* $m \in \mathcal{M}$*, and randomness* $r \in \mathcal{R}$ *as input and outputs a commitment* C*.*

Example 2. An example is the toy commitment scheme:

- $\mathsf{Setup}(1^\lambda) := (\mathcal{M} := \{0,1\}^*, \mathcal{R} := \{0\})$
- $\mathsf{Commit}(pp, m, r) := m$

3.2 Security

Definition 3.2 (Binding). *A commitment scheme* Com *is binding if for any p.p.t. algorithm* $\mathcal{A}$*,*

$$\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A},\mathsf{Com}}(\lambda) := \Pr\left[\mathsf{ComBind}^{\mathcal{A}}_{\mathsf{Com}}(\lambda) = 1\right] = negl(\lambda).$$

If $\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A},\mathsf{Com}}(\lambda) = 0$*, then the commitment scheme is said to be* perfectly binding.

$\mathsf{Game}\ \mathsf{ComBind}^{\mathcal{A}}_{\mathsf{Com}}(\lambda)$

$pp \leftarrow\!\$\ \mathsf{Setup}(1^\lambda)$
$(m_0, m_1, r_0, r_1) \coloneqq \mathcal{A}(pp)$
$C_0 \coloneqq \mathsf{Commit}(pp, m_0, r_0)$
$C_1 \coloneqq \mathsf{Commit}(pp, m_1, r_1)$
return $(C_0 = C_1) \wedge (m_0 \neq m_1)$

Fig. 3.1. Binding game

$\mathsf{Game}\ \mathsf{ComHide}^{\mathcal{A}}_{\mathsf{Com}}(\lambda)$

$pp \leftarrow\!\$\ \mathsf{Setup}(1^\lambda)$
$(m_0, m_1) \coloneqq \mathcal{A}(pp)$
$r \leftarrow\!\$\ pp.\mathcal{R}$
$b \leftarrow\!\$\ \{0,1\}$
$C \coloneqq \mathsf{Commit}(pp, m_b, r)$
$b' \coloneqq \mathcal{A}(pp, C)$
return $(b = b')$

Fig. 3.2. Hiding game

Definition 3.3 (Hiding). *A commitment scheme* Com *is hiding if for any p.p.t. algorithm* $\mathcal{A}$*,*

$$\mathsf{Adv}^{\mathrm{hide}}_{\mathcal{A},\mathsf{Com}}(\lambda) \coloneqq |\Pr\left[\mathsf{ComHide}^{\mathcal{A}}_{\mathsf{Com}}(\lambda) = 1\right] - \frac{1}{2}| = negl(\lambda).$$

If $\mathsf{Adv}^{\mathrm{hide}}_{\mathcal{A},\mathsf{Com}}(\lambda) = 0$*, then the commitment scheme is said to be* perfectly hiding.

Note that in $\mathsf{ComHide}^{\mathcal{A}}_{\mathsf{Com}}(\lambda)$ the adversary $\mathcal{A}$ is run twice with different arguments, first to output two messages and then to output a bit.

3.3 Exercises

Exercise 3.1. Is the toy commitment scheme in Example 2 binding? Prove it.

Solution. Yes, the toy commitment scheme is perfectly binding.

Proof by contradiction: Assume the binding game succeeds, i.e., $\mathsf{ComBind}^{\mathcal{A}}_{\mathsf{Com}}(\lambda) = 1$. This means:

- $C_0 = C_1$ (the commitments are equal)
- $m_0 \neq m_1$ (the messages are different)

However, by the definition of the toy commitment scheme:

- $C_0 = \mathsf{Commit}(pp, m_0, r_0) = m_0$
- $C_1 = \mathsf{Commit}(pp, m_1, r_1) = m_1$

[3] The security parameter 1^λ is often left implicit in algorithm inputs to simplify notation, e.g., writing $\mathsf{Setup}()$ instead of $\mathsf{Setup}(1^\lambda)$.

[4] Public parameters pp are often left implicit in algorithm inputs to simplify notation, e.g., writing $\mathsf{Commit}(m, r)$ instead of $\mathsf{Commit}(pp, m, r)$.

Therefore, $C_0 = C_1$ implies $m_0 = m_1$, which contradicts $m_0 \neq m_1$.

Since no adversary can win the binding game, we have $\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A},\mathsf{Com}}(\lambda) = 0$ for all adversaries $\mathcal{A}$. The toy commitment scheme is therefore perfectly binding. □

Exercise 3.2. A commitment scheme is *strongly binding* if it binds to *both* the message and randomness. Formally define the strong binding game $\mathsf{ComStrongBind}^{\mathcal{A}}_{\mathsf{Com}}(\lambda)$ by modifying $\mathsf{ComBind}^{\mathcal{A}}_{\mathsf{Com}}(\lambda)$.

Solution. The strong binding game $\mathsf{ComStrongBind}^{\mathcal{A}}_{\mathsf{Com}}(\lambda)$ is defined as follows:

Game $\mathsf{ComStrongBind}^{\mathcal{A}}_{\mathsf{Com}}(\lambda)$

$pp \leftarrow\!\$\ \mathsf{Setup}(1^\lambda)$
$(m_0, m_1, r_0, r_1) \coloneqq \mathcal{A}(pp)$
$C_0 \coloneqq \mathsf{Commit}(pp, m_0, r_0)$
$C_1 \coloneqq \mathsf{Commit}(pp, m_1, r_1)$
return $(C_0 = C_1) \wedge ((m_0 \neq m_1) \vee (r_0 \neq r_1))$

The key difference from the regular binding game is in the return condition:

- Regular binding: $(C_0 = C_1) \wedge (m_0 \neq m_1)$
- Strong binding: $(C_0 = C_1) \wedge ((m_0 \neq m_1) \vee (r_0 \neq r_1))$

Strong binding prevents an adversary from finding the same commitment with either different messages OR different randomness. □

Exercise 3.3 (Optional). Describe a scenario where strong binding is important.

Solution. Strong binding is crucial when the randomness itself has semantic meaning in the protocol.

Consider the Taproot commitment scheme where the randomness space $\mathcal{R}$ is a group $\mathbb{G}$. For hash function H and group generator g, the commitment is computed as:

$$\mathsf{Commit}(m, R) = R \cdot g^{\mathsf{H.Eval}(\kappa,(R,m))}$$

In this scheme, m represents the root of a Merkle tree containing Tapscripts, and R serves as an internal key. Assume that the Tapscript language includes an opcode OP_INTERNALKEY that pushes R onto the execution stack.

Without strong binding, an adversary could potentially find an alternative opening (R', m) where $R' \neq R$ but still produces the same commitment C. This would allow the adversary to execute scripts with OP_INTERNALKEY pushing their chosen R' onto the stack instead of the original R.

Strong binding prevents this attack by ensuring that each commitment C has a unique valid opening, making it impossible for an adversary to substitute a different internal key while maintaining the same commitment. □

Exercise 3.4. Design a hash-based commitment scheme $\mathsf{HashCom}[\mathsf{H}]$[5] that is strongly binding when used with a collision-resistant hash function H. State and prove the security theorem.

Note: We do not yet have the tools to prove that $\mathsf{HashCom}[\mathsf{H}]$ is hiding; we will return to this in later chapters.

Solution. Define the hash commitment scheme $\mathsf{HashCom}[\mathsf{H}]$ as follows:

- $\mathsf{Setup}(1^\lambda) \coloneqq (\mathcal{M} \coloneqq \{0,1\}^*, \mathcal{R} \coloneqq \{0,1\}^\lambda, \kappa \coloneqq \mathsf{H.Gen}(1^\lambda))$
- $\mathsf{Commit}(pp, m, r) \coloneqq \mathsf{H.Eval}(\kappa, r \| m)$

[5] The notation $\mathsf{Scheme}[\mathsf{Primitive}]$ indicates that the scheme is parameterized by a cryptographic primitive.

Theorem: Let H be a collision-resistant hash function. Then $\mathsf{HashCom}[\mathsf{H}]$ is strongly binding. More precisely, for any p.p.t. adversary $\mathcal{A}$ against the strong binding property of $\mathsf{HashCom}[\mathsf{H}]$, there exists a p.p.t. adversary $\mathcal{B}$ against the collision resistance of H such that

$$\mathsf{Adv}^{\text{strongbind}}_{\mathcal{A},\mathsf{HashCom}[\mathsf{H}]}(\lambda) = \mathsf{Adv}^{\text{coll}}_{\mathcal{B}^{\mathcal{A}}_{\mathsf{H}},\mathsf{H}}(\lambda).$$

Proof: We prove the contrapositive: if $\mathsf{HashCom}[\mathsf{H}]$ is not strongly binding, then H is not collision-resistant.

Assume $\mathsf{HashCom}[\mathsf{H}]$ is not strongly binding. Then there exists a p.p.t. adversary $\mathcal{A}$ that wins $\mathsf{ComStrongBind}^{\mathcal{A}}_{\mathsf{HashCom}[\mathsf{H}]}(\lambda)$ with non-negligible probability, finding (m_0, r_0) and (m_1, r_1) such that:

- $\mathsf{H.Eval}(\kappa, r_0 \| m_0) = \mathsf{H.Eval}(\kappa, r_1 \| m_1)$
- Either $m_0 \neq m_1$ or $r_0 \neq r_1$

If either $m_0 \neq m_1$ or $r_0 \neq r_1$, then $r_0 \| m_0 \neq r_1 \| m_1$. This means $\mathcal{A}$ has found a collision in H: two different inputs that hash to the same output.

We can construct a p.p.t. adversary $\mathcal{B}$ against the collision resistance of H that runs $\mathcal{A}$ and outputs $(r_0 \| m_0, r_1 \| m_1)$. Since $\mathcal{A}$ succeeds with non-negligible probability and $\mathcal{B}$ is p.p.t. (as it simply runs $\mathcal{A}$ once), $\mathcal{B}$ breaks collision resistance.

Therefore, if H is collision-resistant, then $\mathsf{HashCom}[\mathsf{H}]$ is strongly binding. Note that strong binding implies regular binding, so this scheme is also binding. □

Exercise 3.5. Is the toy commitment scheme in Example 2 hiding? Prove it.

Solution. No, the toy commitment scheme is not hiding.

We construct a p.p.t. adversary $\mathcal{A}$ that wins the hiding game with probability 1:

1. $\mathcal{A}$ receives pp from the setup.
2. $\mathcal{A}$ outputs two distinct messages: $m_0 = 0$ and $m_1 = 1$.
3. The game samples $b \leftarrow\$ \{0,1\}$ and computes $C = \mathsf{Commit}(pp, m_b, r) = m_b$.
4. $\mathcal{A}$ receives C and outputs $b' = C$.
5. Since $C = m_b = b$ (because $m_0 = 0$ and $m_1 = 1$), we have $b' = b$.

Therefore, $\Pr\left[\mathsf{ComHide}^{\mathcal{A}}_{\mathsf{Com}}(\lambda) = 1\right] = 1$, and

$$\mathsf{Adv}^{\text{hide}}_{\mathcal{A},\mathsf{Com}}(\lambda) = |1 - \frac{1}{2}| = \frac{1}{2}$$

which is not negligible.

Intuitively, the toy commitment scheme completely reveals the message, providing no hiding at all. □

Exercise 3.6. Let H be a collision-resistant hash function and define the commitment scheme $\mathsf{HashCom}'[\mathsf{H}]$ as:

- $\mathsf{Setup}(1^\lambda) := (\mathcal{M} := \{0,1\}, \mathcal{R} := \{0\}, \kappa := \mathsf{H.Gen}(1^\lambda))$
- $\mathsf{Commit}(pp, m, r) := \mathsf{H.Eval}(\kappa, m)$

Note that $\mathsf{HashCom}'[\mathsf{H}]$ is binding. Is it hiding? Prove it.

Solution. No, $\mathsf{HashCom}'[\mathsf{H}]$ is not hiding. The issue is that there's no randomness in the commitment.

We construct a p.p.t. adversary $\mathcal{A}$ that wins the hiding game with high probability:

1. $\mathcal{A}$ receives pp containing κ.
2. $\mathcal{A}$ outputs $m_0 = 0$ and $m_1 = 1$.
3. The game samples $b \leftarrow\$ \{0,1\}$ and computes $C = \mathsf{H.Eval}(\kappa, m_b)$.
4. $\mathcal{A}$ receives C and computes:
   - $h_0 = \mathsf{H.Eval}(\kappa, 0)$
   - $h_1 = \mathsf{H.Eval}(\kappa, 1)$
5. If $C = h_0$, output $b' = 0$; if $C = h_1$, output $b' = 1$.

Since H is collision-resistant, with overwhelming probability $\mathsf{H.Eval}(\kappa, 0) \neq \mathsf{H.Eval}(\kappa, 1)$. Therefore, $\mathcal{A}$ correctly identifies b with overwhelming probability, making the scheme not hiding. □

4 Accumulators

4.1 Syntax & Correctness

An accumulator is a compact representation of a set of elements and allows certain operations on the set.

Definition 4.1 (Accumulator). *An accumulator that supports insertions of elements and membership proofs is also called an* additive positive accumulator *[BCY20]. It is a tuple of polynomial-time algorithms* $(\mathsf{Gen}, \mathsf{Init}, \mathsf{Value}, \mathsf{Insert}, \mathsf{ProveMembership}, \mathsf{VerifyMembership})$ *where:*

- $\mathsf{Gen}(1^\lambda) \to pp$ *is a probabilistic algorithm that takes the security parameter as input and outputs public parameters* pp *including the domain* $\mathcal{X}_{\mathsf{Acc}}$ *of elements that can be accumulated.*
- $\mathsf{Init}(pp) \to state$ *is a deterministic algorithm that takes public parameters* pp *as input and outputs the initial state,* $state$*, of the accumulator.*
- $\mathsf{Value}(pp, state) \to v$ *is a deterministic algorithm that takes public parameters* pp *and an accumulator state,* $state$*, as input and outputs the accumulator value* v*.*
- $\mathsf{Insert}(pp, state, x) \to state'$ *is a deterministic algorithm that takes public parameters* pp*, an accumulator state,* $state$*, and an element* $x \in \mathcal{X}_{\mathsf{Acc}}$ *as input and outputs a new state,* $state'$*, after inserting* x*.*
- $\mathsf{ProveMembership}(pp, state, x) \to \pi$ *is a probabilistic algorithm that takes public parameters* pp*, an accumulator state,* $state$*, and an element* x *as input and outputs a membership proof* π*.*
- $\mathsf{VerifyMembership}(pp, v, x, \pi) \to \{0, 1\}$ *is a deterministic algorithm that takes public parameters* pp*, an accumulator value* v*, an element* $x \in \mathcal{X}_{\mathsf{Acc}}$*, and a proof* π *as input and outputs* 1 *(accept) if the proof validates that* x *is a member of the set represented by* v*, and outputs* 0 *(reject) otherwise.*

Typically, an accumulator involves two parties: the manager, who uses Init, Value, Insert, $\mathsf{ProveMembership}$ and the verifier, who uses $\mathsf{VerifyMembership}$.

Example 3. An example is the trivial accumulator, where the accumulator value is simply the set itself and therefore grows linearly with the number of inserted elements. (In contrast, non-trivial accumulators typically have constant-size accumulator values.)

- $\mathsf{Gen}(1^\lambda) \to pp$ where $pp = (\mathcal{X}_{\mathsf{Acc}} = \{0,1\}^*)$
- $\mathsf{Init}(pp) \to \{\}$
- $\mathsf{Value}(pp, state) \to state$
- $\mathsf{Insert}(pp, state, x) \to state \cup \{x\}$
- $\mathsf{ProveMembership}(pp, state, x) \to \bot$
- $\mathsf{VerifyMembership}(pp, v, x, \pi) \to b$ outputs $b = 1$ if $x \in v$. Otherwise, it outputs $b = 0$.

Definition 4.2 (Correctness). *An additive positive accumulator is considered* correct *if, for any element that was inserted into the accumulator, the membership proof generated honestly for it is successfully verified. More formally, for all security parameters* λ*, all public parameters* $pp \coloneqq \mathsf{Gen}(1^\lambda)$*, all positive integers* t *polynomial in* λ*, all tuples* $\vec{y} \in \mathcal{X}_{\mathsf{Acc}}^t$*: for all* $x \in \vec{y}$*,*

$$\Pr\left[\begin{array}{l} state_0 \coloneqq \mathsf{Init}(pp); \\ state_i \coloneqq \mathsf{Insert}(pp, state_{i-1}, y_i) \text{ for } i \in [1,t]; \\ \pi \coloneqq \mathsf{ProveMembership}(pp, state_t, x); \\ v \coloneqq \mathsf{Value}(pp, state_t); \\ \mathsf{VerifyMembership}(pp, v, x, \pi) = 1 \end{array}\right] = 1 - negl(\lambda).$$

4.2 Security

In Game $\mathsf{AccColl}$ (defined in Figure 4.1), the adversary $\mathcal{A}$ is given access to *oracles*, which he can call, but not look inside. In other words, the adversary can insert elements into the accumulator and obtain membership proofs, but does not have access to the state directly.

Definition 4.3 (Collision-Freeness). *An additive positive accumulator is* collision-free *if it is hard to fabricate a membership proof for a value that is not in the set. More formally, an accumulator* Acc *is collision-free if for any p.p.t. algorithm* $\mathcal{A}$

$$\mathsf{Adv}^{\text{collision}}_{\mathcal{A},\mathsf{Acc}}(\lambda) \coloneqq \Pr\left[\mathsf{AccColl}^{\mathcal{A}}_{\mathsf{Acc}}(\lambda) = 1\right] = negl(\lambda).$$

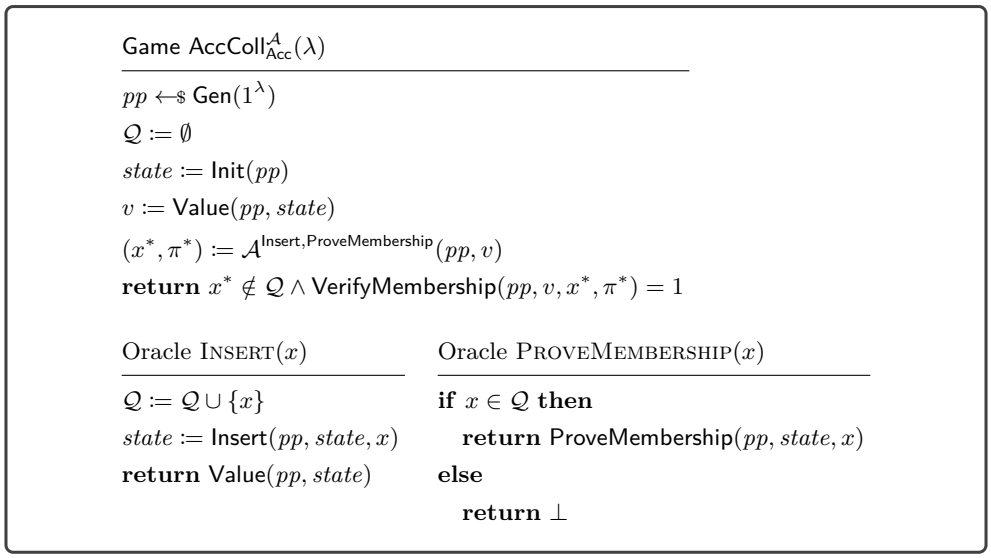

Fig. 4.1. Collision game

4.3 Exercises

Exercise 4.1. Is the trivial accumulator correct? Justify or prove it.

Solution. **Correctness (informal):** An accumulator is correct if for any element that has been inserted, we can generate a membership proof (using the current state) that will verify successfully against the current accumulator value.

Is the trivial accumulator correct? Yes, the trivial accumulator is correct. In the trivial accumulator:

- The accumulator value v is just the state itself, which is the set of all inserted elements
- $\mathsf{ProveMembership}$ returns $\perp$ (no proof needed)
- $\mathsf{VerifyMembership}$ simply checks if $x \in v$

Since the accumulator value is the set of all inserted elements, any element that was inserted will be in the set, so verification always succeeds for inserted elements. This satisfies correctness. □

Exercise 4.2. Is the trivial accumulator collision-free? Justify or prove it.

Solution. **Collision-freeness (informal):** An accumulator is collision-free if it's computationally infeasible for an adversary to produce a valid membership proof for an element that was never inserted into the accumulator.

Is the trivial accumulator collision-free? Yes, the trivial accumulator is collision-free.

In the trivial accumulator, the accumulator value v is exactly the set of all inserted elements. For $\mathsf{VerifyMembership}$ to accept, the element x must be in this set. Since the adversary cannot modify the accumulator value (it's computed honestly by the game), and the value contains exactly the elements that were inserted, it's impossible to create a proof for an element that wasn't inserted.

Therefore, $\mathsf{Adv}^{\mathrm{collision}}_{\mathcal{A},\mathsf{TrivialAcc}}(\lambda) = 0$ for all adversaries, making it perfectly collision-free. □

Exercise 4.3. Using a collision-resistant hash function, give an example of an additive positive accumulator with constant-size accumulator values. Argue that it is correct and collision-free.

Solution. An example is a **Merkle tree**. The accumulator value is the root hash of a Merkle tree containing all inserted elements. The membership proof for an element consists of the authentication path from the leaf to the root.

Correctness: If an element was inserted, it appears as a leaf in the tree. The honestly generated authentication path will correctly hash up to the root, so verification succeeds.

Collision-freeness: To forge a membership proof for an element that wasn't inserted, an adversary would need to provide an authentication path that hashes to the correct root. This would mean finding a different set of sibling hashes that produce the same root hash, which would constitute a collision in the hash function. Since the hash function is collision-resistant, this is infeasible. □

Exercise 4.4 (Optional). Using a collision-resistant hash function, give an example of an additive positive accumulator with constant-size accumulator values and logarithmic-size membership proofs. Argue that it is correct and collision-free.

Solution. A Merkle tree achieves logarithmic-size proofs. Elements are stored at the leaves of a balanced binary tree, and the accumulator value is the root hash. The membership proof consists of the sibling hashes along the path from leaf to root, which has length $O(\log n)$ for n elements.

Correctness and Collision-freeness: Same as the previous exercise. □

Exercise 4.5. Define the syntax, correctness, and collision-freeness of an additive universal accumulator (i.e., an additive positive accumulator that supports non-membership proofs).

Solution. **Syntax:** An additive universal accumulator is a tuple of polynomial-time algorithms

$$(\mathsf{Gen}, \mathsf{Init}, \mathsf{Value}, \mathsf{Insert}, \mathsf{ProveMembership}, \mathsf{VerifyMembership},\\ \mathsf{ProveNonMembership}, \mathsf{VerifyNonMembership}),$$

where the first six algorithms are as in the additive positive accumulator, and:

- $\mathsf{ProveNonMembership}(pp, state, x) \to \pi$ is a probabilistic algorithm that takes public parameters pp, an accumulator state, $state$, and an element x as input and outputs a non-membership proof π.
- $\mathsf{VerifyNonMembership}(pp, v, x, \pi) \to \{0, 1\}$ is a deterministic algorithm that takes public parameters pp, an accumulator value v, an element $x \in \mathcal{X}_{\mathsf{Acc}}$, and a proof π as input and outputs 1 (accept) if the proof validates that x is not a member of the set represented by v, and outputs 0 (reject) otherwise.

Correctness: An additive universal accumulator is *correct* if both membership and non-membership proofs verify correctly. Formally, for all security parameters λ, all public parameters $pp \coloneqq \mathsf{Gen}(1^\lambda)$, all values $x \in \mathcal{X}_{\mathsf{Acc}}$, all positive integers t polynomial in λ, all tuples $\vec{y} \in \mathcal{X}_{\mathsf{Acc}}^t$:

1. Membership correctness: For all $x \in \vec{y}$,

$$\Pr\left[\begin{array}{l} state_0 \coloneqq \mathsf{Init}(pp); \\ state_i \coloneqq \mathsf{Insert}(pp, state_{i-1}, y_i) \text{ for } i \in [1, t]; \\ \pi \coloneqq \mathsf{ProveMembership}(pp, state_t, x); \\ v \coloneqq \mathsf{Value}(pp, state_t); \\ \mathsf{VerifyMembership}(pp, v, x, \pi) = 1 \end{array}\right] = 1 - negl(\lambda).$$

2. Non-membership correctness: For all $x \notin \vec{y}$,

$$\Pr\left[\begin{array}{l} state_0 \coloneqq \mathsf{Init}(pp); \\ state_i \coloneqq \mathsf{Insert}(pp, state_{i-1}, y_i) \text{ for } i \in [1, t]; \\ \pi \coloneqq \mathsf{ProveNonMembership}(pp, state_t, x); \\ v \coloneqq \mathsf{Value}(pp, state_t); \\ \mathsf{VerifyNonMembership}(pp, v, x, \pi) = 1 \end{array}\right] = 1 - negl(\lambda).$$

Collision-Freeness: An additive universal accumulator is *collision-free* if it's hard to fabricate either a membership proof for a non-member or a non-membership proof for a member. We extend the collision game from Figure 4.1 as follows:

$\mathsf{Game}\ \mathsf{UnivAccColl}^{\mathcal{A}}_{\mathsf{Acc}}(\lambda)$

$pp \leftarrow\!\!\$\ \mathsf{Gen}(1^\lambda)$
$\mathcal{Q} := \emptyset$
$state := \mathsf{Init}(pp)$
$v := \mathsf{Value}(pp, state)$
$(x^*, \pi^*, b^*) := \mathcal{A}^{\mathsf{Insert},\mathsf{ProveMembership},\mathsf{ProveNonMembership}}(pp, v)$
if $b^* = 1$ **then**
  return $x^* \notin \mathcal{Q} \wedge \mathsf{VerifyMembership}(pp, v, x^*, \pi^*) = 1$
else
  return $x^* \in \mathcal{Q} \wedge \mathsf{VerifyNonMembership}(pp, v, x^*, \pi^*) = 1$

The oracles Insert and ProveMembership are as before, and ProveNonMembership(x) returns $\mathsf{ProveNonMembership}(pp, state, x)$ if $x \notin \mathcal{Q}$ and $\perp$ otherwise.

An accumulator Acc is collision-free if for any p.p.t. algorithm $\mathcal{A}$:

$$\mathsf{Adv}^{\mathrm{univcoll}}_{\mathcal{A},\mathsf{Acc}}(\lambda) := \Pr\left[\mathsf{UnivAccColl}^{\mathcal{A}}_{\mathsf{Acc}}(\lambda) = 1\right] = negl(\lambda).$$

□

Exercise 4.6 (Optional). Using a collision-resistant hash function, give an example of an additive universal accumulator with constant-size accumulator values and logarithmic-size membership and non-membership proofs. Argue that it is correct and collision-free.

Solution. We can construct a universal accumulator using a **sparse Merkle tree**. In a sparse Merkle tree, we have a fixed domain $\mathcal{X}_{\mathsf{Acc}} = \{0,1\}^\lambda$ and build a complete binary tree of depth λ. Each element $x \in \mathcal{X}_{\mathsf{Acc}}$ maps to a unique leaf position determined by interpreting x as a binary path.

Construction:

- $\mathsf{Gen}(1^\lambda)$: Output $pp = (\mathcal{X}_{\mathsf{Acc}} = \{0,1\}^\lambda, \mathsf{H})$ where H is a collision-resistant hash function.
- $\mathsf{Init}(pp)$: Initialize an empty sparse Merkle tree where all leaves have a default value $\perp$ (representing non-membership).
- $\mathsf{Value}(pp, state)$: Return the root hash of the sparse Merkle tree.
- $\mathsf{Insert}(pp, state, x)$: Set the leaf at position x to a non-$\perp$ value (e.g., $\mathsf{H}(x)$) and update the tree.
- $\mathsf{ProveMembership}(pp, state, x)$: Return the authentication path from leaf x to the root.
- $\mathsf{VerifyMembership}(pp, v, x, \pi)$: Verify that the path leads to root v and the leaf value is non-$\perp$.
- $\mathsf{ProveNonMembership}(pp, state, x)$: Return the authentication path showing the leaf at position x has value $\perp$.
- $\mathsf{VerifyNonMembership}(pp, v, x, \pi)$: Verify that the path leads to root v and the leaf value is $\perp$.

Efficiency: The accumulator value (root hash) has constant size. Both membership and non-membership proofs consist of λ sibling hashes along the path, giving logarithmic proof size.

Correctness:

- Membership: If x was inserted, its leaf has a non-$\perp$ value, and the authentication path correctly verifies to the root.
- Non-membership: If x was never inserted, its leaf remains $\perp$, and the authentication path proves this.

Collision-freeness: To forge a membership proof for a non-member (or vice versa), an adversary would need to provide an authentication path that:

- Hashes to the correct root value
- Shows a different leaf value than what's actually stored

This would require finding a collision in the hash function, which is infeasible by assumption.

Alternative: A sorted Merkle tree could also be used, where elements are stored in sorted order at the leaves. Non-membership proofs would show two adjacent leaves with values that bracket the queried element. □

Exercise 4.7 (Optional). Read Section 2 of Baldimtsi, Canetti, and Yakoubov [BCY20]. Describe the notion of *Strength* for accumulators.

Exercise 4.8 (Optional). Read Sections 3.5, 3.6, and Appendix A.2 of Shielded CSV [NEL25].

1. How do the accumulators required for Shielded CSV differ from the accumulators discussed in this section?
2. How would you approach proving the ToSA-SEC security property (an open problem at the time of writing)?

Exercise 4.9 (Optional). Read Section 5 of Cremer, Loss, and Wagner [CLW24], which provides security notions for various cryptographic primitives. Summarize the key insights.

5 One-Time Signatures

5.1 Syntax & Correctness

Definition 5.1 (One-Time Signature). *A one-time signature scheme* OTS *is a tuple of polynomial-time algorithms* $(\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ *where:*

- $\mathsf{Setup}(1^\lambda) \to pp$ *is a probabilistic algorithm that takes the security parameter as input and outputs public parameters* pp *including the message space* $\mathcal{M}$*.*
- $\mathsf{KeyGen}(pp) \to (sk, pk)$ *is a probabilistic algorithm that takes public parameters* pp *as input and outputs a secret key* sk *and a public key* pk*.*
- $\mathsf{Sign}(pp, sk, m) \to \sigma$ *is a probabilistic algorithm that takes public parameters* pp*, a secret key* sk*, and a message* $m \in \mathcal{M}$ *as input and outputs a signature* σ*.*
- $\mathsf{Verify}(pp, pk, m, \sigma) \to \{0, 1\}$ *is a deterministic algorithm that takes public parameters* pp*, a public key* pk*, a message* $m \in \mathcal{M}$*, and a signature* σ *as input and outputs* 1 *(accept) if the signature is valid, and outputs* 0 *(reject) otherwise.*

Definition 5.2 (Correctness). *A one-time signature scheme* $\mathsf{OTS} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ *is* correct *if for all* $\lambda \in \mathbb{N}$*, all* $pp \in \mathsf{Setup}(1^\lambda)$*, all* $(sk, pk) \in \mathsf{KeyGen}(pp)$*, and all messages* $m \in \mathcal{M}$*:*

$$\Pr[\mathsf{Verify}(pp, pk, m, \mathsf{Sign}(pp, sk, m)) = 1] = 1$$

where the probability is taken over the randomness of Sign*.*

5.2 Security

Definition 5.3 (EUF-CMA Security). *A one-time signature scheme* OTS *is* EUF-CMA secure *(Existential Unforgeability under Chosen Message Attack) if for all p.p.t. adversaries* $\mathcal{A}$*, it holds that*

$$\mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{OTS}}(\lambda) \coloneqq \Pr\left[\mathsf{Game\ EUF\text{-}CMA}^{\mathcal{A}}_{\mathsf{OTS}}(\lambda) = 1\right] = negl(\lambda).$$

Remark 5.1 (Assert notation). Given a predicate P, we use "**assert** P" as a shorthand for "**if** $\neg P$ **then return** 0" in the main procedure or "**if** $\neg P$ **then return** $\bot$" in an oracle procedure.

Game $\mathsf{EUF\text{-}CMA}^{\mathcal{A}}_{\mathsf{OTS}}(\lambda)$

$pp \coloneqq \mathsf{Setup}(1^\lambda)$
$(sk, pk) \coloneqq \mathsf{KeyGen}(pp)$
$\mathcal{Q} \coloneqq \emptyset$
$(m^*, \sigma) \coloneqq \mathcal{A}^{\textsc{Sign}}(pp, pk)$
return $m^* \notin \mathcal{Q} \wedge \mathsf{Verify}(pp, pk, m^*, \sigma) = 1$

Oracle $\textsc{Sign}(m)$

assert $\mathcal{Q} = \emptyset$
$\mathcal{Q} \coloneqq \{m\}$
return $\mathsf{Sign}(pp, sk, m)$

Fig. 5.1. The EUF-CMA security game for a one-time signature scheme.

$\mathsf{Setup}(1^\lambda)$

$pp \coloneqq \mathsf{H.Gen}(1^\lambda)$
return pp

$\mathsf{KeyGen}(pp)$

$x_{i,j} \leftarrow\!\!\$\ \{0,1\}^\lambda$ for $i \in [1,k], j \in \{0,1\}$
$sk \coloneqq (x_{1,0}, x_{1,1}, \ldots, x_{k,0}, x_{k,1})$
$pk \coloneqq (\mathsf{H.Eval}(pp, x_{1,0}), \mathsf{H.Eval}(pp, x_{1,1}), \ldots,$
$\quad \mathsf{H.Eval}(pp, x_{k,0}), \mathsf{H.Eval}(pp, x_{k,1}))$
return (sk, pk)

$\mathsf{Sign}(pp, sk, m)$

// Sign must be called at most once per sk.
$(m_1, \ldots, m_k) \coloneqq m$
$(sk_{1,0}, sk_{1,1}), \ldots, (sk_{k,0}, sk_{k,1}) \coloneqq sk$
$\sigma \coloneqq (sk_{1,m_1}, \ldots, sk_{k,m_k})$
return σ

$\mathsf{Verify}(pp, pk, m, \sigma)$

$(m_1, \ldots, m_k) \coloneqq m$
$(pk_{1,0}, pk_{1,1}), \ldots, (pk_{k,0}, pk_{k,1}) \coloneqq pk$
return $\bigwedge_{i=1}^{k} \mathsf{H.Eval}(pp, \sigma_i) = pk_{i,m_i}$

Fig. 5.2. The one-time Lamport signature scheme $\mathsf{LS}[\mathsf{H}, k]$.

5.3 Lamport Signatures

Definition 5.4 (Lamport Signature Scheme). *Let* H *be a hash function and* k *be a positive integer. The Lamport one-time signature scheme* $\mathsf{LS}[\mathsf{H}, k]$ *with message space* $\mathcal{M} = \{0,1\}^k$ *is defined in Figure 5.2.*

Proposition 5.1 (Correctness of Lamport Signatures). *The Lamport signature scheme* $\mathsf{LS}[\mathsf{H}, k]$ *is correct.*

The proof of Proposition 5.1 is left as an exercise (see Exercise 5.4).

Theorem 5.1 (Security of Lamport Signatures). *Let* H *be a preimage-resistant hash function. Then the Lamport one-time signature scheme* $\mathsf{LS}[\mathsf{H}, k]$ *is EUF-CMA secure. More precisely, for any p.p.t. adversary* $\mathcal{A}$ *against the EUF-CMA security of* $\mathsf{LS}[\mathsf{H}, \mathsf{k}]$*, there exists a p.p.t. adversary* $\mathcal{B}$ *against the preimage resistance of* H *such that*

$$\mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{LS}[\mathsf{H},k]}(\lambda) \leq 2k\mathsf{Adv}^{\mathrm{preimage}}_{\mathcal{B},\mathsf{H}}(\lambda).$$

Proof. Figure 5.3 shows a sequence of games for the proof that Lamport signatures are EUF-CMA secure. The first game is identical to the Game EUF-CMA game. Each subsequent game introduces a small change compared to the previous game. Then we can construct a p.p.t. algorithm that runs a Game EUF-CMA adversary $\mathcal{A}$ internally and wins $\mathsf{Game\ Preimage}^{\mathcal{A}}_{\mathsf{H}}(\lambda)$ with the same probability as $\mathcal{A}$ winning the last Game G_2. The rest of the proof is left as an exercise (see Exercise 5.5). □

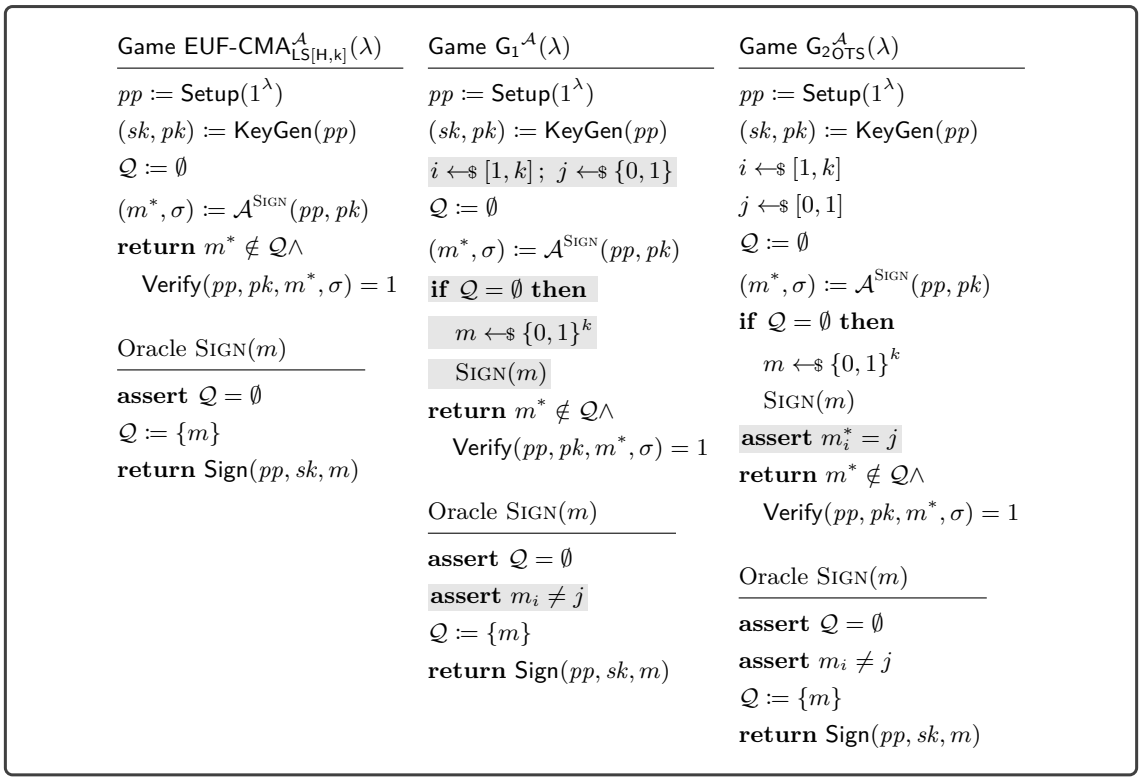

Fig. 5.3. Sequence of games for the proof that Lamport signatures are EUF-CMA secure. Differences are highlighted.

5.4 Exercises

Exercise 5.1. What is "existential unforgeability" in "EUF-CMA" (Definition 5.3) and what is "chosen message attack"?

Solution. **Existential unforgeability:** The adversary wins if it can forge a signature for *any* message of its choice. **Chosen message attack:** The adversary has access to a signing oracle and can obtain a signature for a message of its choice before attempting the forgery. □

Exercise 5.2. To see why the security definition for signatures includes a signing oracle, consider Game $\mathsf{G}^{\mathcal{A}}_{\mathsf{OTS}}(\lambda)$ which is the same as Game EUF-CMA$^{\mathcal{A}}_{\mathsf{OTS}}(\lambda)$ except that the adversary $\mathcal{A}$ is not given access to the signing oracle. Give an example of a one-time signature scheme that is correct and achieves $\Pr\left[\text{Game } \mathsf{G}^{\mathcal{A}}_{\mathsf{OTS}}(\lambda) = 1\right] = negl(\lambda)$ for all p.p.t. adversaries $\mathcal{A}$ but is not EUF-CMA secure.

Solution. Consider the following scheme:

- $\mathsf{KeyGen}(pp) = (sk = x, pk = \mathsf{H.Eval}(pp, x))$ where $x \leftarrow\!\$\ \{0, 1\}^\lambda$.
- $\mathsf{Sign}(pp, sk, m) = sk$ (simply outputs the secret key).
- $\mathsf{Verify}(pp, pk, m, \sigma) = 1$ if and only if $\mathsf{H.Eval}(pp, \sigma) = pk$.

This scheme is correct and would be secure in Game G (without signing oracle) since the adversary cannot forge without knowing sk, which requires solving the preimage problem. However, this scheme is obviously insecure in practice: anyone who sees one signature learns the secret key and can forge signatures for any message! The Game EUF-CMA game with signing oracle correctly captures this insecurity. □

Exercise 5.3. Is $(sk_{1,0}, sk_{2,0}, sk_{3,1})$ a valid signature for the message $m = (0, 0, 1)$ under the Lamport signature scheme $\mathsf{LS}[\mathsf{H}, 3]$?

Solution. Yes, it is a valid signature. For message $m = (0, 0, 1)$ with bits $m_1 = 0$, $m_2 = 0$, $m_3 = 1$, the Lamport signing algorithm outputs $(sk_{1,m_1}, sk_{2,m_2}, sk_{3,m_3}) = (sk_{1,0}, sk_{2,0}, sk_{3,1})$. The verification will check that $\mathsf{H.Eval}(pp, sk_{i,m_i}) = pk_{i,m_i}$ for each $i \in \{1, 2, 3\}$, which holds by construction of the public key during key generation. □

Exercise 5.4. Prove Proposition 5.1.

Solution. We prove that the Lamport signature scheme satisfies the correctness definition (Definition 5.2).

Let $\lambda \in \mathbb{N}$, $pp \in \mathsf{Setup}(1^\lambda)$, $(sk, pk) \in \mathsf{KeyGen}(pp)$, and $m = (m_1, \ldots, m_k) \in \{0,1\}^k$. We need to show that $\Pr[\mathsf{Verify}(pp, pk, m, \mathsf{Sign}(pp, sk, m)) = 1] = 1$.

By the definition of the Lamport scheme:

- KeyGen sets $sk = (x_{1,0}, x_{1,1}, \ldots, x_{k,0}, x_{k,1})$ and $pk = (pk_{1,0}, pk_{1,1}, \ldots, pk_{k,0}, pk_{k,1})$ where $pk_{i,b} = \mathsf{H.Eval}(pp, x_{i,b})$.
- $\mathsf{Sign}(pp, sk, m)$ outputs $\sigma = (sk_{1,m_1}, \ldots, sk_{k,m_k})$.
- $\mathsf{Verify}(pp, pk, m, \sigma)$ outputs 1 if and only if $\bigwedge_{i=1}^{k} \mathsf{H.Eval}(pp, \sigma_i) = pk_{i,m_i}$.

For the signature σ produced by Sign, we have $\sigma_i = sk_{i,m_i} = x_{i,m_i}$ for each $i \in [1, k]$. Therefore:

$$\mathsf{H.Eval}(pp, \sigma_i) = \mathsf{H.Eval}(pp, x_{i,m_i}) = pk_{i,m_i}$$

holds for all $i \in [1, k]$. Thus $\mathsf{Verify}(pp, pk, m, \sigma) = 1$ with probability 1 (the Lamport signing algorithm is deterministic). □

Exercise 5.5. Complete the proof of Theorem 5.1:

1. Express $\Pr\left[\mathsf{Game}\ \mathsf{G_1}^{\mathcal{A}}(\lambda) = 1\right]$ in terms of $\Pr\left[\mathsf{Game\ EUF\text{-}CMA}^{\mathcal{A}}_{\mathsf{LS[H,k]}}(\lambda) = 1\right]$.
2. Express $\Pr\left[\mathsf{Game}\ \mathsf{G_2}^{\mathcal{A}}(\lambda) = 1\right]$ in terms of $\Pr\left[\mathsf{Game}\ \mathsf{G_1}^{\mathcal{A}}(\lambda) = 1\right]$.
3. Give a p.p.t. algorithm $\mathcal{B}^{\mathcal{A}}$ that wins $\mathsf{Game\ Preimage}^{\mathcal{A}}_{\mathsf{H}}(\lambda)$ with probability $\Pr\left[\mathsf{Game}\ \mathsf{G_2}^{\mathcal{A}}(\lambda) = 1\right]$.
4. Put it all together to prove Theorem 5.1.

Solution. We have

$$\begin{aligned}
\Pr\left[\mathsf{Game}\ \mathsf{G_1}^{\mathcal{A}}(\lambda) = 1\right] &= \Pr\left[(\mathsf{Game\ EUF\text{-}CMA}^{\mathcal{A}}_{\mathsf{LS[H,k]}}(\lambda)) = 1) \wedge (m_i \neq j)\right] \\
&= \Pr\left[\mathsf{Game\ EUF\text{-}CMA}^{\mathcal{A}}_{\mathsf{LS[H,k]}}(\lambda) = 1 | m_i \neq j\right] \Pr\left[m_i \neq j\right] \\
&= \frac{1}{2} \Pr\left[\mathsf{Game\ EUF\text{-}CMA}^{\mathcal{A}}_{\mathsf{LS[H,k]}}(\lambda) = 1\right]
\end{aligned}$$

$\mathsf{Game}\ \mathsf{G_2}^{\mathcal{A}}(\lambda) = 1$ only when $m^* \neq m$ and $m^*_i = j$. Therefore,

$$\begin{aligned}
\Pr\left[\mathsf{Game}\ \mathsf{G_2}^{\mathcal{A}}(\lambda) = 1\right] &= \Pr\left[\mathsf{Game}\ \mathsf{G_1}^{\mathcal{A}}(\lambda) = 1 \wedge m^*_i = j\right] \\
&= \Pr\left[m^*_i = j | \mathsf{Game}\ \mathsf{G_1}^{\mathcal{A}}(\lambda) = 1\right] \Pr\left[\mathsf{Game}\ \mathsf{G_1}^{\mathcal{A}}(\lambda) = 1\right] \\
&\geq \frac{1}{k} \Pr\left[\mathsf{Game}\ \mathsf{G_1}^{\mathcal{A}}(\lambda) = 1\right]
\end{aligned}$$

The last inequality holds because when $\mathsf{Game}\ \mathsf{G_1}$ succeeds, we have $m \neq m^*$ (at least one position differs), $m_i \neq j$, and i is uniformly random. The probability that position i is one where m and m^* differ is at least $1/k$. When they differ at position i and $m_i \neq j$, then $m^*_i = j$ (since the message space is binary). $\mathcal{B}^{\mathcal{A}}$ is the same as $\mathsf{Game}\ \mathsf{G_2}$ except that

- it gets a preimage challenge y from a challenger and
- replaces $pk_{i,j}$ with y

– it returns σ_i.

Since $m_i \neq j$, the inputs to the adversary (including the signing oracle call) has the same distribution as in Game G_2. Moreover, if Game ${\mathsf{G}_2}^{\mathcal{A}}(\lambda)$ returns 1, then $\mathsf{H.Eval}(pp, \sigma_i) = pk_{i,j}$ Therefore,

$$\mathsf{Adv}^{\text{preimage}}_{\mathcal{B}^{\mathcal{A}},\mathsf{H}}(\lambda) = \Pr\left[\text{Game } {\mathsf{G}_2}^{\mathcal{A}}(\lambda) = 1\right]$$

and then combining the inequalities:

$$\begin{aligned}
\mathsf{Adv}^{\text{euf-cma}}_{\mathcal{A},\mathsf{LS[H,k]}}(\lambda) &= \Pr\left[\text{Game EUF-CMA}^{\mathcal{A}}_{\mathsf{LS[H,k]}}(\lambda) = 1\right] \\
&= 2 \cdot \Pr\left[\text{Game } {\mathsf{G}_1}^{\mathcal{A}}(\lambda) = 1\right] \\
&\leq 2k \cdot \Pr\left[\text{Game } {\mathsf{G}_2}^{\mathcal{A}}(\lambda) = 1\right] \\
&= 2k \cdot \mathsf{Adv}^{\text{preimage}}_{\mathcal{B}^{\mathcal{A}},\mathsf{H}}(\lambda)
\end{aligned}$$

□

Exercise 5.6. Let Game G' be the same as Game EUF-CMA except that the signing oracle can be called twice. Where does the proof of Exercise 5.5 break?

Solution. The proof breaks at Game G_1, which asserts that $m_i \neq j$. With two signing queries, an adversary can query messages that cover both possible values at every position. For example, the adversary can query $m^{(1)} = (0, 0, \ldots, 0)$ and $m^{(2)} = (1, 1, \ldots, 1)$. For any choice of $i \in [1, k]$ and $j \in \{0, 1\}$, either $m_i^{(1)} = j$ or $m_i^{(2)} = j$, causing Game G_1 to abort. Therefore, $\Pr[\text{Game } \mathsf{G}_1 = 1] = 0$ for this adversary, breaking the reduction. This shows why Lamport signatures are only secure for one-time use. □

Exercise 5.7 (Optional). Why are Lamport signatures considered post-quantum secure?

Solution. Lamport signatures are considered post-quantum secure because their security relies solely on the preimage resistance of the underlying hash function. Unlike RSA or ECDSA signatures that rely on the hardness of factoring or discrete logarithm problems (which can be solved efficiently by quantum computers using Shor's algorithm), hash function preimage resistance is believed to remain hard even for quantum computers. Grover's algorithm provides at most a quadratic speedup for finding preimages, which can be compensated by doubling the hash output length. Therefore, with appropriately chosen parameters, Lamport signatures remain secure against quantum adversaries. □

Exercise 5.8 (Optional). Consider a variant of Lamport signatures that uses an accumulator (Definition 4.1). Instead of storing $pk = (pk_{1,0}, pk_{1,1}, \ldots, pk_{k,0}, pk_{k,1})$, we insert all $2k$ values $\{pk_{i,b} : i \in [1, k], b \in \{0, 1\}\}$ into an accumulator and set the public key to be the accumulator value.

1. Describe how signing and verification would work.
2. What are the trade-offs compared to standard Lamport signatures?
3. For security, what property of the accumulator is needed? Sketch how you would modify the EUF-CMA proof.

Solution. 1. **Signing and Verification:**
   – **KeyGen:** Generate Lamport keys as usual: $x_{i,b} \leftarrow\!\!\$\ \{0,1\}^\lambda$ and $pk_{i,b} = \mathsf{H.Eval}(pp, x_{i,b})$ for $i \in [1, k], b \in \{0, 1\}$. Insert all $pk_{i,b}$ values into an accumulator and output $pk = v$ (the accumulator value) and keep $sk = ((x_{1,0}, x_{1,1}), \ldots, (x_{k,0}, x_{k,1}), state_{acc})$ where $state_{acc}$ is the accumulator state.
   – **Sign:** For message $m = (m_1, \ldots, m_k)$, compute membership proofs

$$\pi_i := \mathsf{Acc.ProveMembership}(pp_{acc}, state_{acc}, pk_{i,m_i})$$

   for each $i \in [1, k]$. Output $\sigma = ((x_{1,m_1}, \pi_1), \ldots, (x_{k,m_k}, \pi_k))$.

– **Verify:** For each $i \in [1,k]$, compute $pk'_{i,m_i} = \mathsf{H.Eval}(pp, x_{i,m_i})$ and check that

$$\mathsf{Acc.VerifyMembership}(pp_{acc}, v, pk'_{i,m_i}, \pi_i) = 1.$$

Output 1 if all checks pass.

2. **Trade-offs:**
   – **Advantage:** Smaller public key (just the accumulator value) instead of $2k$ hash values.
   – **Disadvantage:** Larger signatures (must include k membership proofs).
3. **Security:** We need the accumulator to be collision-free.
   Theorem: Let H be a preimage-resistant hash function and Acc be a collision-free accumulator. Then the Lamport signature scheme with accumulated public key is EUF-CMA secure. More precisely, for any p.p.t. adversary $\mathcal{A}$, there exist p.p.t. adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that:

$$\mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{LS\text{-}Acc}}(\lambda) \leq 2k \cdot (\mathsf{Adv}^{\mathrm{preimage}}_{\mathcal{B}_1,\mathsf{H}}(\lambda) + \mathsf{Adv}^{\mathrm{coll}}_{\mathcal{B}_2,\mathsf{Acc}}(\lambda))$$

   Proof sketch: We follow the game sequence from the original Lamport proof. Let $\mathsf{Game}\ \mathsf{G}_2$ be the game where we guess (i^*, j^*) and abort unless the adversary queries m with $m_{i^*} \neq j^*$ and forges on m^* with $m^*_{i^*} = j^*$. As in the original proof, $\Pr[\mathsf{Game}\ \mathsf{G}_2 = 1] \geq \frac{1}{2k} \cdot \mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{LS\text{-}Acc}}(\lambda)$.
   Now we build a p.p.t. reduction $\mathcal{B}_1$ that simulates $\mathsf{Game}\ \mathsf{G}_2$:
   – $\mathcal{B}_1$ receives preimage challenge y, embeds it as $pk_{i^*,j^*} = y$, and runs $\mathsf{Game}\ \mathsf{G}_2$ with the adversary.
   – When the adversary succeeds in $\mathsf{Game}\ \mathsf{G}_2$, they provide $(x^*_{i^*,j^*}, \pi^*_{i^*})$ where the membership proof verifies for some value $pk' = \mathsf{H.Eval}(pp, x^*_{i^*,j^*})$.
   – If $pk' = y$, then $\mathcal{B}_1$ outputs $x^*_{i^*,j^*}$ as the preimage of y.
   However, the adversary might succeed with $pk' \neq y$. This means the adversary found a valid membership proof for a value different from what we inserted at position (i^*, j^*). To handle this, we build a p.p.t. reduction $\mathcal{B}_2$:
   – $\mathcal{B}_2$ also simulates $\mathsf{Game}\ \mathsf{G}_2$ but generates all keys honestly (no preimage challenge).
   – When the adversary succeeds with some $(x^*_{i^*,j^*}, \pi^*_{i^*})$ where $pk' = \mathsf{H.Eval}(pp, x^*_{i^*,j^*}) \neq pk_{i^*,j^*}$, this constitutes a collision for the accumulator.
   – $\mathcal{B}_2$ outputs this as evidence of breaking collision-freeness.
   We have:

$$\begin{aligned}\Pr[\mathsf{Game}\ \mathsf{G}_2 = 1] &= \Pr[\mathsf{Game}\ \mathsf{G}_2 = 1 \wedge pk' = pk_{i^*,j^*}] + \Pr[\mathsf{Game}\ \mathsf{G}_2 = 1 \wedge pk' \neq pk_{i^*,j^*}] \\ &\leq \mathsf{Adv}^{\mathrm{preimage}}_{\mathcal{B}_1,\mathsf{H}}(\lambda) + \mathsf{Adv}^{\mathrm{coll}}_{\mathcal{B}_2,\mathsf{Acc}}(\lambda)\end{aligned}$$

   Since $\mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{LS\text{-}Acc}}(\lambda) \leq 2k \cdot \Pr[\mathsf{Game}\ \mathsf{G}_2 = 1]$ (from the original proof), we get:

$$\mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{LS\text{-}Acc}}(\lambda) \leq 2k \cdot (\mathsf{Adv}^{\mathrm{preimage}}_{\mathcal{B}_1,\mathsf{H}}(\lambda) + \mathsf{Adv}^{\mathrm{coll}}_{\mathcal{B}_2,\mathsf{Acc}}(\lambda))$$

□

Exercise 5.9 (Optional). Describe how to build a stateful many-time signature scheme from a one-time signature scheme and an accumulator. Your construction should support signing up to n messages (where n is fixed at setup).

Solution. Here is a construction for a stateful many-time signature scheme:

– **Setup/KeyGen:**
  1. Generate n one-time signature key pairs: $(sk_i, pk_i) \coloneqq \mathsf{OTS.KeyGen}(pp)$ for $i \in [1,n]$.
  2. Insert all n public keys $\{pk_1, \ldots, pk_n\}$ into an accumulator.
  3. The master public key is $pk = v$ (the accumulator value).
  4. The master secret key is $sk = ((sk_1, pk_1), \ldots, (sk_n, pk_n), state_{acc})$ where $state_{acc}$ is the accumulator state.
  5. Initialize a counter $ctr = 0$.
– **Sign (stateful):**
  1. Increment $ctr \coloneqq ctr + 1$.
  2. If $ctr > n$, abort (all keys have been used).

3. Sign the message using the ctr-th OTS key: $\sigma_{ots} \coloneqq \mathsf{OTS.Sign}(pp, sk_{ctr}, m)$.
4. Generate a membership proof: $\pi \coloneqq \mathsf{Acc.ProveMembership}(pp_{acc}, state_{acc}, pk_{ctr})$.
5. Output $\sigma = (ctr, pk_{ctr}, \sigma_{ots}, \pi)$.

– **Verify:**
1. Parse $\sigma = (i, pk_i, \sigma_{ots}, \pi)$.
2. Verify the OTS signature: check that $\mathsf{OTS.Verify}(pp, pk_i, m, \sigma_{ots}) = 1$.
3. Verify accumulator membership: check that $\mathsf{Acc.VerifyMembership}(pp_{acc}, v, pk_i, \pi) = 1$.
4. Output 1 if both checks pass, 0 otherwise.

The scheme is stateful because the signer must track which keys have been used (via ctr) to ensure no OTS key is reused. Security relies on the one-time unforgeability of the OTS scheme and the collision-freeness of the accumulator. □

Exercise 5.10 (Optional). Read Sections 2.2, Theorem 1, and 3.2 of Hülsing [Hül13] about the Winternitz One-Time Signature scheme. Explain what challenges the security reduction embeds and how they are embedded.

6 Discrete Logarithm

6.1 The Discrete Logarithm Problem

Definition 6.1 (Group Generation Algorithm). *A* group generation algorithm GrGen *is a p.p.t. algorithm that takes the security parameter* 1^λ *as input and outputs a group description* $(\mathbb{G}, p, g)$*, where* $\mathbb{G}$ *is a cyclic group of prime order* p *with* $p \geq 2^{\lambda-1}$*, and* g *is a generator of* $\mathbb{G}$*.*

Remark 6.1. The group operation is denoted multiplicatively ($g \cdot g = g^2$).

$\mathsf{Game}\ \mathsf{DL}^{\mathcal{A}}_{\mathsf{GrGen}}(\lambda)$

$(\mathbb{G}, p, g) \coloneqq \mathsf{GrGen}(1^\lambda)$
$x \leftarrow\!\!\$\ \mathbb{Z}_p$
$X \coloneqq g^x$
$x' \coloneqq \mathcal{A}((\mathbb{G}, p, g), X)$
return $x' = x$

Fig. 6.1. The discrete logarithm game

Definition 6.2 (Discrete Logarithm Assumption). *Let* GrGen *be a group generation algorithm, and let* $\mathsf{Game}\ \mathsf{DL}^{\mathcal{A}}_{\mathsf{GrGen}}(\lambda)$ *be as defined in Figure 6.1. The discrete logarithm (DL) problem is hard for* GrGen *if for any p.p.t. algorithm* $\mathcal{A}$*,*

$$\mathsf{Adv}^{\mathrm{dl}}_{\mathcal{A},\mathsf{GrGen}}(\lambda) \coloneqq \Pr\left[\mathsf{Game}\ \mathsf{DL}^{\mathcal{A}}_{\mathsf{GrGen}}(\lambda) = 1\right] = negl(\lambda).$$

6.2 Pedersen Commitments

Definition 6.3 (Pedersen Commitment Scheme). *Let* GrGen *be a group generation algorithm. The Pedersen commitment scheme* $\mathsf{PedCom}[\mathsf{GrGen}]$ *is defined as follows:*

– $\mathsf{Setup}(1^\lambda) \to pp$*: Run* $(\mathbb{G}, p, g) \coloneqq \mathsf{GrGen}(1^\lambda)$*, sample* $h \leftarrow\!\!\$\ \mathbb{G} \setminus \{1_{\mathbb{G}}\}$*, and output* $pp = ((\mathbb{G}, p, g), h, \mathcal{M} \coloneqq \mathbb{Z}_p, \mathcal{R} \coloneqq \mathbb{Z}_p)$*.*
– $\mathsf{Commit}(pp, m, r) \to C$*: Parse* $pp = ((\mathbb{G}, p, g), h, \mathcal{M}, \mathcal{R})$*. For message* $m \in \mathcal{M}$ *and randomness* $r \in \mathcal{R}$*, output* $C = g^m h^r$*.*

Remark 6.2. Pedersen commitments are *homomorphic*: for commitments $C_1 = \mathsf{Commit}(pp, m_1, r_1)$ and $C_2 = \mathsf{Commit}(pp, m_2, r_2)$, we have

$$C_1 \cdot C_2 = g^{m_1+m_2} h^{r_1+r_2} = \mathsf{Commit}(pp, m_1 + m_2, r_1 + r_2).$$

Theorem 6.1 (Pedersen Commitments are Binding). *Let* GrGen *be a group generation algorithm for which the discrete logarithm problem is hard, then the Pedersen commitment scheme* $\mathsf{PedCom}[\mathsf{GrGen}]$ *is binding. More precisely, for any p.p.t. adversary* $\mathcal{A}$ *against the binding property of* $\mathsf{PedCom}[\mathsf{GrGen}]$*, there exists a p.p.t. adversary* $\mathcal{B}$ *against the discrete logarithm problem such that*

$$\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A},\mathsf{PedCom}}(\lambda) = \mathsf{Adv}^{\mathrm{dl}}_{\mathcal{B},\mathsf{GrGen}}(\lambda).$$

The proof is left as an exercise (see Exercise 6.2).

Theorem 6.2 (Pedersen Commitments are Hiding). *The Pedersen commitment scheme* $\mathsf{PedCom}[\mathsf{GrGen}]$ *is perfectly hiding. More precisely, for any (possibly unbounded) adversary* $\mathcal{A}$*,*

$$\mathsf{Adv}^{\mathrm{hid}}_{\mathcal{A},\mathsf{PedCom}}(\lambda) = 0.$$

The proof is left as an exercise (see Exercise 6.3).

6.3 The DDH Assumption

Game $\mathsf{DDH}^{\mathcal{A}}_{\mathsf{GrGen}}(\lambda)$

$(\mathbb{G}, p, g) \coloneqq \mathsf{GrGen}(1^\lambda)$
$a, b \leftarrow\!\!\$\ \mathbb{Z}_p$
$A \coloneqq g^a, B \coloneqq g^b$
$d \leftarrow\!\!\$\ \{0,1\}$
if $d = 0$ **then**
  $c \leftarrow\!\!\$\ \mathbb{Z}_p$
  $C \coloneqq g^c$
else
  $C \coloneqq g^{ab}$
$d' \coloneqq \mathcal{A}((\mathbb{G}, p, g), A, B, C)$
return $d' = d$

Fig. 6.2. The decisional Diffie-Hellman game

Definition 6.4 (Decisional Diffie-Hellman Assumption). *Let* GrGen *be a group generation algorithm, and let* $\mathsf{Game}\ \mathsf{DDH}^{\mathcal{A}}_{\mathsf{GrGen}}(\lambda)$ *be as defined in Figure 6.2. The decisional Diffie-Hellman (DDH) problem is hard for* GrGen *if for any p.p.t. algorithm* $\mathcal{A}$*,*

$$\mathsf{Adv}^{\mathrm{ddh}}_{\mathcal{A},\mathsf{GrGen}}(\lambda) \coloneqq \left| \Pr\left[\mathsf{Game}\ \mathsf{DDH}^{\mathcal{A}}_{\mathsf{GrGen}}(\lambda) = 1\right] - \frac{1}{2} \right| = \mathit{negl}(\lambda).$$

6.4 ElGamal Commitments

Definition 6.5 (ElGamal Commitment Scheme). *Let* GrGen *be a group generation algorithm. The ElGamal commitment scheme* $\mathsf{ElGamalCom}[\mathsf{GrGen}]$ *is defined as follows:*

– $\mathsf{Setup}(1^\lambda) \to pp$: *Run* $(\mathbb{G}, p, g) \coloneqq \mathsf{GrGen}(1^\lambda)$, *sample* $h \leftarrow\!\!\$\ \mathbb{G} \setminus \{1_\mathbb{G}\}$, *and output* $pp = ((\mathbb{G}, p, g), h, \mathcal{M} \coloneqq \mathbb{Z}_p, \mathcal{R} \coloneqq \mathbb{Z}_p)$.
– $\mathsf{Commit}(pp, m, r) \to C$: *Parse* $pp = ((\mathbb{G}, p, g), h, \mathcal{M}, \mathcal{R})$. *For message* $m \in \mathcal{M}$ *and randomness* $r \in \mathcal{R}$, *output* $C = (g^r, h^{m+r})$.

Theorem 6.3 (ElGamal Commitments are Hiding). *Let* GrGen *be a group generation algorithm for which the DDH problem is hard, then the ElGamal commitment scheme* ElGamalCom[GrGen] *is hiding. More precisely, for any p.p.t. adversary* $\mathcal{A}$ *against the hiding property of* ElGamalCom[GrGen], *there exists a p.p.t. adversary* $\mathcal{B}$ *against DDH such that*

$$\mathsf{Adv}^{\mathrm{hide}}_{\mathcal{A},\mathsf{ElGamalCom}}(\lambda) = 2 \cdot \mathsf{Adv}^{\mathrm{ddh}}_{\mathcal{B},\mathsf{GrGen}}(\lambda).$$

The proof is left as an exercise (see Exercise 6.4).

6.5 The OMDL and AOMDL Problems

Game $\mathsf{OMDL}^{\mathcal{A}}_{\mathsf{GrGen}}(\lambda)$

$(\mathbb{G}, p, g) \coloneqq \mathsf{GrGen}(1^\lambda)$
$\ell \coloneqq 0\,;\ q \coloneqq 0$
$\vec{y} \coloneqq \mathcal{A}^{\textsc{Chal},\mathrm{DL}}((\mathbb{G}, p, g))$
$\vec{x} \coloneqq (x_1, \ldots, x_\ell)$
return $(\vec{y} = \vec{x} \wedge q < \ell)$

Oracle $\textsc{Chal}()$

$\ell \coloneqq \ell + 1$
$x_\ell \leftarrow\!\!\$\ \mathbb{Z}_p$
$X_\ell \coloneqq g^{x_\ell}$
return X_ℓ

Oracle $\mathrm{DL}(X)$

$q \coloneqq q + 1$
return $\log_g(X)$

Game $\mathsf{AOMDL}^{\mathcal{A}}_{\mathsf{GrGen}}(\lambda)$

$(\mathbb{G}, p, g) \coloneqq \mathsf{GrGen}(1^\lambda)$
$\ell \coloneqq 0\,;\ q \coloneqq 0$
$\vec{y} \coloneqq \mathcal{A}^{\textsc{Chal},\,\mathrm{ADL}}((\mathbb{G}, p, g))$
$\vec{x} \coloneqq (x_1, \ldots, x_\ell)$
return $(\vec{y} = \vec{x} \wedge q < \ell)$

Oracle $\mathrm{ADL}((\alpha, \beta_1, \ldots, \beta_\ell))$

$q \coloneqq q + 1$
return $\alpha + \sum_{i=1}^{\ell} \beta_i x_i$
// $= \log_g \left(g^\alpha \prod_{i=1}^{\ell} X_i^{\beta_i} \right)$

Fig. 6.3. The one-more discrete logarithm (OMDL) and algebraic one-more discrete logarithm (AOMDL) games. The difference is highlighted.

Definition 6.6 (One-More Discrete Logarithm Assumption). *Let* GrGen *be a group generation algorithm, and let* Game $\mathsf{OMDL}^{\mathcal{A}}_{\mathsf{GrGen}}(\lambda)$ *be as defined in Figure 6.3. The one-more discrete logarithm (OMDL) problem is hard for* GrGen *if for any p.p.t. algorithm* $\mathcal{A}$,

$$\mathsf{Adv}^{\mathrm{omdl}}_{\mathcal{A},\mathsf{GrGen}}(\lambda) \coloneqq \Pr\left[\text{Game } \mathsf{OMDL}^{\mathcal{A}}_{\mathsf{GrGen}}(\lambda) = 1\right] = negl(\lambda).$$

Definition 6.7 (Algebraic One-More Discrete Logarithm Assumption). *Let* GrGen *be a group generation algorithm, and let* Game $\mathsf{AOMDL}^{\mathcal{A}}_{\mathsf{GrGen}}(\lambda)$ *be as defined in Figure 6.3. The algebraic one-more discrete logarithm (AOMDL) problem is hard for* GrGen *if for any p.p.t. algorithm* $\mathcal{A}$,

$$\mathsf{Adv}^{\mathrm{aomdl}}_{\mathcal{A},\mathsf{GrGen}}(\lambda) \coloneqq \Pr\left[\text{Game } \mathsf{AOMDL}^{\mathcal{A}}_{\mathsf{GrGen}}(\lambda) = 1\right] = negl(\lambda).$$

Remark 6.3. When proving the security of a scheme using the AOMDL assumption, the ADL oracle is (typically) used by the reduction and not by the adversary against the scheme. Therefore, the security proof is not restricted to adversaries that know the algebraic representations of all group elements that they output.

Remark 6.4 (Falsifiable Assumptions). In order to be able to evaluate whether an assumption is true or not, it must be falsifiable, i.e., there must be a (constructive) way to demonstrate that it is false, if this is the case. More precisely, a cryptographic assumption is *falsifiable* if there exists a p.p.t. challenger algorithm that interacts with an adversary and decides whether the adversary breaks the assumption.

6.6 Exercises

Exercise 6.1. In the framework of asymptotic security, why is the DL problem on secp256k1 not hard? What does GrGen do?

Solution. The DL problem on secp256k1 is not hard in the asymptotic security framework because secp256k1 has a fixed 256-bit group order, independent of the security parameter λ. For asymptotic hardness, the group size must grow with λ. The group generation algorithm GrGen generates groups where $p \geq 2^{\lambda-1}$, ensuring that the problem scales appropriately with the security parameter. □

Exercise 6.2. Argue that Pedersen commitments are binding under the DL assumption (Theorem 6.1).

Solution. Let $\mathcal{A}$ be an adversary that breaks the binding property of Pedersen commitments. That is, $\mathcal{A}$ outputs (m_0, r_0, m_1, r_1) such that $m_0 \neq m_1$ and $g^{m_0}h^{r_0} = g^{m_1}h^{r_1}$. We construct a p.p.t. reduction $\mathcal{B}$ that breaks DL as follows:

- $\mathcal{B}$ receives $((\mathbb{G}, p, g), h)$ where $h = g^z$ for unknown z.
- $\mathcal{B}$ runs $\mathcal{A}$ with parameters $pp = ((\mathbb{G}, p, g), h)$.
- When $\mathcal{A}$ outputs (m_0, r_0, m_1, r_1), we have $g^{m_0}h^{r_0} = g^{m_1}h^{r_1}$.
- This implies $g^{m_0-m_1} = h^{r_1-r_0}$, so $g^{m_0-m_1} = g^{z(r_1-r_0)}$.
- If $r_0 = r_1$, then $g^{m_0-m_1} = 1$, which contradicts $m_0 \neq m_1$ (since g generates $\mathbb{G}$).
- Therefore $r_0 \neq r_1$, and we can compute $z = \frac{m_0-m_1}{r_1-r_0} \bmod p$.
- $\mathcal{B}$ outputs z.

The reduction is perfect: whenever $\mathcal{A}$ succeeds in breaking binding, $\mathcal{B}$ succeeds in computing the discrete log. Therefore, $\mathsf{Adv}^{\mathrm{dl}}_{\mathcal{B},\mathsf{GrGen}}(\lambda) = \mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A},\mathsf{PedCom}}(\lambda)$. □

Exercise 6.3. Prove that Pedersen commitments are perfectly hiding (Theorem 6.2).

Solution. For any message $m \in \mathbb{Z}_p$, the commitment $C = g^m h^r$ is uniformly distributed over $\mathbb{G}$ when r is chosen uniformly from $\mathbb{Z}_p$. Since $h \in \mathbb{G} \setminus \{1_\mathbb{G}\}$ and $\mathbb{G}$ has prime order p, h generates $\mathbb{G}$ (every non-identity element generates a prime-order group). Thus the map $r \mapsto h^r$ is a bijection from $\mathbb{Z}_p$ to $\mathbb{G}$. For fixed m, the map $r \mapsto g^m h^r$ is also a bijection from $\mathbb{Z}_p$ to $\mathbb{G}$. Therefore, for any two messages m_0, m_1, the distributions of $\mathsf{Commit}(pp, m_0, r)$ and $\mathsf{Commit}(pp, m_1, r)$ for uniform r are both uniform over $\mathbb{G}$. □

Exercise 6.4. Prove that ElGamal commitments are hiding under the DDH assumption (Theorem 6.3).

Solution. Let $\mathcal{A}$ be an adversary against the hiding property of ElGamal. We construct a p.p.t. reduction $\mathcal{B}$ against DDH:

- $\mathcal{B}$ receives $((\mathbb{G}, p, g), A, B, C)$ where $A = g^a$, $B = g^b$, and either $C = g^{ab}$ or $C = g^c$ for random c.
- $\mathcal{B}$ sets $h = B$ and gives $pp = ((\mathbb{G}, p, g), h)$ to $\mathcal{A}$.
- When $\mathcal{A}$ outputs (m_0, m_1), $\mathcal{B}$ chooses $e \leftarrow\!\!\$\ \{0, 1\}$ and computes: If $C = g^{ab}$, set implicitly $r = a$, so $(g^r, h^{m_e+r}) = (g^a, g^{b(m_e+a)}) = (A, g^{bm_e+ab}) = (A, B^{m_e} \cdot C)$. If $C = g^c$ for random c, then $(A, B^{m_e} \cdot C) = (g^a, g^{bm_e+c})$.
- $\mathcal{B}$ gives $(A, B^{m_e} \cdot C)$ to $\mathcal{A}$.
- When $\mathcal{A}$ outputs e', $\mathcal{B}$ outputs 1 if $e' = e$ and 0 otherwise.

Analysis: We have:

- When $C = g^{ab}$ (real tuple): $\mathcal{B}$ perfectly simulates the hiding game, so $|\Pr[e = e' | C = g^{ab}] - \frac{1}{2}| = \mathsf{Adv}^{\mathrm{hide}}_{\mathcal{A},\mathsf{ElGamalCom}}(\lambda)$.
- When $C = g^c$ (random tuple): The commitment reveals no information about e, so $\Pr[e = e' | C = g^c] = \frac{1}{2}$.

For the DDH advantage:

$$\begin{aligned}
\mathsf{Adv}^{\mathrm{ddh}}_{\mathcal{B},\mathsf{GrGen}}(\lambda) &= |\Pr[d = d'] - \frac{1}{2}| \\
\Pr[d = d'] &= \Pr[d' = 1 \wedge d = 1] + \Pr[d' = 0 \wedge d = 0] \\
&= \Pr[d' = 1 \wedge C = g^{ab}] + \Pr[d' = 0 \wedge C = g^c] \\
&= \Pr[d' = 1 | C = g^{ab}] \cdot \Pr[C = g^{ab}] + \Pr[d' = 0 | C = g^c] \cdot \Pr[C = g^c] \\
&= \frac{1}{2}\Pr[d' = 1 | C = g^{ab}] + \frac{1}{2}\Pr[d' = 0 | C = g^c] \\
&= \frac{1}{2}\Pr[d' = 1 | C = g^{ab}] + \frac{1}{2}(1 - \Pr[d' = 1 | C = g^c]) \\
&= \frac{1}{2}\Pr[d' = 1 | C = g^{ab}] + \frac{1}{2} - \frac{1}{2}\Pr[d' = 1 | C = g^c]
\end{aligned}$$

Therefore:

$$\begin{aligned}
\mathsf{Adv}^{\mathrm{ddh}}_{\mathcal{B},\mathsf{GrGen}}(\lambda) &= \left|\frac{1}{2}\Pr[d' = 1 | C = g^{ab}] - \frac{1}{2}\Pr[d' = 1 | C = g^c]\right| \\
&= \frac{1}{2}|\Pr[d' = 1 | C = g^{ab}] - \Pr[d' = 1 | C = g^c]| \\
&= \frac{1}{2}|\Pr[e = e' | C = g^{ab}] - \Pr[e = e' | C = g^c]| \\
&= \frac{1}{2}\left|\Pr[e = e' | C = g^{ab}] - \frac{1}{2}\right| \\
&= \frac{1}{2} \cdot \mathsf{Adv}^{\mathrm{hide}}_{\mathcal{A},\mathsf{ElGamalCom}}(\lambda)
\end{aligned}$$

This gives us: $\mathsf{Adv}^{\mathrm{hide}}_{\mathcal{A},\mathsf{ElGamalCom}}(\lambda) = 2 \cdot \mathsf{Adv}^{\mathrm{ddh}}_{\mathcal{B},\mathsf{GrGen}}(\lambda)$. $\mathcal{B}$ is p.p.t. if $\mathcal{A}$ is. □

Exercise 6.5. Argue that if OMDL is hard for a group generation algorithm then DL is hard.

Solution. If OMDL is hard, then DL must be hard. Given a p.p.t. DL adversary $\mathcal{A}$, we construct a p.p.t. OMDL adversary $\mathcal{B}$:

- $\mathcal{B}$ calls CHAL() once to get X_1.
- $\mathcal{B}$ runs $\mathcal{A}((\mathbb{G}, p, g), X_1)$ to get x_1'.
- $\mathcal{B}$ outputs (x_1').

If $\mathcal{A}$ succeeds, then $X_1 = g^{x_1'}$, so $\mathcal{B}$ solves one discrete log with zero challenge queries, winning the OMDL game. □

Exercise 6.6. Consider an algorithm $\mathcal{A}$ playing the AOMDL game. It queries CHAL twice to obtain X_1 and X_2, then samples $\alpha, \beta_1, \beta_2 \leftarrow\!\!\$\ \mathbb{Z}_p$.

1. How should $\mathcal{A}$ call the ADL oracle to obtain $\log_g(P)$ where $P = g^\alpha X_1^{\beta_1} X_2^{\beta_2}$?
2. Assuming $\mathcal{A}$ makes no more CHAL queries, how many more ADL queries can it make?
3. What must $\mathcal{A}$ return to win the game?

Solution. 1. $\mathcal{A}$ should call ADL$((\alpha, \beta_1, \beta_2))$. The oracle will return

$$\begin{aligned}
\alpha + \beta_1 x_1 + \beta_2 x_2 &= \log_g(g^\alpha X_1^{\beta_1} X_2^{\beta_2}) \\
&= \log_g(P)
\end{aligned}$$

.

2. Since $\ell = 2$ (two CHAL queries) and $q = 1$ (one ADL query), and the winning condition requires $q < \ell$, the adversary can make at most 0 more queries (as $1 < 2$ but $2 \not< 2$).
3. $\mathcal{A}$ must return (x_1, x_2), the discrete logarithms of X_1 and X_2.

□

Exercise 6.7. Is the OMDL assumption falsifiable? Is the AOMDL assumption falsifiable? Do we prefer the security of a scheme to be based on OMDL or AOMDL?

Solution. The OMDL assumption is *non-falsifiable* because verifying the adversary's output requires computing discrete logarithms to check that $g^{x_i} = X_i$, which cannot be done in polynomial time classically.

The AOMDL assumption is *falsifiable* because:

- The ADL oracle can be implemented efficiently: given $(\alpha, \beta_1, \ldots, \beta_\ell)$, it simply returns $\alpha + \sum_{i=1}^{\ell} \beta_i x_i$
- The winning condition can be verified by checking that the returned values equal the stored x_i values
- All operations (additions, multiplications) are polynomial time

We prefer basing security on falsifiable assumptions like AOMDL. Note that AOMDL is a weaker assumption than OMDL—if AOMDL doesn't hold, then OMDL doesn't hold either (since any OMDL adversary is also an AOMDL adversary). □

Exercise 6.8 (Optional). Why is the DL problem not hard in general?

Solution. The DL problem is not hard in general because quantum computers can solve DL in polynomial time using Shor's algorithm. □

Exercise 6.9 (Optional). Read about the Generic Group Model (GGM) in the introduction of [Sho97] and [Mau05], and argue why the DLP is hard for classical computational models.

Exercise 6.10 (Optional). How does Silent Payments [BIP352] rely on the DDH assumption?

Exercise 6.11 (Optional). Study the concept of falsifiability of cryptographic assumptions by reading Naor [Nao03]. Analyze the following assumptions and explain the extent to which each one is falsifiable:

- Factoring
- The RSA assumption
- The Decisional Diffie-Hellman assumption
- The Knowledge-of-Exponent assumption

7 Random Oracle Model

7.1 Random Oracles

A *random oracle* (RO) is an oracle that can be provided to an algorithm that takes inputs in $\{0,1\}^*$ and returns elements from some set S. The two key properties of a random oracle are: (1) it returns uniformly random and independent values from S for distinct queries, and (2) it responds consistently, meaning identical queries always yield identical responses.

Formally, a random oracle can be described as follows. Consider the random oracle H in Figure 7.1. The oracle maintains a table T to ensure consistency, where all entries are initially $\perp$ (undefined). When queried with input x, it checks whether $T[x] = \perp$. If so, it samples a uniformly random element from S and stores it as $T[x]$. The oracle returns $T[x]$.

Before examining the consequences of random oracles, we present a simple lemma that will be useful for analyzing games that differ only when certain events occur.

Lemma 7.1 (Difference Lemma). *Let A, B and E be events in a probability space. If* $\Pr[A \wedge \neg E] = \Pr[B \wedge \neg E]$*, then*

$$|\Pr[A] - \Pr[B]| \leq \Pr[E].$$

Proof.

$$\begin{aligned} |\Pr[A] - \Pr[B]| &= |\Pr[A \wedge E] + \Pr[A \wedge \neg E] - (\Pr[B \wedge E] + \Pr[B \wedge \neg E])| \\ &= |\Pr[A \wedge E] - \Pr[B \wedge E]| \end{aligned}$$

Since $\Pr[A \wedge E] \leq \Pr[E]$ and $\Pr[B \wedge E] \leq \Pr[E]$, both terms are in $[0, \Pr[E]]$, so their difference is at most $\Pr[E]$. □

Game $\mathsf{ROExample}^{\mathcal{A}}(\lambda)$	Oracle $\mathrm{H}(x)$
$T \coloneqq \emptyset$	**if** $T[x] = \bot$ **then**
$r \leftarrow\!\!\$\ \{0,1\}^{\lambda}$	$\quad T[x] \leftarrow\!\!\$\ S$
$h \coloneqq \mathrm{H}(r)$	**return** $T[x]$
return $\mathcal{A}^{\mathrm{H}}(h)$	

Fig. 7.1. Random oracle example game

Lemma 7.2. *No adversary against* ROExample *can distinguish between being given $h = \mathrm{H}(r)$ for a random r and being given a uniformly random value from S, except with probability negligible in the number of oracle queries. More precisely, let* Game $\mathsf{ROExample}^{\mathcal{A}}(\lambda)$ *be as defined in Figure 7.1. For any adversary $\mathcal{A}$ making q queries to the random oracle* H*, we have*

$$\left|\Pr[\mathsf{ROExample}^{\mathcal{A}}(\lambda) = 1] - \Pr[\mathsf{ROExample}_1{}^{\mathcal{A}}(\lambda) = 1]\right| \leq \frac{q}{2^{\lambda}}$$

where $\mathsf{ROExample}_1{}^{\mathcal{A}}(\lambda)$ *is defined in Figure 7.2.*

Game $\mathsf{ROExample}_1{}^{\mathcal{A}}(\lambda)$	Oracle $\mathrm{H}(x)$
$T \coloneqq \emptyset$	**assert** $x \neq r$
$r \leftarrow\!\!\$\ \{0,1\}^{\lambda}$	**if** $T[x] = \bot$ **then**
	$\quad T[x] \leftarrow\!\!\$\ S$
$h' \leftarrow\!\!\$\ S$	**return** $T[x]$
return $\mathcal{A}^{\mathrm{H}}(h')$	

Fig. 7.2. Game hop for random oracle indistinguishability

Proof. By inspection of the code, we observe that $\mathsf{ROExample}_1{}^{\mathcal{A}}(\lambda)$ is identical to $\mathsf{ROExample}^{\mathcal{A}}(\lambda)$ except that:

1. The input to $\mathcal{A}$ is changed from $h = \mathrm{H}(r)$ to a uniformly random $h' \leftarrow\!\!\$\ S$.
2. The oracle H asserts that no query equals r.

Since h' is uniformly random in S and has the same distribution as $\mathrm{H}(r)$ (when r is not queried), the games are identical unless the adversary queries r to the oracle.

We apply the Difference Lemma (Lemma 7.1). Let E denote the event that $\mathcal{A}$ queries r to the oracle.

When $\neg E$ occurs (i.e., r is not queried), the two games behave identically, so $\Pr[\mathsf{ROExample}^{\mathcal{A}}(\lambda) = 1 \wedge \neg E] = \Pr[\mathsf{ROExample}_1{}^{\mathcal{A}}(\lambda) = 1 \wedge \neg E]$.

By Lemma 7.1:

$$\begin{aligned}
\left|\Pr[\mathsf{ROExample}^{\mathcal{A}}(\lambda) = 1] - \Pr[\mathsf{ROExample}_1{}^{\mathcal{A}}(\lambda) = 1]\right| &\leq \Pr[E] \\
&= \Pr[\exists i \in [q] : x_i = r] \\
&\leq \sum_{i=1}^{q} \Pr[x_i = r] \qquad \text{(union bound)} \\
&= \sum_{i=1}^{q} \frac{1}{2^{\lambda}} = \frac{q}{2^{\lambda}}
\end{aligned}$$

where $x_1, \ldots, x_q$ are the queries made by $\mathcal{A}$ to H. □

7.2 ROM

For a large class of cryptographic schemes, the properties of hash functions seen so far are insufficient to prove security. Instead, we prove security of a modified scheme where hash function evaluations are replaced with random oracle calls. This is the *random oracle model* (ROM) [BR93]. Random oracles cannot be instantiated in the real world because their output must be distributed uniformly at random unless explicitly queried. On the other hand, the output of a hash function is deterministic once the key is known. Moreover, there's no way to formally define an "explicit query" for public hash functions.

There exist contrived schemes provably secure in the ROM that become insecure when instantiated with any concrete hash function (see optional exercise). Despite these theoretical concerns, the ROM has proven very useful in practice for analyzing real-world cryptographic protocols [KM15].

7.3 Hash Commitments

Definition 7.1 (Hash Commitment Scheme). *Let* H *be a hash function. The hash commitment scheme* $\mathsf{HashCom}[\mathsf{H}]$ *is defined as follows:*

- $\mathsf{Setup}(1^\lambda) \to pp$*: Output* $pp = (\mathcal{M} := \{0,1\}^*, \mathcal{R} := \{0,1\}^\lambda)$.
- $\mathsf{Commit}(pp, m, r) \to C$*: Output* $C = \mathsf{H.Eval}(m \| r)$.

Lemma 7.3. *The hash commitment scheme* $\mathsf{HashCom}[\mathsf{H}]$ *is hiding in the random oracle model for* H*. More precisely, for any algorithm* $\mathcal{A}$ *making at most* q *queries to* H*,*

$$\mathsf{Adv}^{\mathrm{hide}}_{\mathcal{A},\mathsf{HashCom}}(\lambda) \leq \frac{2q}{2^\lambda}.$$

The proof is left as an exercise (see Exercise 7.2).

7.4 Taproot Commitments

Definition 7.2 (Taproot Commitment Scheme [BIP341]). *Let* GrGen *be a group generation algorithm and* H *be a hash function with output in* $\mathbb{Z}_p$ *where* p *is the group order. The Taproot commitment scheme* $\mathsf{TapCom}[\mathsf{GrGen}, \mathsf{H}]$ *is defined as follows:*

- $\mathsf{Setup}(1^\lambda) \to pp$*: Run* $(\mathbb{G}, p, g) := \mathsf{GrGen}(1^\lambda)$ *and* $\kappa := \mathsf{H.Gen}(1^\lambda)$*. Output* $pp = ((\mathbb{G}, p, g), \kappa, \mathcal{M} := \{0,1\}^*, \mathcal{R} := \mathbb{G})$.
- $\mathsf{Commit}(pp, m, R) \to C$*: Parse* pp *to obtain* g *and* κ*. Output* $C = R \cdot g^{\mathsf{H.Eval}(\kappa,(R,m))}$.

Lemma 7.4. *Let* GrGen *be a group generation algorithm and* H *be a hash function with output in* $\mathbb{Z}_p$ *where* p *is the group order. The Taproot commitment scheme* $\mathsf{TapCom}[\mathsf{GrGen}, \mathsf{H}]$ *is binding in the random oracle model for* H*. More precisely, for any algorithm* $\mathcal{A}$ *making at most* q *queries to* H*,*

$$\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A},\mathsf{TapCom}}(\lambda) \leq \frac{(q+2)^2}{2^\lambda}.$$

The proof is left as an exercise (see Exercise 7.3).

7.5 Exercises

Exercise 7.1. Consider a modified hash commitment scheme $\mathsf{HashCom}_y[\mathsf{H}]$ defined as follows:

- $\mathsf{Setup}(1^\lambda) \to pp$: Same as $\mathsf{HashCom}[\mathsf{H}]$.
- $\mathsf{Commit}(pp, m, r) \to C$:

$$C = \begin{cases} \mathsf{H.Eval}(m) & \text{if } \mathsf{H.Eval}(0) = y \\ \mathsf{H.Eval}(m \| r) & \text{otherwise} \end{cases}$$

1. Using Lemma 7.3, argue that for all $y \in \{0,1\}^\lambda$, $\mathsf{HashCom}_y[\mathsf{H}]$ is hiding in the ROM.
2. Show that there exists y such that $\mathsf{HashCom}_y$ instantiated with SHA256 is not hiding.

Solution. 1. In the ROM, H behaves as a random oracle. For any fixed y, we have $\Pr[\mathsf{H.Eval}(0) = y] = \frac{1}{2^\lambda}$, which is negligible. With overwhelming probability, the scheme behaves identically to the standard $\mathsf{HashCom}[\mathsf{H}]$, which is hiding by Lemma 7.3.
More formally, the hiding advantage is bounded by:

$$\mathsf{Adv}^{\mathrm{hide}}_{\mathcal{A},\mathsf{HashCom}_y}(\lambda) \le \Pr[\mathsf{H.Eval}(0) = y] + \mathsf{Adv}^{\mathrm{hide}}_{\mathcal{A},\mathsf{HashCom}}(\lambda) \le \frac{1}{2^\lambda} + \frac{2q}{2^\lambda}$$

2. Let $y = \mathrm{SHA256}(0)$. When instantiated with SHA256, we always have $\mathsf{H.Eval}(0) = y$, so the commitment becomes $C = \mathrm{SHA256}(m)$ without any randomness. This is deterministic and therefore not hiding: given two messages m_0, m_1, the adversary can compute $\mathrm{SHA256}(m_0)$ and $\mathrm{SHA256}(m_1)$ and compare with C to determine which message was committed to.

□

Exercise 7.2. Prove Lemma 7.3 using a similar technique as in Lemma 7.2.

Solution. Figure 7.3 shows the hiding game for $\mathsf{HashCom}[\mathsf{H}]$ where H is modeled as a random oracle.

$\mathsf{Game\ ComHide}^{\mathcal{A}}_{\mathsf{HashCom}}(\lambda)$	$\mathrm{Oracle\ H}(x)$
$pp \coloneqq \mathsf{Setup}(1^\lambda)$	**if** $T[x] = \bot$ **then**
$(m_0, m_1) \coloneqq \mathcal{A}^{\mathrm{H}}(pp)$	$\quad T[x] \leftarrow\!\$\ \{0,1\}^\lambda$
$T \coloneqq \emptyset$	**return** $T[x]$
$b \leftarrow\!\$\ \{0,1\}$	
$r \leftarrow\!\$\ \{0,1\}^\lambda$	
$c \coloneqq \mathrm{H}(m_b \Vert r)$	
$b' \coloneqq \mathcal{A}^{\mathrm{H}}(c)$	
return $b = b'$	

Fig. 7.3. The hiding game for $\mathsf{HashCom}[\mathsf{H}]$ in the random oracle model.

Now consider the modified game shown in Figure 7.4:

$\mathsf{Game\ ComHide}_1{}^{\mathcal{A}}_{\mathsf{HashCom}}(\lambda)$	$\mathrm{Oracle\ H}(x)$
$pp \coloneqq \mathsf{Setup}(1^\lambda)$	**assert** $x \neq m_0 \Vert r$
$(m_0, m_1) \coloneqq \mathcal{A}(pp)$	**assert** $x \neq m_1 \Vert r$
$T \coloneqq \emptyset$	**if** $T[x] = \bot$ **then**
$b \leftarrow\!\$\ \{0,1\}$	$\quad T[x] \leftarrow\!\$\ \{0,1\}^\lambda$
$r \leftarrow\!\$\ \{0,1\}^\lambda$	**return** $T[x]$
$c' \leftarrow\!\$\ \{0,1\}^\lambda$	
$b' \coloneqq \mathcal{A}^{\mathrm{H}}(c')$	
return $b = b'$	

Fig. 7.4. Modified hiding game for $\mathsf{HashCom}[\mathsf{H}]$ with changes highlighted.

Note that in $\mathsf{ComHide}_1^{\mathcal{A}}{}_{\mathsf{HashCom}}(\lambda)$, the value b is independent of the adversary's view, so:

$$\Pr[\mathsf{ComHide}_1^{\mathcal{A}}{}_{\mathsf{HashCom}}(\lambda) = 1] = \frac{1}{2}$$

The games differ only when $\mathcal{A}$ queries either $m_0\|r$ or $m_1\|r$ to the oracle. By the Difference Lemma (Lemma 7.1), we can bound the difference between the games by the probability of this event:

$$\begin{aligned}
\mathsf{Adv}^{\text{hide}}_{\mathcal{A},\mathsf{HashCom}}(\lambda) &= \left|\Pr[\mathsf{ComHide}^{\mathcal{A}}_{\mathsf{HashCom}}(\lambda) = 1] - \frac{1}{2}\right| \\
&= \left|\Pr[\mathsf{ComHide}^{\mathcal{A}}_{\mathsf{HashCom}}(\lambda) = 1] - \Pr[\mathsf{ComHide}_1^{\mathcal{A}}{}_{\mathsf{HashCom}}(\lambda) = 1]\right| \\
&\leq \Pr[\exists i \in [q] : x_i \in \{m_0\|r, m_1\|r\}]
\end{aligned}$$

Since r is uniformly random from $\{0,1\}^\lambda$ and unknown to $\mathcal{A}$, and the adversary knows both m_0 and m_1, we apply the union bound:

$$\begin{aligned}
\Pr[\exists i \in [q] : x_i \in \{m_0\|r, m_1\|r\}] &\leq \sum_{i=1}^{q} \Pr[x_i = m_0\|r \text{ or } x_i = m_1\|r] \\
&\leq \sum_{i=1}^{q} (\Pr[x_i = m_0\|r] + \Pr[x_i = m_1\|r]) \\
&= \sum_{i=1}^{q} \left(\frac{1}{2^\lambda} + \frac{1}{2^\lambda}\right) \\
&= \sum_{i=1}^{q} \frac{2}{2^\lambda} = \frac{2q}{2^\lambda}
\end{aligned}$$

□

Exercise 7.3. Prove Lemma 7.4.

Solution. Let $\mathcal{A}$ be an adversary against the binding property of $\mathsf{TapCom}[\mathsf{GrGen}, \mathsf{H}]$. The adversary wins if it outputs (m_0, m_1, R_0, R_1) with $m_0 \neq m_1$ such that:

$$R_0 \cdot g^{\mathsf{H}(R_0, m_0)} = R_1 \cdot g^{\mathsf{H}(R_1, m_1)}$$

Rearranging, this equality holds if and only if:

$$R_0 \cdot g^{\mathsf{H}(R_0, m_0)} \cdot R_1^{-1} = g^{\mathsf{H}(R_1, m_1)}$$

Our strategy is to check for binding collisions as queries are made to the random oracle. Consider the distinct queries to the random oracle of the form (R_i, m_i) and define the collision predicate:

$$c(i,j) = \text{true} \iff R_i \cdot g^{\mathsf{H}(R_i, m_i)} \cdot R_j^{-1} = g^{\mathsf{H}(R_j, m_j)}$$

The probability that the adversary breaks binding is:

$$\mathsf{Adv}^{\text{bind}}_{\mathcal{A},\mathsf{TapCom}}(\lambda) = \Pr\left[\bigvee_{i=1}^{q'} \bigvee_{j=i+1}^{q'} c(i,j) = \text{true}\right]$$

where q' is the number of distinct queries.

For all $j > i$, we have $\Pr[c(i,j) = \text{true}] = \frac{1}{p}$, since $\mathsf{H}(R_j, m_j)$ is a fresh uniformly random value in $\mathbb{Z}_p$, independent of R_i, $\mathsf{H}(R_i, m_i)$, and R_j.

By the union bound:

$$\begin{aligned}
\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A},\mathsf{TapCom}}(\lambda) &= \Pr\left[\bigvee_{i=1}^{q'}\bigvee_{j=i+1}^{q'} c(i,j) = \text{true}\right] \\
&\leq \sum_{i=1}^{q'}\sum_{j=i+1}^{q'} \Pr[c(i,j) = \text{true}] \\
&= \sum_{i=1}^{q'}\sum_{j=i+1}^{q'} \frac{1}{p} \\
&= \binom{q'}{2}\cdot\frac{1}{p} < \frac{(q')^2}{2p}
\end{aligned}$$

The total number of queries includes the adversary's queries (at most q) plus the 2 queries made by the binding game when checking $C_0 = C_1$. Thus $q' \leq q + 2$.

Since $p \geq 2^{\lambda-1}$, we have:

$$\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A},\mathsf{TapCom}}(\lambda) < \frac{(q+2)^2}{2p} \leq \frac{(q+2)^2}{2\cdot 2^{\lambda-1}} = \frac{(q+2)^2}{2^\lambda}$$

□

Exercise 7.4 (Optional).

1. Let H be a hash function with output in $\{0,1\}^\lambda$ and let κ be chosen uniformly at random from a set of size 2^λ. Define $A = \mathsf{H.Eval}(\kappa, 0)\|\mathsf{H.Eval}(\kappa, 1)$. What is the size of the set over which A is uniformly distributed?
2. Let H be a random oracle with output in $\{0,1\}^\lambda$. Define $B = \mathsf{H}(0)\|\mathsf{H}(1)$. What is the size of the set over which B is uniformly distributed?

Solution. 1. There are 2^λ possibilities for κ, and each determines a unique value of A. Therefore, A is uniformly distributed over a set of size at most 2^λ.

2. Since $\mathsf{H}(0)$ and $\mathsf{H}(1)$ are independent uniformly random values in $\{0,1\}^\lambda$, B is uniformly distributed over a set of size $2^{2\lambda}$.

□

Exercise 7.5 (Optional). Let H be a random oracle with output in $\{0,1\}^\lambda$. Define the function $G : (\{0,1\}^*)^{\lambda+1} \to \{0,1\}^\lambda$ as:

$$G(x_0, x_1, \ldots, x_\lambda) = \mathsf{H}(x_0) \oplus \mathsf{H}(x_1) \oplus \cdots \oplus \mathsf{H}(x_\lambda)$$

where $\oplus$ denotes bitwise XOR. Is G preimage resistant? That is, given $y \in \{0,1\}^\lambda$, is it hard for a p.p.t. adversary with oracle access to H to find $(x_0, \ldots, x_\lambda)$ such that $G(x_0, \ldots, x_\lambda) = y$?

Solution. No, G is not preimage resistant. Wagner's algorithm for the generalized birthday problem [Wag02] can find a preimage efficiently. □

Exercise 7.6 (Optional). Why is the ROM not considered adequate to model quantum computation?

Exercise 7.7 (Optional). Study the diagonalization argument in Section 4 of Canetti, Goldreich, and Halevi [CGH98]. Explain how this argument demonstrates the existence of a contrived signature scheme that is provably secure in the random oracle model but becomes insecure when the random oracle is instantiated with any concrete hash function (not just a specific fixed hash function as considered in Exercise 7.1).

8 Recap Quiz

Exercise 8.1. Let's say you have a game $\mathsf{Game}\ G$ that you want to prove is hard, and you have the option to upper bound $\Pr[\mathrm{G}_{\mathcal{A}}(\lambda) = 1]$ by $\mathsf{Adv}^{\mathrm{aomdl}}_{\mathcal{B},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{omdl}}_{\mathcal{C},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{omdl}}_{\mathcal{D},\mathsf{GrGen}}(\lambda)$ in the ROM, or $\mathsf{Adv}^{\mathrm{dl}}_{\mathcal{E},\mathsf{GrGen}}(\lambda)$. What is your order of preference?

1. $\mathsf{Adv}^{\mathrm{dl}}_{\mathcal{E},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{aomdl}}_{\mathcal{B},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{omdl}}_{\mathcal{C},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{omdl}}_{\mathcal{D},\mathsf{GrGen}}(\lambda)$ in the ROM
2. $\mathsf{Adv}^{\mathrm{dl}}_{\mathcal{E},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{omdl}}_{\mathcal{C},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{aomdl}}_{\mathcal{B},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{omdl}}_{\mathcal{D},\mathsf{GrGen}}(\lambda)$ in the ROM
3. $\mathsf{Adv}^{\mathrm{omdl}}_{\mathcal{D},\mathsf{GrGen}}(\lambda)$ in the ROM, $\mathsf{Adv}^{\mathrm{dl}}_{\mathcal{E},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{aomdl}}_{\mathcal{B},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{omdl}}_{\mathcal{C},\mathsf{GrGen}}(\lambda)$
4. $\mathsf{Adv}^{\mathrm{aomdl}}_{\mathcal{B},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{dl}}_{\mathcal{E},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{omdl}}_{\mathcal{C},\mathsf{GrGen}}(\lambda)$, $\mathsf{Adv}^{\mathrm{omdl}}_{\mathcal{D},\mathsf{GrGen}}(\lambda)$ in the ROM

Solution. The answer is (1). □

Exercise 8.2. Consider the following game hop from $\mathsf{Game}\ \mathsf{Guess}^{\mathcal{A}}(\lambda)$ to $\mathsf{Game}\ \mathsf{Guess}_2{}^{\mathcal{A}}(\lambda)$:

$\mathsf{Game}\ \mathsf{Guess}_2{}^{\mathcal{A}}(\lambda)$

$n \coloneqq 2^\lambda$

$x \leftarrow\!\!\$\ \mathbb{Z}_n$

assert $x \neq 0$

$x' \coloneqq \mathcal{A}()$

return $x = x'$

Fig. 8.1. Without the highlighted change, this is the guessing game from Figure 1.1. With the change, this is $\mathsf{Game}\ \mathsf{Guess}_2{}^{\mathcal{A}}(\lambda)$.

Which of the following is correct?

1. $|\mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) - \Pr[\mathsf{Guess}_2{}^{\mathcal{A}}(\lambda) = 1]| \leq \Pr[x = 0]$
2. $|\mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) - \Pr[\mathsf{Guess}_2{}^{\mathcal{A}}(\lambda) = 1]| = \Pr[x = 0]$
3. $|\mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) - \Pr[\mathsf{Guess}_2{}^{\mathcal{A}}(\lambda) = 1]| \geq \Pr[x = 0]$

Solution. The answer is (1). By the Difference Lemma (Lemma 7.1), when the games are identical except when event E occurs (here E is "$x = 0$"), the difference in success probabilities is bounded by $\Pr[E]$. □

Exercise 8.3. Using the result from the previous exercise, what can we conclude about $\Pr[\mathsf{Guess}_2{}^{\mathcal{A}}(\lambda) = 1]$?

1. $\Pr[\mathsf{Guess}_2{}^{\mathcal{A}}(\lambda) = 1] \geq \mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) - 2^{-\lambda}$
2. $\Pr[\mathsf{Guess}_2{}^{\mathcal{A}}(\lambda) = 1] \geq \mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) + 2^{-\lambda}$
3. $\Pr[\mathsf{Guess}_2{}^{\mathcal{A}}(\lambda) = 1] < \mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) - 2^{-\lambda}$

Solution. The answer is (1).
From the previous exercise, we have $|\mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) - \Pr[\mathsf{Guess}_2{}^{\mathcal{A}}(\lambda) = 1]| \leq 2^{-\lambda}$.
This means: $-2^{-\lambda} \leq \mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) - \Pr[\mathsf{Guess}_2{}^{\mathcal{A}}(\lambda) = 1] \leq 2^{-\lambda}$
From the right inequality: $\mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) - \Pr[\mathsf{Guess}_2{}^{\mathcal{A}}(\lambda) = 1] \leq 2^{-\lambda}$
Rearranging: $\Pr[\mathsf{Guess}_2{}^{\mathcal{A}}(\lambda) = 1] \geq \mathsf{Adv}^{\mathrm{guess}}_{\mathcal{A}}(\lambda) - 2^{-\lambda}$ □

Exercise 8.4. Let $\mathcal{A}$ be an algorithm with access to a random oracle $\mathrm{H} : \{0,1\}^* \to \{0,1\}^\lambda$. The algorithm queries the random oracle with $x \in \{0,1\}^\lambda$. What is $\Pr[\mathrm{H}(x) = x]$?

1. 0
2. $2^{-\lambda}$

3. 1
4. $2^{-2\lambda}$

Solution. The answer is (2). The random oracle returns a uniformly random value from $\{0,1\}^\lambda$ on the first query to any input. Since there are 2^λ possible outputs and exactly one of them equals x, we have $\Pr[\mathrm{H}(x) = x] = 2^{-\lambda}$. □

Exercise 8.5. Let p be a prime and let $\mathcal{A}$ be an algorithm with access to a random oracle $\mathrm{H} : \{0,1\}^* \to \mathbb{Z}_p$. Before returning, the adversary queries the random oracle with distinct $x_1, x_2 \in \mathbb{Z}_p$ (in that order) and receives $y_1 = \mathrm{H}(x_1)$, $y_2 = \mathrm{H}(x_2)$. What is $\Pr[y_2 = x_1 y_1 x_2^{-1}]$?

1. 0
2. $1/p$
3. $1/(p-1)$
4. 1

Solution. The answer is (2).

Since $x_1 \neq x_2$, the queries are to different inputs, so y_1 and y_2 are independent uniform random values from $\mathbb{Z}_p$. Given any fixed x_1, x_2, y_1, the value $x_1 y_1 x_2^{-1}$ is a fixed element of $\mathbb{Z}_p$. Since y_2 is uniformly random and independent of y_1, we have $\Pr[y_2 = x_1 y_1 x_2^{-1}] = 1/p$. □

Exercise 8.6. Let $(\mathbb{G}, p, g)$ be a group description. Let $\mathcal{A}$ be an algorithm with access to a random oracle $\mathrm{H} : \{0,1\}^* \to \mathbb{Z}_p$ where p is prime. The adversary makes two distinct queries (R_0, m_0) and (R_1, m_1) where $R_i \in \mathbb{G}$ and $m_i \in \{0,1\}^*$, receiving $y_0 = \mathrm{H}(R_0, m_0)$ and $y_1 = \mathrm{H}(R_1, m_1)$. What is $\Pr[R_0 \cdot g^{y_0} \cdot R_1^{-1} = g^{y_1}]$?

1. 0
2. $1/p$
3. $1/(p-1)$
4. 1

Solution. The answer is (2).

Since the queries (R_0, m_0) and (R_1, m_1) are distinct, y_0 and y_1 are independent uniform random values from $\mathbb{Z}_p$.

For any fixed $R_0, R_1 \in \mathbb{G}$ and $y_0 \in \mathbb{Z}_p$, the value $R_0 \cdot g^{y_0} \cdot R_1^{-1}$ is a fixed element of $\mathbb{G}$. Since y_1 is uniformly random in $\mathbb{Z}_p$, g^{y_1} is uniformly random in $\mathbb{G}$. Therefore, $\Pr[R_0 \cdot g^{y_0} \cdot R_1^{-1} = g^{y_1}] = 1/p$.

Note: If this event occurred, we would have $R_0 \cdot g^{\mathrm{H}(R_0,m_0)} = R_1 \cdot g^{\mathrm{H}(R_1,m_1)}$, meaning

$$\mathsf{TapCom.Commit}(R_0, m_0) = \mathsf{TapCom.Commit}(R_1, m_1)$$

and the adversary would have won $\mathsf{Game\ ComBind}^{\mathcal{A}}_{\mathsf{TapCom}}(\lambda)$ in the random oracle model. In Lemma 7.4, we generalize this to q random oracle queries. □

Exercise 8.7. TODO: Insert question about reading random oracle queries (table T), which is not possible with hash functions.

Exercise 8.8. Which of the following represents a gap between security proofs and real-world security?

1. In practice, λ is a fixed value (e.g., 128 or 256)
2. Concrete hash functions like SHA256 are not hash functions
3. Unfalsifiable assumptions cannot be validated experimentally
4. Security models may not capture all attack vectors
5. Random oracles do not exist
6. Security proofs don't model users writing keys on post-it notes
7. All of the above

Solution. The answer is (7). □

9 Programmable ROM and Forking Lemma

9.1 Programmable Random Oracles

In the random oracle model, we can program the oracle to return specific values for certain inputs, as long as we maintain consistency and the correct distribution. This technique is particularly useful in security reductions.

For example, consider the hash commitment scheme $\mathsf{HashCom}[\mathsf{H}]$ where commitments are of the form $\mathsf{H}(m\|r)$. A reduction can program the oracle such that certain algebraic relations hold between committed messages, while maintaining the adversary's view.

The following proposition demonstrates that careful programming preserves the adversary's advantage. Notably, Game_1 forces the XOR of committed messages to be equal to a message m^* that is determined before even running the adversary. The game first draws m^*, runs the adversary with a random hash h and obtains a hash h' from the adversary. If the adversary created h' as $\mathsf{Commit}(m', r')$, then when the adversary is invoked a second time we have

$$\begin{aligned} \mathsf{Commit}(m, r) &= h \\ \mathsf{Commit}(m', r') &= h' \\ m \oplus m' &= m^*. \end{aligned}$$

Note that the challenger's commitment h was given to the adversary before the adversary returned commitment h' and the adversary has never revealed the opening (m', r') to the challenger.

A similar technique is used in MuSig multi-signature scheme [Max+19] to simulate signing queries (i.e., to answer signing queries without knowledge of the secret key, as explained in the next section).

$\mathsf{Game}_0{}^{\mathcal{A}}(\lambda)$

$pp \coloneqq \mathsf{HashCom.Setup}(1^\lambda)$
$T \coloneqq \emptyset$
$m \leftarrow\!\$\ \{0,1\}$
$r \leftarrow\!\$\ \{0,1\}^\lambda$
$h \coloneqq \mathrm{H}(m\|r)$
$(state, h') \coloneqq \mathcal{A}^{\mathrm{H}}(pp, h)$
return $\mathcal{A}^{\mathrm{H}}(state, m, r)$

Oracle $\mathrm{H}(x)$

if $T[x] = \bot$ **then**
  $T[x] \leftarrow\!\$\ \{0,1\}^\lambda$
return $T[x]$

$\mathsf{Game}_1{}^{\mathcal{A}}(\lambda)$

$pp \coloneqq \mathsf{HashCom.Setup}(1^\lambda)$
$T \coloneqq \emptyset$
$m^* \leftarrow\!\$\ \{0,1\}$
$h \leftarrow\!\$\ \{0,1\}^\lambda$
$(state, h') \coloneqq \mathcal{A}^{\mathrm{H}}(pp, h)$
if $\exists (m', r') : T[m'\|r'] = h'$ **then**
  Choose any such (m', r')
  $m \coloneqq m^* \oplus m'$
else
  $m \leftarrow\!\$\ \{0,1\}$
$r \leftarrow\!\$\ \{0,1\}^\lambda$
assert $T[m\|r] = \bot$
$T[m\|r] \coloneqq h$
return $\mathcal{A}^{\mathrm{H}}(state, m, r)$

Fig. 9.1. Games Game_0 and Game_1

Proposition 9.1. *Let* $\mathsf{Game}_0{}^{\mathcal{A}}(\lambda)$ *and* $\mathsf{Game}_1{}^{\mathcal{A}}(\lambda)$ *be as defined in Figure 9.1. For any adversary* $\mathcal{A}$ *making at most* q *queries to the oracle* H*, we have*

$$\left|\Pr[\mathsf{Game}_0{}^{\mathcal{A}}(\lambda) = 1] - \Pr[\mathsf{Game}_1{}^{\mathcal{A}}(\lambda) = 1]\right| \leq \frac{q}{2^\lambda}$$

The proof is left as an exercise (see Exercise 9.3).

9.2 The Forking Lemma

The forking lemma is a fundamental tool for analyzing algorithms that interact with random oracles. It considers an adversary $\mathcal{A}$ with oracle access to H, which is executed twice on different but related instances of the oracle:

- In the first execution, all random oracle responses are sampled uniformly as usual.
- In the second execution, the responses are identical to those from the first execution up to a specific query called the *forking point*, whose response is *refreshed* (resampled with fresh randomness).

As a result, the two executions of $\mathcal{A}$ proceed identically up to the forking point but diverge afterwards. The forking lemma quantifies how often an adversary that succeeds in the first execution also succeeds in the second execution, where the oracle gives a different response at the forking point.

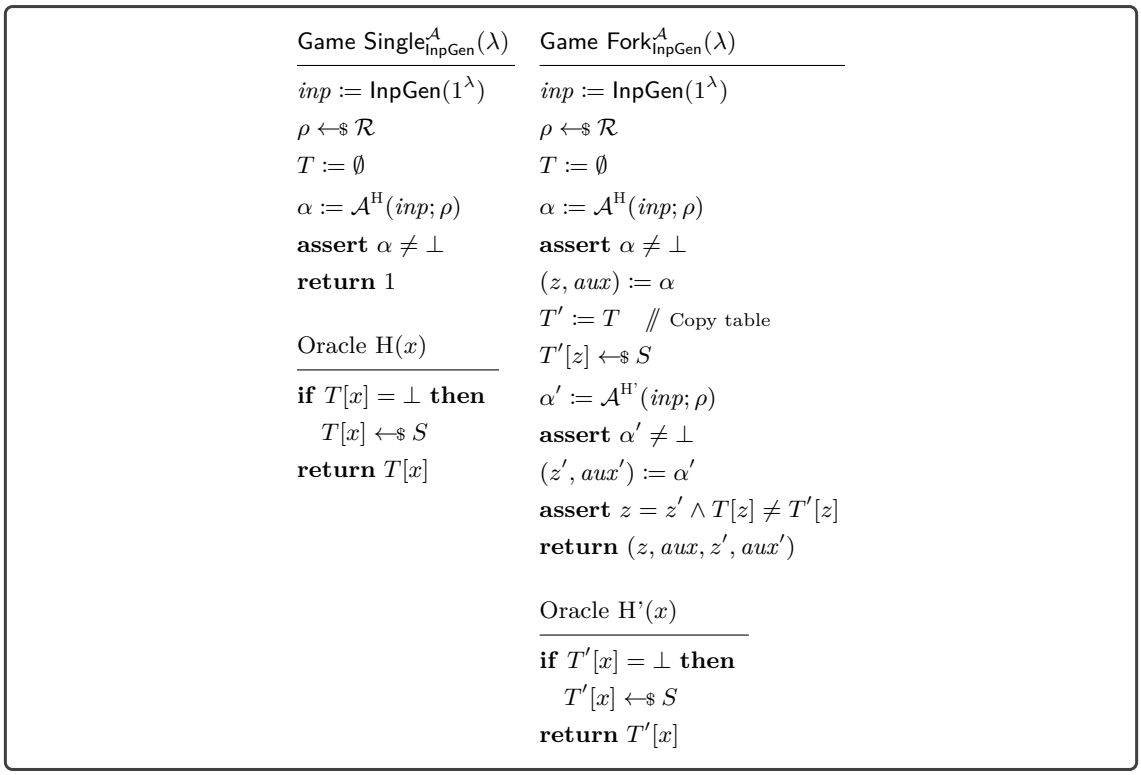

Fig. 9.2. The Single and Fork games

Lemma 9.1 (Local Forking Lemma [BDL19]). *Let $q \geq 1$ be an integer and* InpGen *be a randomized algorithm that outputs some input inp. Let $\mathcal{A}$ be a randomized algorithm that takes input inp and randomness $\rho \in \mathcal{R}$, has oracle access to $\mathrm{H} : \{0,1\}^* \to S$ where $|S| \geq 2^\lambda$, makes at most q queries to* H*, and returns either $\bot$ or a pair (z, aux) where $z \in \{0,1\}^*$ and aux is auxiliary output.*

For games $\mathsf{Single}^{\mathcal{A}}_{\mathsf{InpGen}}(\lambda)$ and $\mathsf{Fork}^{\mathcal{A}}_{\mathsf{InpGen}}(\lambda)$ defined in Figure 9.2, define:

$$\mathsf{Adv}^{\mathrm{single}}_{\mathcal{A},\mathsf{InpGen}}(\lambda) \coloneqq \Pr[\mathsf{Single}^{\mathcal{A}}_{\mathsf{InpGen}}(\lambda) = 1]$$
$$\mathsf{Adv}^{\mathrm{fork}}_{\mathcal{A},\mathsf{InpGen}}(\lambda) \coloneqq \Pr[\mathsf{Fork}^{\mathcal{A}}_{\mathsf{InpGen}}(\lambda) \neq \bot]$$

Then:

$$\mathsf{Adv}^{\mathrm{fork}}_{\mathcal{A},\mathsf{InpGen}}(\lambda) \geq \mathsf{Adv}^{\mathrm{single}}_{\mathcal{A},\mathsf{InpGen}}(\lambda) \left(\frac{\mathsf{Adv}^{\mathrm{single}}_{\mathcal{A},\mathsf{InpGen}}(\lambda)}{q} - \frac{1}{2^\lambda} \right)$$

9.3 Exercises

Exercise 9.1. Describe the similarities and differences between games Game_0 and Game_1 in Figure 9.1.

Solution. In both games:

- The oracle H behaves identically as a random oracle
- The adversary receives a commitment h and later the opening (m, r)
- The games return the same bit b output by the adversary

The key differences are:

- In Game_0: m and r are chosen first, then $h = \mathrm{H}(m\|r)$
- In Game_1: h is chosen uniformly at random, then we program the oracle so that $\mathrm{H}(m\|r) = h$
- In Game_1: If h' is a valid hash commitment (i.e., $\exists(m', r')$ such that $\mathrm{H}(m'\|r') = h'$), then $m \oplus m' = m^*$, where m^* was determined before running $\mathcal{A}$.

□

Exercise 9.2. Consider Game_F which is exactly like Game_1 except that $m^* \coloneqq 0$ (instead of sampling uniformly). Give a p.p.t. adversary $\mathcal{A}$ that makes one random oracle query and achieves

$$\left|\Pr[\mathsf{Game}_0{}^{\mathcal{A}}(\lambda) = 1] - \Pr[\mathsf{Game}_\mathrm{F}{}^{\mathcal{A}}(\lambda) = 1]\right| = \frac{1}{2}.$$

Solution. Consider the following adversary $\mathcal{A}$:

First invocation:

- Query $h' \coloneqq \mathrm{H}(0\|0^\lambda)$
- Output $(\bot, h')$

Second invocation:

- Output m

Analysis:

- In Game_0: m is uniform, so $\Pr[\mathsf{Game}_0{}^{\mathcal{A}}(\lambda) = 1] = \frac{1}{2}$.
- In Game_F: Since $m^* = 0$ (fixed in Game_F) and $m' = 0$, we have $m = m^* \oplus m' = 0 \oplus 0 = 0$. Thus $\Pr[\mathsf{Game}_\mathrm{F}{}^{\mathcal{A}}(\lambda) = 1] = 0$.

Therefore: $\left|\Pr[\mathsf{Game}_0{}^{\mathcal{A}}(\lambda) = 1] - \Pr[\mathsf{Game}_\mathrm{F}{}^{\mathcal{A}}(\lambda) = 1]\right| = \left|\frac{1}{2} - 0\right| = \frac{1}{2}$. □

Exercise 9.3. Sketch a proof of Proposition 9.1. **Hints:**

1. Let E be the event that the adversary queries $m\|r$ to the oracle during its first invocation (i.e., the event that triggers the assertion). Argue that $\Pr[\mathsf{Game}_0{}^{\mathcal{A}}(\lambda) = 1 \wedge \neg E] = \Pr[\mathsf{Game}_1{}^{\mathcal{A}}(\lambda) = 1 \wedge \neg E]$.
2. Bound the probability of E.

Solution. Let E be the event that the adversary queries $m\|r$ to the oracle during its first invocation (when it outputs $(state, h')$).

We claim that $\Pr[\mathsf{Game}_0{}^{\mathcal{A}}(\lambda) = 1 \wedge \neg E] = \Pr[\mathsf{Game}_1{}^{\mathcal{A}}(\lambda) = 1 \wedge \neg E]$. To see this, we analyze the distribution of all values conditioned on $\neg E$:

- **The value h:** In Game_0, $h = \mathrm{H}(m\|r)$ where the oracle uses lazy sampling. Since $\neg E$ occurs, the adversary never queried $m\|r$ during the first invocation, so h is a fresh uniform sample. In Game_1, h is explicitly drawn uniformly at random. Thus h is uniform in $\{0,1\}^\lambda$ in both games.
- **The value r:** In both games, r is drawn uniformly from $\{0,1\}^\lambda$.
- **The value m:** In Game_0, m is drawn uniformly from $\{0,1\}$. In Game_1:
  - If h' is a valid commitment: $m = m^* \oplus m'$ where m^* is uniform in $\{0,1\}$ and independent of the adversary's view

- If h' is not a valid commitment: m is drawn uniformly from $\{0,1\}$

In both cases, m is uniform in $\{0,1\}$.

– **The oracle programming:** In Game_1, we set $T[m\|r] = h$. Since $\neg E$ occurs, the input $m\|r$ is fresh (never queried before), so this programming is invisible to the adversary and maintains consistency with $h = \mathrm{H}(m\|r)$.

Since all values have identical distributions and the oracle behaves identically in both games (returning the same values for the same queries), the adversary's view is bit-by-bit identical conditioned on $\neg E$. Therefore $\Pr[\mathsf{Game}_0{}^{\mathcal{A}}(\lambda) = 1 \wedge \neg E] = \Pr[\mathsf{Game}_1{}^{\mathcal{A}}(\lambda) = 1 \wedge \neg E]$.

By Lemma 7.1, we have:

$$\left|\Pr[\mathsf{Game}_0{}^{\mathcal{A}}(\lambda) = 1] - \Pr[\mathsf{Game}_1{}^{\mathcal{A}}(\lambda) = 1]\right| \leq \Pr[E]$$

To bound $\Pr[E]$, note that E occurs if the adversary queries $m\|r$ during the first invocation. The key observation is that $r \in \{0,1\}^\lambda$ is chosen uniformly at random and independently of the adversary's first invocation:

– In Game_0: r is chosen at the beginning but is unknown to the adversary
– In Game_1: r is chosen after the first invocation

In both cases, for any specific query x_i made by the adversary during the first invocation, the probability that $x_i = m\|r$ is at most $\frac{1}{2^\lambda}$ (the adversary must guess the λ-bit value r correctly).

Since the adversary makes at most q queries during the first invocation, by the union bound:

$$\Pr[E] \leq \sum_{i=1}^{q} \Pr[x_i = m\|r] \leq \sum_{i=1}^{q} \frac{1}{2^\lambda} = \frac{q}{2^\lambda}$$

Therefore:

$$\left|\Pr[\mathsf{Game}_0{}^{\mathcal{A}}(\lambda) = 1] - \Pr[\mathsf{Game}_1{}^{\mathcal{A}}(\lambda) = 1]\right| \leq \frac{q}{2^\lambda}$$

□

Exercise 9.4. Consider an algorithm $\mathcal{A}$ that never queries H. What is $\mathsf{Adv}^{\mathrm{fork}}_{\mathcal{A},\mathsf{InpGen}}(\lambda)$?

Solution. If $\mathcal{A}$ never queries H, then the table T remains empty throughout the first execution. In the Fork game:

– First execution: $\mathcal{A}$ outputs $\alpha = (z, aux)$ with probability $\mathsf{Adv}^{\mathrm{single}}_{\mathcal{A},\mathsf{InpGen}}(\lambda)$
– Since z was never queried, $T[z] = \perp$ (undefined)
– We copy T to T', so $T'[z] = \perp$ as well
– We then set $T'[z] \leftarrow\!\!\$\ S$ (a fresh random value)
– Second execution: Since $\mathcal{A}$ receives the same *inp* and ρ, and never queries the oracle, it outputs the same (z, aux) deterministically
– The condition $z = z'$ is satisfied
– The condition $T[z] \neq T'[z]$ is satisfied since $T[z] = \perp$ while $T'[z] \in S$

Therefore:

$$\mathsf{Adv}^{\mathrm{fork}}_{\mathcal{A},\mathsf{InpGen}}(\lambda) = \mathsf{Adv}^{\mathrm{single}}_{\mathcal{A},\mathsf{InpGen}}(\lambda)$$

□

Exercise 9.5. Consider an algorithm $\mathcal{A}$ that makes exactly one query $z \coloneqq \mathrm{H}(\rho)$ and outputs $(z, \perp)$. What is $\mathsf{Adv}^{\mathrm{fork}}_{\mathcal{A},\mathsf{InpGen}}(\lambda)$?

Solution. In this case, $\mathcal{A}$ always succeeds in outputting a non-$\perp$ value, so $\mathsf{Adv}^{\mathrm{single}}_{\mathcal{A},\mathsf{InpGen}}(\lambda) = 1$. In the Fork game:

– First execution: $\mathcal{A}$ queries $z \coloneqq \mathrm{H}(\rho)$ and outputs $(z, \perp)$. Since ρ is fixed, z is determined by the oracle's response.
– We copy T to T', so $T'[\rho] = T[\rho] = z$

– We then set $T'[z] \leftarrow\!\!\$\ S$ (refreshing the value at the forking point)
– Second execution: $\mathcal{A}$ queries $z' \coloneqq \mathrm{H}'(\rho) = T'[\rho] = z$ (unchanged) and outputs $(z', \perp) = (z, \perp)$
– The condition $z = z'$ is satisfied
– The condition $T[z] \neq T'[z]$ is satisfied with probability $1 - \frac{1}{|S|} \geq 1 - \frac{1}{2^\lambda}$ (since $T'[z]$ was refreshed to a random value)

Therefore:

$$\mathsf{Adv}^{\mathrm{fork}}_{\mathcal{A},\mathsf{InpGen}}(\lambda) = 1 \cdot \left(1 - \frac{1}{2^\lambda}\right) = 1 - \frac{1}{2^\lambda}$$

This matches the forking lemma bound when $q = 1$ and $\mathsf{Adv}^{\mathrm{single}}_{\mathcal{A},\mathsf{InpGen}}(\lambda) = 1$:

$$1 \cdot \left(\frac{1}{1} - \frac{1}{2^\lambda}\right) = 1 - \frac{1}{2^\lambda}$$

□

Exercise 9.6. Let InpGen output $inp = (\mathbb{G}, p, g)$ where g is a generator of a cyclic group $\mathbb{G}$ of prime order $p \geq 2^{\lambda-1}$. Let $\mathcal{A}$ be an algorithm that makes at most q queries to a random oracle $\mathrm{H} : \{0,1\}^* \to \mathbb{Z}_p$ and always succeeds in outputting (z, s) such that $s = z + \mathrm{H}(z) \cdot x \bmod p$ for some fixed $x \in \mathbb{Z}_p$. Using the forking lemma, give a p.p.t. algorithm $\mathcal{B}$ that outputs x and bound its success probability.

Solution. We construct a p.p.t. algorithm $\mathcal{B}$ that runs the forking game and extracts x from the outputs. If $\mathsf{Fork}^{\mathcal{A}}_{\mathsf{InpGen}}(\lambda)$ returns $(z, s, z', s') \neq \perp$, then we have:

– From the first execution: $(z, s) = \alpha$ where $s = z + T[z] \cdot x \bmod p$
– From the second execution: $(z', s') = \alpha'$ where $s' = z' + T'[z'] \cdot x \bmod p$
– The conditions $z = z'$ and $T[z] \neq T'[z]$ hold

Therefore: $s = z + T[z] \cdot x$ and $s' = z + T'[z] \cdot x \pmod p$.

Subtracting yields $s - s' = (T[z] - T'[z]) \cdot x \pmod p$. Since $T[z] \neq T'[z]$ and p is prime, we can compute:

$$x = (s - s') \cdot (T[z] - T'[z])^{-1} \bmod p$$

Since $\mathcal{A}$ always succeeds, $\mathsf{Adv}^{\mathrm{single}}_{\mathcal{A},\mathsf{InpGen}}(\lambda) = 1$. By the forking lemma:

$$\Pr[\mathcal{B} \text{ outputs } x] = \mathsf{Adv}^{\mathrm{fork}}_{\mathcal{A},\mathsf{InpGen}}(\lambda) \geq 1 \cdot \left(\frac{1}{q} - \frac{1}{2^\lambda}\right) = \frac{1}{q} - \frac{1}{2^\lambda}$$

□

Exercise 9.7. Consider an algorithm $\mathcal{A}$ that queries $h_0 \coloneqq \mathrm{H}(0)$ and outputs $(0, \perp)$ if $h_0 \bmod 2 = 0$, otherwise outputs $(h_0, \perp)$. What is $\mathsf{Adv}^{\mathrm{fork}}_{\mathcal{A},\mathsf{InpGen}}(\lambda)$?

Solution. First, note that $\mathsf{Adv}^{\mathrm{single}}_{\mathcal{A},\mathsf{InpGen}}(\lambda) = 1$ since $\mathcal{A}$ always outputs a non-$\perp$ value.

The adversary only queries $\mathrm{H}(0)$, so after the first execution, $T[0] = h_0$ and all other entries remain $\perp$.

Case 1: $h_0 \bmod 2 = 0$ (happens with probability $\frac{1}{2}$)

– First execution: $\mathcal{A}$ outputs $(0, \perp)$
– We set $T'[0] \leftarrow\!\!\$\ S$ (refreshing the value at key 0)
– Second execution: $\mathcal{A}$ queries $h'_0 = T'[0]$ (the refreshed value)
– If $h'_0 \bmod 2 = 0$: outputs $(0, \perp)$, so $z = z' = 0$ and $T[0] = h_0 \neq h'_0 = T'[0]$ with probability $1 - \frac{1}{2^\lambda}$
– If $h'_0 \bmod 2 = 1$: outputs $(h'_0, \perp)$, so $z = 0 \neq h'_0 = z'$ (fork fails)
– Fork succeeds with probability $\frac{1}{2} \cdot (1 - \frac{1}{2^\lambda})$

Case 2: $h_0 \bmod 2 = 1$ (happens with probability $\frac{1}{2}$)

– First execution: $\mathcal{A}$ outputs $(h_0, \perp)$

– Since h_0 was never queried, $T[h_0] = \perp$
– We set $T'[h_0] \leftarrow\!\$\ S$
– Second execution: $\mathcal{A}$ queries $h_0' = T'[0] = T[0] = h_0$ (unchanged)
– Since $h_0 \bmod 2 = 1$, outputs $(h_0, \perp)$
– So $z = z' = h_0$ and $T[h_0] = \perp \neq T'[h_0]$
– Fork succeeds with probability 1

Therefore:

$$\mathsf{Adv}^{\mathrm{fork}}_{\mathcal{A},\mathsf{InpGen}}(\lambda) = \frac{1}{2} \cdot \frac{1}{2} \cdot \left(1 - \frac{1}{2^\lambda}\right) + \frac{1}{2} \cdot 1 = \frac{1}{4} - \frac{1}{2^{\lambda+2}} + \frac{1}{2} \approx \frac{3}{4}$$

□

Exercise 9.8 (Optional). Find an algorithm $\mathcal{A}$ making exactly q queries for which $\mathsf{Adv}^{\mathrm{fork}}_{\mathcal{A},\mathsf{InpGen}}(\lambda)$ is as close to $\mathsf{Adv}^{\mathrm{single}}_{\mathcal{A},\mathsf{InpGen}}(\lambda)\left(\frac{\mathsf{Adv}^{\mathrm{single}}_{\mathcal{A},\mathsf{InpGen}}(\lambda)}{q} - \frac{1}{2^\lambda}\right)$ as possible.

Exercise 9.9 (Optional). Compare the Generalized Forking Lemma of Bellare and Neven [BN06] with the Local Forking Lemma. What are the key differences in their formulations and applications?

10 Signatures

10.1 Syntax & Correctness

Definition 10.1 (Signature Scheme). *A signature scheme* Sig *is a tuple of polynomial-time algorithms* $(\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ *where:*

– $\mathsf{Setup}(1^\lambda) \to pp$ *is a probabilistic algorithm that takes the security parameter as input and outputs public parameters* pp *including the message space* $\mathcal{M}$ *and the signature space* $\mathcal{S}$*.*
– $\mathsf{KeyGen}(pp) \to (sk, pk)$ *is a probabilistic algorithm that takes public parameters* pp *as input and outputs a secret key* sk *and a public key* pk*.*
– $\mathsf{Sign}(pp, sk, m) \to \sigma$ *is a probabilistic algorithm that takes public parameters* pp*, a secret key* sk*, and a message* $m \in \mathcal{M}$ *as input and outputs a signature* $\sigma \in \mathcal{S}$*.*
– $\mathsf{Verify}(pp, pk, m, \sigma) \to \{0, 1\}$ *is a deterministic algorithm that takes public parameters* pp*, a public key* pk*, a message* $m \in \mathcal{M}$*, and a signature* $\sigma \in \mathcal{S}$ *as input and outputs* 1 *(accept) if the signature is valid, and outputs* 0 *(reject) otherwise.*

Definition 10.2 (Correctness). *A signature scheme* $\mathsf{Sig} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ *is* correct *if for all* $\lambda \in \mathbb{N}$*, all* $pp \in \mathsf{Setup}(1^\lambda)$*, all* $(sk, pk) \in \mathsf{KeyGen}(pp)$*, and all messages* $m \in \mathcal{M}$*:*

$$\Pr[\mathsf{Verify}(pp, pk, m, \mathsf{Sign}(pp, sk, m)) = 1] = 1$$

where the probability is taken over the randomness of Sign*.*

In other words, a signature scheme is correct if honestly generated signatures always verify correctly.

10.2 Security

The standard security notion for digital signatures is existential unforgeability under chosen message attack (EUF-CMA). Unlike one-time signatures, the adversary can request signatures on multiple messages.

Definition 10.3 (EUF-CMA Security). *A signature scheme* Sig *is* existentially unforgeable under chosen message attack (EUF-CMA) *if for all p.p.t. adversaries* $\mathcal{A}$*:*

$$\mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{Sig}}(\lambda) \coloneqq \Pr\left[\mathsf{Game\ EUF\text{-}CMA}^{\mathcal{A}}_{\mathsf{Sig}}(\lambda) = 1\right] = negl(\lambda)$$

where $\mathsf{Game\ EUF\text{-}CMA}^{\mathcal{A}}_{\mathsf{Sig}}(\lambda)$ *is defined in Figure 10.1.*

Game $\mathsf{EUF\text{-}CMA}^{\mathcal{A}}_{\mathsf{Sig}}(\lambda)$	Oracle $\textsc{Sign}(m)$
$pp \coloneqq \mathsf{Setup}(1^\lambda)$	$\sigma \coloneqq \mathsf{Sign}(pp, sk, m)$
$(sk, pk) \coloneqq \mathsf{KeyGen}(pp)$	$\mathcal{Q} \coloneqq \mathcal{Q} \cup \{m\}$
$\mathcal{Q} \coloneqq \emptyset$	**return** σ
$(m^*, \sigma^*) \coloneqq \mathcal{A}^{\textsc{Sign}}(pp, pk)$	
return $m^* \notin \mathcal{Q} \wedge$	
$\quad \mathsf{Verify}(pp, pk, m^*, \sigma^*) = 1$	

Fig. 10.1. The EUF-CMA security game for digital signatures

10.3 Schnorr Signatures

Definition 10.4 (Schnorr Signature Scheme). *Let* GrGen *be a group generation algorithm and* H *be a hash function with output space* $\mathbb{Z}_p$*. The Schnorr signature scheme* $\mathsf{SchnorrSig}[\mathsf{GrGen}, \mathsf{H}]$ *is defined as follows:*

- $\mathsf{Setup}(1^\lambda) \to pp$*: Run* $(\mathbb{G}, p, g) \coloneqq \mathsf{GrGen}(1^\lambda)$ *and* $\kappa \coloneqq \mathsf{H.Gen}(1^\lambda)$*. The message space is* $\mathcal{M} = \{0,1\}^*$ *and the signature space is* $\mathcal{S} = \mathbb{G} \times \mathbb{Z}_p$*. Output* $pp = ((\mathbb{G}, p, g), \kappa, \mathcal{M}, \mathcal{S})$*.*
- $\mathsf{KeyGen}(pp) \to (sk, pk)$*: Sample* $x \leftarrow\!\!\$\, \mathbb{Z}_p$*. Compute* $X = g^x$*. Output* $sk = x$ *and* $pk = X$*.*
- $\mathsf{Sign}(pp, sk, m) \to \sigma$*: Parse* $pp = ((\mathbb{G}, p, g), \kappa, \mathcal{M}, \mathcal{S})$ *and* $sk = x$*. Sample* $r \leftarrow\!\!\$\, \mathbb{Z}_p$*. Compute:*

$$\begin{aligned} R &= g^r \\ c &= \mathsf{H.Eval}(\kappa, (R, m)) \\ s &= r + c \cdot x \bmod p \end{aligned}$$

  Output $\sigma = (R, s)$*.*
- $\mathsf{Verify}(pp, pk, m, \sigma) \to \{0,1\}$*: Parse* $pp = ((\mathbb{G}, p, g), \kappa, \mathcal{M}, \mathcal{S})$*,* $pk = X$ *and* $\sigma = (R, s)$*. Compute* $c = \mathsf{H.Eval}(\kappa, (R, m))$*. Accept if and only if:*

$$g^s = R \cdot X^c$$

Lemma 10.1 (Correctness of Schnorr Signatures). *The Schnorr signature scheme* $\mathsf{SchnorrSig}[\mathsf{GrGen}, \mathsf{H}]$ *is correct.*

Proof. Let $\sigma = (R, s)$ be a signature generated by $\mathsf{Sign}(pp, sk, m)$ where $sk = x$ and $pk = X = g^x$. By the signing algorithm, we have $R = g^r$ for some $r \leftarrow\!\!\$\, \mathbb{Z}_p$, $c = \mathsf{H.Eval}(\kappa, (R, m))$, and $s = r + c \cdot x \bmod p$. The verification equation holds because:

$$g^s = g^{r+cx} = g^r \cdot g^{cx} = R \cdot (g^x)^c = R \cdot X^c$$

Therefore, $\mathsf{Verify}(pp, pk, m, \sigma) = 1$ for all honestly generated signatures. □

Remark 10.1. In the security analysis that follows, we work in the Random Oracle Model where the hash function $\mathsf{H.Eval}(\kappa, \cdot)$ is modeled as a truly random function. In the ROM analysis, we write $\mathsf{H}(\cdot)$ instead of $\mathsf{H.Eval}(\kappa, \cdot)$, with the understanding that H represents a random oracle.

Lemma 10.2. *Let* $\mathcal{A}$ *be an adversary against the EUF-CMA security of* $\mathsf{SchnorrSig}[\mathsf{GrGen}, \mathsf{H}]$ *in the random oracle model for* H *making at most* q_s *queries to* $\textsc{Sign}$ *and at most* q_h *queries to* H*.*

Let InpGen *be the algorithm which on input* 1^λ *runs* $(\mathbb{G}, p, g) \coloneqq \mathsf{GrGen}(1^\lambda)$*, draws* $X \leftarrow\!\!\$\, \mathbb{G}$*, and returns* $((\mathbb{G}, p, g), X)$*.*

Consider algorithm $\mathcal{C}$ *defined in Figure 10.2 that takes as input* $((\mathbb{G}, p, g), X) \coloneqq \mathsf{InpGen}(1^\lambda)$ *and has access to a random oracle* H*. Then* $\mathcal{C}$ *makes at most* $q_h + 1$ *queries to* H*, and satisfies*

$$\left| \mathsf{Adv}^{\text{single}}_{\mathcal{C}, \mathsf{InpGen}}(\lambda) - \mathsf{Adv}^{\text{euf-cma}}_{\mathcal{A}, \mathsf{SchnorrSig}[\mathsf{GrGen}, \mathsf{H}]}(\lambda) \right| \leq \frac{2q_s(q_s + q_h)}{2^\lambda}$$

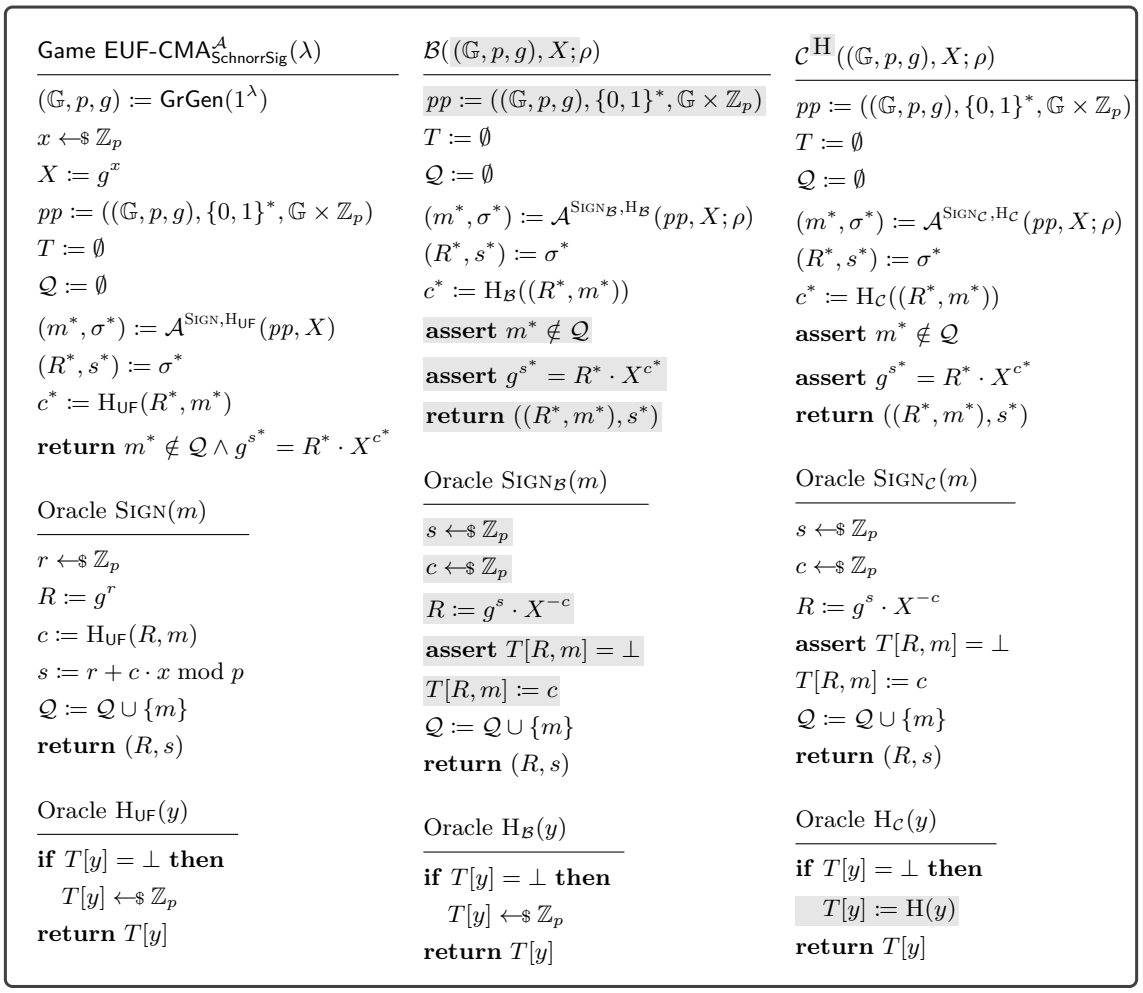

Fig. 10.2. Security reduction for Schnorr signatures: EUF-CMA game (left), algorithm $\mathcal{B}$ (middle), and algorithm $\mathcal{C}$ (right). Changes from the previous game are marked with $\cdot$.

with $\mathsf{Adv}^{\text{single}}_{\mathcal{C},\mathsf{InpGen}}(\lambda)$ *as defined in the forking lemma (Lemma 9.1).*

Moreover, when $\mathcal{C}$ *returns a non-*$\perp$ *output* (z, aux)*, then* $z = (R^*, m^*)$*,* $aux = s^*$*, and*

$$g^{s^*} = R^* \cdot X^{\mathsf{H}(R^*,m^*)}$$

The proof is left as an exercise (see Exercise 10.3).

Theorem 10.1 (Schnorr signatures are EUF-CMA secure). *Let* GrGen *be a group generation algorithm for which the discrete logarithm problem is hard and let* H *be a hash function with output space* $\mathbb{Z}_p$*. Then the Schnorr signature scheme* $\mathsf{SchnorrSig}[\mathsf{GrGen}, \mathsf{H}]$ *is EUF-CMA secure in the random oracle model for* H*. More precisely, for any p.p.t. adversary* $\mathcal{A}$ *against the EUF-CMA security of* $\mathsf{SchnorrSig}[\mathsf{GrGen}, \mathsf{H}]$ *making at most* q_s *queries to the signing oracle and at most* q_h *queries to the random oracle* H*, there exists a p.p.t. adversary* $\mathcal{D}$ *against the discrete logarithm problem such that*

$$\mathsf{Adv}^{\text{euf-cma}}_{\mathcal{A},\mathsf{SchnorrSig}[\mathsf{GrGen},\mathsf{H}]}(\lambda) \leq \frac{2q_s(q_s+q_h)}{2^\lambda} + \sqrt{(q_h+1)\left(\mathsf{Adv}^{\text{dlog}}_{\mathcal{D},\mathsf{GrGen}}(\lambda) + \frac{1}{2^\lambda}\right)}.$$

The proof is left as an exercise (see Exercise 10.4).

10.4 Exercises

Exercise 10.1. The EUF-CMA security definition in Definition 10.3 only requires the forged message to be new. An alternative notion called *strong EUF-CMA* would allow the adversary to win if they produce a different signature for a message even if it was previously queried. Define a strong EUF-CMA security game that captures this intuition.

Solution. For strong EUF-CMA security, we modify the signing oracle to maintain a set of message-signature pairs:

- Initialize $\mathcal{Q} := \emptyset$
- In oracle $\textsc{Sign}(m)$: compute $\sigma := \mathsf{Sign}(pp, sk, m)$, set $\mathcal{Q} := \mathcal{Q} \cup \{(m, \sigma)\}$, and return σ
- Winning condition: $(m^*, \sigma^*) \notin \mathcal{Q} \wedge \mathsf{Verify}(pp, pk, m^*, \sigma^*) = 1$

The key difference from standard EUF-CMA is that $\mathcal{Q}$ stores pairs (m, σ) rather than just messages, and the adversary must output a pair that was never returned by the signing oracle, even if the message component was queried before. □

Exercise 10.2. In the Generic Group Model (GGM) [Sho97], the advantage of solving the discrete logarithm problem with q group operations is bounded by $\mathsf{Adv}^{\text{dlog}}_{\mathcal{A},\mathsf{GrGen}}(\lambda) \leq \frac{q^2}{2p}$ where p is the prime group order.

Consider Schnorr signatures over a 256-bit elliptic curve group (e.g., secp256k1 where $p > 2^{255}$). Suppose an adversary can make:

- $q_s = 2^{10}$ signing queries
- $q_h = 2^{85}$ hash queries
- $q = 2^{85}$ group operations

Calculate the concrete EUF-CMA advantage using the bound from Theorem 10.1 with $\lambda = 256$, and compare it to the discrete logarithm advantage $\mathsf{Adv}^{\text{dlog}}_{\mathcal{A},\mathsf{GrGen}}(\lambda)$ in the GGM.

Solution. From Theorem 10.1:

$$\mathsf{Adv}^{\text{euf-cma}}_{\mathcal{A},\mathsf{SchnorrSig}}(\lambda) \leq \frac{q_s(q_s+q_h)}{2^\lambda} + \sqrt{(q_h+1)\left(\mathsf{Adv}^{\text{dlog}}_{\mathcal{D},\mathsf{GrGen}}(\lambda) + \frac{1}{2^\lambda}\right)}$$

First, let's calculate the discrete logarithm advantage in the GGM:

$$\mathsf{Adv}^{\text{dlog}}_{\mathcal{A},\mathsf{GrGen}}(\lambda) \leq \frac{q^2}{2p} = \frac{(2^{85})^2}{2^{257}} = \frac{2^{170}}{2^{257}} = 2^{-87}$$

Now for the EUF-CMA advantage. First term:

$$\frac{q_s(q_s+q_h)}{2^\lambda} \approx \frac{2^{10}\cdot 2^{85}}{2^{256}} = \frac{2^{95}}{2^{256}} = 2^{-161}$$

Second term:

$$\sqrt{(q_h+1)\left(\mathsf{Adv}^{\mathrm{dlog}}_{\mathcal{D},\mathsf{GrGen}}(\lambda)+\frac{1}{2^\lambda}\right)} \approx \sqrt{2^{85}\cdot 2^{-87}} = \sqrt{2^{-2}} = 2^{-1} = 0.5$$

Therefore:

$$\mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{SchnorrSig}}(\lambda) \leq 2^{-161} + 0.5 \approx 0.5$$

Comparison: The discrete logarithm advantage is 2^{-87}, while the EUF-CMA advantage for Schnorr signatures is approximately 2^{-1}.

This dramatic difference—losing over 86 bits of security—is due to the square root loss in the forking lemma.

Remark: In the Algebraic Group Model (AGM) [FKL18], Schnorr signatures can be proven secure with a much tighter reduction. The AGM assumes that whenever an adversary outputs a group element, it must also provide an algebraic representation in terms of previously seen group elements. Because the AGM proof doesn't require rewinding (unlike the forking lemma), it avoids the square root loss entirely, achieving security that nearly matches the hardness of discrete logarithm. □

Exercise 10.3. Prove Lemma 10.2 using the games in Figure 10.2.

Solution. We prove the lemma by a sequence of game hops between the three games in Figure 10.2.

From Game EUF-CMA *to* $\mathsf{Single}^{\mathcal{B}}_{\mathsf{InpGen}}(\lambda)$. The main differences are:

- $\mathcal{B}$ receives $((\mathbb{G},p,g),X)$ from $\mathsf{InpGen}(1^\lambda)$ instead of generating the key pair (x,X).
- The signing oracle $\textsc{Sign}_{\mathcal{B}}$ simulates signing without knowing x by programming the random oracle.
- $\mathcal{B}$ explicitly asserts the winning conditions of Game EUF-CMA, and outputs $((R^*,m^*),s^*)$.

Let E be the event that $\mathcal{B}$ aborts during a signing oracle query(i.e., $T_{\mathcal{B}}[Rm] \neq \bot$). We now prove that that $\Pr\left[\text{Game EUF-CMA}^{\mathcal{A}}_{\mathsf{SchnorrSig}}(\lambda)=1\right]$ is equal to $\Pr\left[\mathsf{Single}^{\mathcal{B}}_{\mathsf{InpGen}}(\lambda)=1\right]$ if E does not occur.

- $\mathsf{InpGen}(1^\lambda)$ outputs $((\mathbb{G},p,g),X)$ where $X \leftarrow\!\!\$\ \mathbb{G}$, which has the same distribution as the key generation in Game EUF-CMA$^{\mathcal{A}}_{\mathsf{SchnorrSig}}(\lambda)$ since $X=g^x$ for uniformly random $x \leftarrow\!\!\$\ \mathbb{Z}_p$.
- The signatures output by the signing oracle have the same distribution in both games when event E does not occur. To see why, let's examine how signatures are generated:
  - In Game EUF-CMA$^{\mathcal{A}}_{\mathsf{SchnorrSig}}(\lambda)$: Sample $r \leftarrow\!\!\$\ \mathbb{Z}_p$, compute $R=g^r$, set $c=\mathsf{H}(R,m)$, and output (R,s) where $s=r+x\cdot c$
  - In $\mathcal{B}$: Sample $(c,s) \leftarrow\!\!\$\ \mathbb{Z}_p\times\mathbb{Z}_p$, compute $R=g^s\cdot X^{-c}$, program $T_{\mathcal{B}}[(R,m)]=c$, and output (R,s)

  Both methods produce signatures satisfying the verification equation $g^s = R\cdot X^c$. The key insight is that:
  - In the EUF-CMA game: Starting from uniform r, we get uniform $R\in\mathbb{G}$ and compute s deterministically
  - In $\mathcal{B}$: Starting from uniform (c,s), we compute R deterministically to satisfy the same equation

  Since there's a bijection between the randomness and valid signatures in both cases, and the oracle programming ensures $c=\mathsf{H}(R,m)$ when queried, both games produce identical signature distributions when E does not occur.

– The programmed oracle $\mathrm{H}_{\mathcal{B}}$ is indistinguishable from the real random oracle H when E does not occur. The oracle $\mathrm{H}_{\mathcal{B}}$ returns either programmed values (for queries (R, m) where (R, s) was output by $\textsc{Sign}_{\mathcal{B}}(m)$) or fresh random values from H. Since programmed values are chosen uniformly at random when signatures are created, all oracle responses are uniformly distributed in $\mathbb{Z}_p$. The adversary can only distinguish between $\mathrm{H}_{\mathcal{B}}$ and H if it queries $\mathrm{H}_{\mathcal{B}}((R, m))$ before $\textsc{Sign}_{\mathcal{B}}(m)$ happens to generate the same R – but this is precisely event E.

Thus we conclude that

$$\Pr\left[\mathsf{Game\ EUF\text{-}CMA}^{\mathcal{A}}_{\mathsf{SchnorrSig}}(\lambda) = 1 \wedge \neg E\right] = \Pr\left[\mathsf{Single}^{\mathcal{B}}_{\mathsf{InpGen}}(\lambda) = 1 \wedge \neg E\right]$$

and by the Difference Lemma (Lemma 7.1) we have

$$|\Pr\left[\mathsf{Game\ EUF\text{-}CMA}^{\mathcal{A}}_{\mathsf{SchnorrSig}}(\lambda) = 1\right] - \Pr\left[\mathsf{Single}^{\mathcal{B}}_{\mathsf{InpGen}}(\lambda) = 1\right]| \leq \Pr[E]$$

.

To bound the probability of event E, we first observe that when event E occurs there exists a key (R', m') in $T_{\mathcal{B}}$ such that R' is equal to value R computed in the signing oracle. The probability that R equals a specific R' is $1/|\mathbb{G}|$ since R is uniformly distributed in $\mathbb{G}$: for any $R \in \mathbb{G}$ and any $s \in \mathbb{Z}_p$, there is exactly one $c \in \mathbb{Z}_p$ such that $R = g^s \cdot X^{-c}$. Moreover, at the time of each signing query, the table $T_{\mathcal{B}}$ contains at most $q_s + q_h$ entries (from previous signing queries and hash queries). Therefore, the probability that (R, m) matches any existing entry in $T_{\mathcal{B}}$ is at most $\frac{q_s+q_h}{|\mathbb{G}|}$.

By union bound over all q_s signing queries:

$$\Pr[E] \leq \frac{q_s(q_s + q_h)}{|\mathbb{G}|} \leq \frac{2q_s(q_s + q_h)}{2^\lambda}$$

where we used $|\mathbb{G}| = p \geq 2^{\lambda-1}$.

From Algorithm $\mathcal{B}$ to Algorithm $\mathcal{C}$. The only difference between $\mathcal{B}$ and $\mathcal{C}$ is that $\mathcal{C}$ has access to an external random oracle H, and uses a modified oracle $\mathrm{H}_{\mathcal{C}}$ that:

– Returns the programmed value $T[y]$ if it exists
– Otherwise queries the external oracle $\mathrm{H}(y)$

This is a purely syntactic change. In $\mathcal{B}$, non-programmed queries sample fresh randomness directly; in $\mathcal{C}$, they get it from the external oracle. Since both sources provide independent uniform values from $\mathbb{Z}_p$, the adversary's view is identical. Therefore:

$$\Pr[\mathsf{Single}^{\mathcal{B}}_{\mathsf{InpGen}}(\lambda) = 1] = \Pr[\mathsf{Single}^{\mathcal{C}}_{\mathsf{InpGen}}(\lambda) = 1]$$

Conclusion.. Finally, we need to prove that when $\mathcal{C}$ succeeds (i.e., doesn't abort), the output satisfies $g^{s^*} = R^* \cdot X^{\mathsf{H}(R^*,m^*)}$ where H is the external oracle. We know that $g^{s^*} = R^* \cdot X^{c^*}$ where $c^* = \mathrm{H}_{\mathcal{C}}(R^*, m^*)$. We claim that $\mathrm{H}_{\mathcal{C}}(R^*, m^*) = \mathrm{H}(R^*, m^*)$.

Assume for contradiction that $\mathrm{H}_{\mathcal{C}}(R^*, m^*) \neq \mathrm{H}(R^*, m^*)$. This means $T[R^*, m^*] \neq \mathrm{H}(R^*, m^*)$, so $T[R^*, m^*]$ must have been assigned during a $\textsc{Sign}_{\mathcal{C}}$ query. But whenever $\textsc{Sign}_{\mathcal{C}}$ sets $T[R, m] = c$, it also adds m to $\mathcal{Q}$. Therefore $m^* \in \mathcal{Q}$. However, $\mathcal{C}$ asserts that $m^* \notin \mathcal{Q}$, which is a contradiction. Thus $c^* = \mathrm{H}(R^*, m^*)$.

Combining the bounds:

$$\left|\mathsf{Adv}^{\mathrm{single}}_{\mathcal{C},\mathsf{InpGen}}(\lambda) - \mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{SchnorrSig}[\mathsf{GrGen},\mathsf{H}]}(\lambda)\right| \leq \frac{2q_s(q_s + q_h)}{2^\lambda}$$

□

Exercise 10.4. Prove Theorem 10.1. Use Lemma 10.2 and the forking lemma (Lemma 9.1).

Solution. We construct a p.p.t. algorithm $\mathcal{D}$ that solves the discrete logarithm problem using a p.p.t. EUF-CMA adversary $\mathcal{A}$ against Schnorr signatures.

Given a discrete logarithm instance $((\mathbb{G}, p, g), X)$ where $X = g^x$ for unknown x, algorithm $\mathcal{D}$ works as follows:

1. Run the forking game $\mathsf{Fork}^{\mathcal{C}}_{\mathsf{InpGen}}(\lambda)$ with algorithm $\mathcal{C}$ from Lemma 10.2
2. If the forking game returns $((R^*, m^*), s_1, (R^*, m^*), s_2) \neq \perp$, then we have:
   - From the first execution: By Lemma 10.2, we have $g^{s_1} = R^* \cdot X^{h_1}$ where $h_1 = \mathsf{H}(R^*, m^*)$
   - From the second execution: By Lemma 10.2, we have $g^{s_2} = R^* \cdot X^{h_2}$ where $h_2 = \mathsf{H}'(R^*, m^*)$
   - By the assertion $T[z] \neq T'[z]$ in $\mathsf{Fork}^{\mathcal{A}}_{\mathsf{InpGen}}(\lambda)$ with $z = (R^*, m^*)$: we have $h_1 \neq h_2$
3. From the two verification equations:

$$g^{s_1} = R^* \cdot X^{h_1} \text{ and } g^{s_2} = R^* \cdot X^{h_2}$$

   Dividing the first by the second:

$$g^{s_1 - s_2} = X^{h_1 - h_2}$$

4. Since $h_1 \neq h_2$ and we work in $\mathbb{Z}_p$, we can compute:

$$x = \frac{s_1 - s_2}{h_1 - h_2} \bmod p$$

5. Output x

Success Probability: Algorithm $\mathcal{D}$ succeeds in solving the discrete logarithm problem exactly when the forking game returns non-$\perp$ outputs. Therefore:

$$\mathsf{Adv}^{\mathrm{dlog}}_{\mathcal{D},\mathsf{GrGen}}(\lambda) = \mathsf{Adv}^{\mathrm{fork}}_{\mathcal{C},\mathsf{InpGen}}(\lambda)$$

By the forking lemma:

$$\begin{aligned}\mathsf{Adv}^{\mathrm{fork}}_{\mathcal{C},\mathsf{InpGen}}(\lambda) &\geq \mathsf{Adv}^{\mathrm{single}}_{\mathcal{C},\mathsf{InpGen}}(\lambda)\left(\frac{\mathsf{Adv}^{\mathrm{single}}_{\mathcal{C},\mathsf{InpGen}}(\lambda)}{q_h + 1} - \frac{1}{2^\lambda}\right) \\ &\geq \frac{\mathsf{Adv}^{\mathrm{single}}_{\mathcal{C},\mathsf{InpGen}}(\lambda)^2}{q_h + 1} - \frac{1}{2^\lambda}\end{aligned}$$

because $\mathsf{Adv}^{\mathrm{single}}_{\mathcal{C},\mathsf{InpGen}}(\lambda) \leq 1$.

By Lemma 10.2, we have:

$$\left|\mathsf{Adv}^{\mathrm{single}}_{\mathcal{C},\mathsf{InpGen}}(\lambda) - \mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{SchnorrSig}[\mathsf{GrGen},\mathsf{H}]}(\lambda)\right| \leq \frac{2q_s(q_s + q_h)}{2^\lambda}$$

This means:

$$\mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{SchnorrSig}[\mathsf{GrGen},\mathsf{H}]}(\lambda) - \frac{2q_s(q_s + q_h)}{2^\lambda} \leq \mathsf{Adv}^{\mathrm{single}}_{\mathcal{C},\mathsf{InpGen}}(\lambda) \leq \mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{SchnorrSig}[\mathsf{GrGen},\mathsf{H}]}(\lambda) + \frac{2q_s(q_s + q_h)}{2^\lambda}$$

Therefore:

$$\begin{aligned}\mathsf{Adv}^{\mathrm{dlog}}_{\mathcal{D},\mathsf{GrGen}}(\lambda) &\geq \frac{\mathsf{Adv}^{\mathrm{single}}_{\mathcal{C},\mathsf{InpGen}}(\lambda)^2}{q_h + 1} - \frac{1}{2^\lambda} \\ &\geq \frac{\left(\mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{SchnorrSig}[\mathsf{GrGen},\mathsf{H}]}(\lambda) - \frac{2q_s(q_s+q_h)}{2^\lambda}\right)^2}{q_h + 1} - \frac{1}{2^\lambda}\end{aligned}$$

Rearranging:

$$(q_h + 1)\left(\mathsf{Adv}^{\mathrm{dlog}}_{\mathcal{D},\mathsf{GrGen}}(\lambda) + \frac{1}{2^\lambda}\right) \geq \left(\mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{SchnorrSig}[\mathsf{GrGen},\mathsf{H}]}(\lambda) - \frac{2q_s(q_s + q_h)}{2^\lambda}\right)^2$$

Therefore:

$$\mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{SchnorrSig}[\mathsf{GrGen},\mathsf{H}]}(\lambda) \leq \frac{2q_s(q_s + q_h)}{2^\lambda} + \sqrt{(q_h + 1)\left(\mathsf{Adv}^{\mathrm{dlog}}_{\mathcal{D},\mathsf{GrGen}}(\lambda) + \frac{1}{2^\lambda}\right)}$$

q_s, q_h are polynomial in λ since $\mathcal{A}$ is p.p.t. Assuming the discrete logarithm problem is hard (i.e., $\mathsf{Adv}^{\mathrm{dlog}}_{\mathcal{D},\mathsf{GrGen}}(\lambda)$ is negligible), we conclude that $\mathsf{Adv}^{\mathrm{euf-cma}}_{\mathcal{A},\mathsf{SchnorrSig}[\mathsf{GrGen},\mathsf{H}]}(\lambda)$ is negligible. □

Exercise 10.5 (Optional). Study the formalization of adaptor signatures in Gerhart, Schröder, Soni, and Thyagarajan [Ger+24]. Explain what gaps existed in previous formalizations and how this work addresses them.

11 Signatures with Key Tweaking

Key tweaking is fundamental in hierarchical deterministic (HD) Bitcoin wallets [BIP32] and Taproot commitments [BIP341], enabling the derivation of multiple keys from a single master key and committing to additional data in public keys.

11.1 Syntax

Definition 11.1 (Tweaking Scheme for Signatures). *A tweaking scheme* TwSch *for a signature scheme* Sig *is a tuple* $(\mathcal{T}, \mathsf{TwSK}, \mathsf{TwPK})$ *where:*

- $\mathcal{T}$ *is a set of allowed tweaks, parameterized by the public parameters* pp *output by* Setup.
- $\mathsf{TwSK}(sk, t) \to \tilde{sk}$ *is a deterministic polynomial-time algorithm that takes a secret key* sk *and a tweak* $t \in \mathcal{T}$ *as input and outputs a tweaked secret key* $\tilde{sk}$.
- $\mathsf{TwPK}(pk, t) \to \tilde{pk}$ *is a deterministic polynomial-time algorithm that takes a public key* pk *and a tweak* $t \in \mathcal{T}$ *as input and outputs a tweaked public key* $\tilde{pk}$.

11.2 Key Tweaking for Schnorr Signatures

Definition 11.2 (SchnorrTS). *The tweaking scheme* $\mathsf{SchnorrTS}$ *for Schnorr signatures is defined as follows:*

- *The tweak set is* $\mathcal{T} := \mathbb{Z}_p \times \mathbb{Z}_p^*$ *where* $\mathbb{Z}_p^* = \mathbb{Z}_p \setminus \{0\}$.
- *For* $(\alpha, \beta) \in \mathcal{T}$, *the secret key tweaking algorithm is:*

$$\mathsf{TwSK}(x, (\alpha, \beta)) := \alpha + \beta \cdot x \bmod p$$

  for every $x \in \mathbb{Z}_p$.
- *For* $(\alpha, \beta) \in \mathcal{T}$, *the public key tweaking algorithm is:*

$$\mathsf{TwPK}(X, (\alpha, \beta)) := g^{\alpha} \cdot X^{\beta}$$

  for every $X \in \mathbb{G}$.

Definition 11.3 (Schnorr Signature with Key Prefixing). *The Schnorr signature scheme with key prefixing* $\mathsf{SchnorrSigKP}[\mathsf{GrGen}, \mathsf{H}]$ *is identical to* $\mathsf{SchnorrSig}[\mathsf{GrGen}, \mathsf{H}]$ *except that the hash is computed as:*

$$c = \mathsf{H.Eval}(\kappa, (R, X, m))$$

instead of $c = \mathsf{H.Eval}(\kappa, (R, m))$, *where* X *is the public key.*

11.3 Security with Tweaked Keys

The security notion of *existential unforgeability under chosen message attack with tweaked keys (EUF-CMA-TK)* extends the standard EUF-CMA notion to capture security when an adversary can obtain signatures under tweaked keys. Informally, the adversary is given a public key and can request signatures on messages of their choice, but now with an additional capability: for each signing query, they can specify a tweak in addition to the message. The signing oracle will return a signature that verifies under the tweaked public key (i.e., a signature created with the correspondingly tweaked secret key). The adversary wins by outputting (m^*, t^*, σ^*) such that the signature verifies under the tweaked public key for t^*, and the pair (m^*, t^*) was never queried to the signing oracle.

The formal definition of EUF-CMA-TK is left as an exercise (see Exercise 11.3).

Theorem 11.1 (SchnorrSigKP with SchnorrTS is EUF-CMA-TK secure). *Let* GrGen *be a group generation algorithm for which the discrete logarithm problem is hard and let* H *be a hash function with output space* $\mathbb{Z}_p$. *Then the Schnorr signature scheme with key prefixing* $\mathsf{SchnorrSigKP}[\mathsf{GrGen}, \mathsf{H}]$ *with tweaking scheme* $\mathsf{SchnorrTS}$ *is EUF-CMA-TK secure in the random oracle model for* H.

The proof is left as an exercise (see Exercise 11.6).

11.4 Exercises

Exercise 11.1. Define correctness for a signature scheme with tweaking. Specifically, modify standard signature scheme correctness (Definition 10.2) to ensure that signatures created with a tweaked secret key verify correctly under the corresponding tweaked public key.

Solution. A signature scheme $\mathsf{Sig} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ with tweaking scheme $\mathsf{TwSch} = (\mathcal{T}, \mathsf{TwSK}, \mathsf{TwPK})$ is *correct with tweaking* if:

1. The signature scheme Sig is correct (as per the standard definition).
2. For all $\lambda \in \mathbb{N}$, all $pp \in \mathsf{Setup}(1^\lambda)$, all $(sk, pk) \in \mathsf{KeyGen}(pp)$, all tweaks $t \in \mathcal{T}$, and all messages $m \in \mathcal{M}$:
$$\Pr[\mathsf{Verify}(pp, \mathsf{TwPK}(pk, t), m, \mathsf{Sign}(pp, \mathsf{TwSK}(sk, t), m)) = 1] = 1$$
where the probability is taken over the randomness of Sign.

In other words, signatures created with a tweaked secret key must always verify correctly under the corresponding tweaked public key. □

Exercise 11.2 (Optional). Argue that the tweaking scheme SchnorrTS (Definition 11.2) is correct with the Schnorr signature scheme (Definition 10.4). That is, show that signatures created with a tweaked secret key verify correctly under the corresponding tweaked public key.

Solution. For SchnorrTS with Schnorr signatures, if $sk = x$ and $pk = X = g^x$, then for tweak $(\alpha, \beta) \in \mathcal{T}$:

- Tweaked secret key: $\tilde{x} = \mathsf{TwSK}(x, (\alpha, \beta)) = \alpha + \beta \cdot x$
- Tweaked public key: $\tilde{X} = \mathsf{TwPK}(X, (\alpha, \beta)) = g^\alpha \cdot X^\beta = g^\alpha \cdot g^{\beta x} = g^{\alpha + \beta x} = g^{\tilde{x}}$

The key observation is that the discrete logarithm relationship is preserved: $\tilde{X} = g^{\tilde{x}}$. This means that $(\tilde{x}, \tilde{X})$ is a valid key pair that could have been output by KeyGen. By the correctness of Schnorr signatures (Lemma 10.1), signatures created with $\tilde{x}$ will verify under $\tilde{X}$. □

Exercise 11.3. Formalize the EUF-CMA-TK security notion by adapting EUF-CMA security (Definition 10.3). Your definition should capture the informal description given in subsection 11.3.

Solution. A signature scheme $\mathsf{Sig} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ with tweaking scheme $\mathsf{TwSch} = (\mathcal{T}, \mathsf{TwSK}, \mathsf{TwPK})$ is *existentially unforgeable under chosen message attack with tweaking (EUF-CMA-TK)* if for all p.p.t. adversaries $\mathcal{A}$:

$$\mathsf{Adv}^{\mathrm{euf-cma-tk}}_{\mathcal{A},\mathsf{Sig},\mathsf{TwSch}}(\lambda) := \Pr\left[\mathsf{Game\ EUF\text{-}CMA\text{-}TK}^{\mathcal{A}}_{\mathsf{Sig},\mathsf{TwSch}}(\lambda) = 1\right] = negl(\lambda)$$

where $\mathsf{Game\ EUF\text{-}CMA\text{-}TK}^{\mathcal{A}}_{\mathsf{Sig},\mathsf{TwSch}}(\lambda)$ is defined as follows:

$\mathsf{Game\ EUF\text{-}CMA\text{-}TK}^{\mathcal{A}}_{\mathsf{Sig},\mathsf{TwSch}}(\lambda)$	$\mathrm{Oracle\ SIGN}(m, t)$
$pp := \mathsf{Setup}(1^\lambda)$	$\tilde{sk} := \mathsf{TwSK}(sk, t)$
$(sk, pk) := \mathsf{KeyGen}(pp)$	$\sigma := \mathsf{Sign}(pp, \tilde{sk}, m)$
$\mathcal{Q} := \emptyset$	$\mathcal{Q} := \mathcal{Q} \cup \{(m, t)\}$
$(m^*, t^*, \sigma^*) := \mathcal{A}^{\mathrm{SIGN}}(pp, pk)$	**return** σ
return $(m^*, t^*) \notin \mathcal{Q} \wedge$	
$\mathsf{Verify}(pp, \mathsf{TwPK}(pk, t^*), m^*, \sigma^*) = 1$	

Fig. 11.1. The EUF-CMA-TK security game for signatures with tweaking. Changes from the standard EUF-CMA game are highlighted.

□

Exercise 11.4. Give a p.p.t. adversary that breaks the EUF-CMA-TK security of SchnorrSig with the SchnorrTS tweaking scheme. Hints:

- SchnorrSig does not use key prefixing.
- You only need to use the signing oracle once.
- Pick a tweak t and look at the Schnorr verification equation for a signature (R, s) under public key X. Compute s' by adding a value to s such that (R, s') is a valid signature for $\mathsf{TwPK}(X, t)$.

Solution. The adversary $\mathcal{A}$ that breaks EUF-CMA-TK security is shown in Figure 11.2.

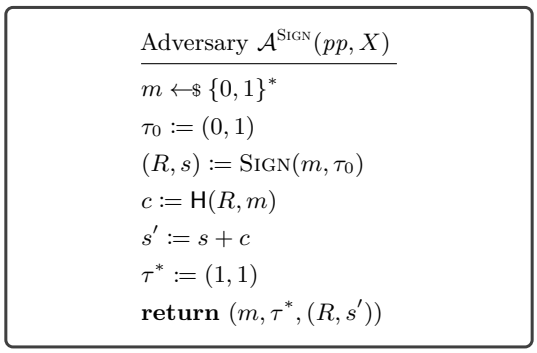

Fig. 11.2. Adversary breaking EUF-CMA-TK security of Schnorr signatures with SchnorrTS tweaking.

Why this attack succeeds:
When querying the signing oracle with $\tau_0 = (0, 1)$:

- The tweaked public key is $\mathsf{TwPK}(X, (0,1)) = g^0 \cdot X^1 = X$ (the original key).
- The signature (R, s) satisfies: $g^s = R \cdot X^c$ where $c = \mathsf{H}(R, m)$.

For the forgery with $\tau^* = (1, 1)$:

- The tweaked public key is $\mathsf{TwPK}(X, (1,1)) = g^1 \cdot X = g \cdot X$.
- The game verifies: $g^{s'} = g^{s+c} \stackrel{?}{=} R \cdot \mathsf{TwPK}(X, (1,1))^c$.

The verification succeeds because:

$$\begin{aligned} g^{s+c} &= g^s \cdot g^c \\ &= R \cdot X^c \cdot g^c \\ &= R \cdot (X \cdot g)^c \\ &= R \cdot \mathsf{TwPK}(X, (1,1))^c \end{aligned}$$

Since $(m, (0,1)) \neq (m, (1,1))$, this is a valid forgery that breaks EUF-CMA-TK security. □

Exercise 11.5. Explain why we cannot directly apply the EUF-CMA security proof of SchnorrSig to prove EUF-CMA-TK security for SchnorrSig with SchnorrTS tweaking.

Hint: Examine what Lemma 10.2 guarantees about $\mathcal{C}$'s output and why this guarantee holds. Then consider whether a similar guarantee could be established for a reduction $\mathcal{C}'$ that uses an adversary $\mathcal{A}$ breaking EUF-CMA-TK for SchnorrSig with SchnorrTS.

Solution. The EUF-CMA security proof for SchnorrSig (Lemma 10.2) requires that when the reduction algorithm $\mathcal{C}$ outputs (z, aux) with $z = (R^*, m^*)$ and $aux = s^*$, we have:

$$g^{s^*} = R^* \cdot X^{\mathsf{H}(R^*, m^*)}$$

The proof of this property relies on showing that the programmed oracle value $T[R^*, m^*]$ equals the external oracle $\mathsf{H}(R^*, m^*)$. This is proven by contradiction: if they differ, then $T[R^*, m^*]$ must have been set during a signing query, which means $m^* \in \mathcal{Q}$. But the reduction asserts $m^* \notin \mathcal{Q}$, giving us the contradiction.

In the EUF-CMA-TK setting, this proof breaks down:

– The adversary outputs (m^*, t^*, σ^*) for some tweak t^*.
– The winning condition checks $(m^*, t^*) \notin \mathcal{Q}$, not $m^* \notin \mathcal{Q}$.
– This means the adversary could have queried the signing oracle with m^* with a different tweak $t' \neq t^*$.
– If m^* was queried to the signing oracle with tweak t', and the adversary's forgery reuses the same R from that query, then $T[R^*, m^*]$ has been assigned to a uniformly random value, which is unequal to $\mathsf{H}(R^*, m^*)$ with overwhelming probability.

Since $\mathsf{H}_{\mathcal{C}}(R^*, m^*) \neq \mathsf{H}(R^*, m^*)$ (with overwhelming probability), we cannot use the forking lemma's guarantee that $\mathsf{H}(R^*, m^*) \neq \mathsf{H}'(R^*, m^*)$ to extract the secret key from the two signatures. □

Exercise 11.6. Prove the following lemma, which is an adaptation of Lemma 10.2 for the EUF-CMA-TK security of SchnorrSigKP with SchnorrTS.

Lemma 11.1. *Let $\mathcal{A}$ be an adversary against the EUF-CMA-TK security of* SchnorrSigKP *with SchnorrTS*[GrGen, H] *in the random oracle model for* H *making at most q_s queries to* Sign *and at most q_h queries to* H.

Let InpGen *be the algorithm which on input 1^λ runs $(\mathbb{G}, p, g) \coloneqq \mathsf{GrGen}(1^\lambda)$, draws $X \leftarrow\!\!\$\ \mathbb{G}$, and returns $((\mathbb{G}, p, g), X)$.*

There exists algorithm $\mathcal{C}$ that takes as input $((\mathbb{G}, p, g), X) \coloneqq \mathsf{InpGen}(1^\lambda)$, has access to a random oracle H*, makes at most $q_h + 1$ queries to* H*, and satisfies*

$$\left|\mathsf{Adv}^{\text{single}}_{\mathcal{C},\mathsf{InpGen}}(\lambda) - \mathsf{Adv}^{\text{euf-cma-tk}}_{\mathcal{A},\mathsf{SchnorrSigKP},\mathsf{SchnorrTS}}(\lambda)\right| \leq \frac{q_s(q_s + q_h)}{2^\lambda}$$

with $\mathsf{Adv}^{\text{single}}_{\mathcal{C},\mathsf{InpGen}}(\lambda)$ as defined in the forking lemma (Lemma 9.1).

Moreover, when $\mathcal{C}$ returns a non-$\perp$ output (z, aux), then $z = (R^*, \mathsf{TwPK}(X, t^*), m^*, t^*)$, $aux = s^*$, and

$$g^{s^*} = R^* \cdot \mathsf{TwPK}(X, t^*)^{\mathsf{H}(R^*, \mathsf{TwPK}(X,t^*), m^*)}$$

Adapt the proof of Lemma 10.2 to prove this lemma.

Note: Applying the forking lemma to $\mathcal{C}$ would allow extracting the discrete logarithm via $g^{s_1 - s_2} = (g^\alpha \cdot X^\beta)^{h_1 - h_2}$ where $(\alpha, \beta) = t^*$, thus establishing the EUF-CMA-TK security of SchnorrSigKP with SchnorrTS.

Solution. We adapt the proof of Lemma 10.2 by modifying the games to handle tweaking and key prefixing.

The games are shown in Figure 11.3.

Proof that $g^{s^*} = R^* \cdot \mathsf{TwPK}(X, t^*)^{\mathsf{H}(R^*, \mathsf{TwPK}(X,t^*), m^*)}$: When $\mathcal{C}$ succeeds, we have $g^{s^*} = R^* \cdot \tilde{X}^{*c^*}$ where $\tilde{X}^* = \mathsf{TwPK}(X, t^*)$ and $c^* = \mathrm{H}_{\mathcal{C}}(R^*, \tilde{X}^*, m^*)$. We claim that $\mathrm{H}_{\mathcal{C}}(R^*, \tilde{X}^*, m^*) = \mathsf{H}(R^*, \tilde{X}^*, m^*)$.

Assume for contradiction that $\mathrm{H}_{\mathcal{C}}(R^*, \tilde{X}^*, m^*) \neq \mathsf{H}(R^*, \tilde{X}^*, m^*)$. This means $T[R^*, \tilde{X}^*, m^*]$ was assigned during a $\textsc{Sign}_{\mathcal{C}}$ query. But whenever $\textsc{Sign}_{\mathcal{C}}$ sets $T[R, \tilde{X}, m] = c$ for some (m, t) with $\tilde{X} = \mathsf{TwPK}(X, t)$, it also adds (m, t) to $\mathcal{Q}$. Since $T[R^*, \tilde{X}^*, m^*]$ was set, there must exist some signing query where the key for T was $(R^*, \tilde{X}^*, m^*)$. This means $R = R^*$, $\tilde{X} = \tilde{X}^*$, and $m = m^*$ for that query. From $\tilde{X} = \tilde{X}^*$ and $\tilde{X} = \mathsf{TwPK}(X, t)$, $\tilde{X}^* = \mathsf{TwPK}(X, t^*)$, we have $\mathsf{TwPK}(X, t) = \mathsf{TwPK}(X, t^*)$. If $t \neq t^*$, then $\mathcal{C}$ would have detected this collision and directly extracted the discrete logarithm. Otherwise $t = t^*$, so $(m^*, t^*) = (m, t) \in \mathcal{Q}$. However, $\mathcal{C}$ asserts that $(m^*, t^*) \notin \mathcal{Q}$, which is a contradiction. Thus $c^* = \mathsf{H}(R^*, \tilde{X}^*, m^*)$.

Note on collision extraction: If $(t, \tilde{X}) \in \mathcal{T}$ with $t = (\alpha, \beta) \neq t^* = (\alpha^*, \beta^*)$ but $\mathsf{TwPK}(X, t) = \mathsf{TwPK}(X, t^*)$, then $g^\alpha \cdot X^\beta = g^{\alpha^*} \cdot X^{\beta^*}$. This implies $g^\alpha \cdot g^{\beta x} = g^{\alpha^*} \cdot g^{\beta^* x}$, so $(\beta - \beta^*)x = \alpha^* - \alpha$. If $\beta = \beta^*$, then $\alpha = \alpha^*$, contradicting $t \neq t^*$. Thus $\beta \neq \beta^*$ and we can extract $x = (\alpha^* - \alpha)/(\beta - \beta^*) \bmod p$.

The bound $|\mathsf{Adv}^{\text{single}}_{\mathcal{C},\mathsf{InpGen}}(\lambda) - \mathsf{Adv}^{\text{euf-cma-tk}}_{\mathcal{A},\mathsf{SchnorrSigKP},\mathsf{SchnorrTS}}(\lambda)| \leq \frac{q_s(q_s+q_h)}{2^\lambda}$ follows from the same analysis as in Lemma 10.2, where the error term accounts for the probability of collisions in the programmed oracle. □

$\mathsf{EUF\text{-}CMA\text{-}TK}^{\mathcal{A}}_{\mathsf{SchnorrSigKP},\mathsf{SchnorrTS}}(\lambda)$

$(\mathbb{G}, p, g) \coloneqq \mathsf{GrGen}(1^\lambda)$
$x \leftarrow\!\!\$\ \mathbb{Z}_p$
$X \coloneqq g^x$
$pp \coloneqq ((\mathbb{G}, p, g), \{0,1\}^*, \mathbb{G} \times \mathbb{Z}_p)$
$T \coloneqq \emptyset$
$\mathcal{Q} \coloneqq \emptyset$
$(m^*, t^*, \sigma^*) \coloneqq \mathcal{A}^{\text{SIGN},\text{H}}(pp, X)$
$(R^*, s^*) \coloneqq \sigma^*$
$\tilde{X}^* \coloneqq \mathsf{TwPK}(X, t^*)$
$c^* \coloneqq \text{H}(R^*, \tilde{X}^*, m^*)$
return $(m^*, t^*) \notin \mathcal{Q} \wedge$
$\quad g^{s^*} = R^* \cdot \tilde{X}^{*c^*}$

Oracle $\text{SIGN}(m, t)$

$\tilde{x} \coloneqq \mathsf{TwSK}(x, t)$
$\tilde{X} \coloneqq \mathsf{TwPK}(X, t)$
$r \leftarrow\!\!\$\ \mathbb{Z}_p$
$R \coloneqq g^r$
$c \coloneqq \text{H}(R, \tilde{X}, m)$
$s \coloneqq r + c \cdot \tilde{x} \bmod p$
$\mathcal{Q} \coloneqq \mathcal{Q} \cup \{(m, t)\}$
return (R, s)

Oracle $\text{H}(y)$

if $T[y] = \bot$ **then**
$\quad T[y] \leftarrow\!\!\$\ \mathbb{Z}_p$
return $T[y]$

$\mathcal{B}((\mathbb{G}, p, g), X; \rho)$

$pp \coloneqq ((\mathbb{G}, p, g), \{0,1\}^*, \mathbb{G} \times \mathbb{Z}_p)$
$T \coloneqq \emptyset$
$\mathcal{Q} \coloneqq \emptyset$
$(m^*, t^*, \sigma^*) \coloneqq$
$\quad \mathcal{A}^{\text{SIGN}_\mathcal{B},\text{H}_\mathcal{B}}(pp, X)$
$(R^*, s^*) \coloneqq \sigma^*$
$\tilde{X}^* \coloneqq \mathsf{TwPK}(X, t^*)$
$c^* \coloneqq \text{H}_\mathcal{B}(R^*, \tilde{X}^*, m^*)$
assert $(m^*, t^*) \notin \mathcal{Q}$
assert $g^{s^*} = R^* \cdot \tilde{X}^{*c^*}$
return $((R^*, \tilde{X}^*, m^*, t^*), s^*)$

Oracle $\text{SIGN}_\mathcal{B}(m, t)$

$\tilde{X} \coloneqq \mathsf{TwPK}(X, t)$
$s \leftarrow\!\!\$\ \mathbb{Z}_p$
$c \leftarrow\!\!\$\ \mathbb{Z}_p$
$R \coloneqq g^s \cdot \tilde{X}^{-c}$
assert $T[R, \tilde{X}, m] = \bot$
$T[R, \tilde{X}, m] \coloneqq c$
$\mathcal{Q} \coloneqq \mathcal{Q} \cup \{(m, t)\}$
return (R, s)

Oracle $\text{H}_\mathcal{B}(y)$

if $T[y] = \bot$ **then**
$\quad T[y] \leftarrow\!\!\$\ \mathbb{Z}_p$
return $T[y]$

$\mathcal{C}^{\text{H}}((\mathbb{G}, p, g), X; \rho)$

$pp \coloneqq ((\mathbb{G}, p, g), \{0,1\}^*, \mathbb{G} \times \mathbb{Z}_p)$
$T \coloneqq \emptyset$
$\mathcal{Q} \coloneqq \emptyset$
$\quad$ // Track $(t, \tilde{X})$ pairs
$\mathcal{T} \coloneqq \emptyset$
$(m^*, t^*, \sigma^*) \coloneqq$
$\quad \mathcal{A}^{\text{SIGN}_\mathcal{C},\text{H}_\mathcal{C}}(pp, X)$
$(R^*, s^*) \coloneqq \sigma^*$
$\tilde{X}^* \coloneqq \mathsf{TwPK}(X, t^*)$
$c^* \coloneqq \text{H}_\mathcal{C}(R^*, \tilde{X}^*, m^*)$
assert $(m^*, t^*) \notin \mathcal{Q}$
assert $g^{s^*} = R^* \cdot \tilde{X}^{*c^*}$
$\quad$ // Check for tweaking collision
if $\exists (t, \tilde{X}) \in \mathcal{T}:$
$\quad \tilde{X} = \tilde{X}^* \wedge t \neq t^*$ **then**
$\quad (\alpha, \beta) \coloneqq t; (\alpha^*, \beta^*) \coloneqq t^*$
$\quad x \coloneqq (\alpha - \alpha^*)/(\beta^* - \beta) \bmod p$
$\quad$ **return** $((\bot, \bot, \bot, \bot), x)$
return $((R^*, \tilde{X}^*, m^*, t^*), s^*)$

Oracle $\text{SIGN}_\mathcal{C}(m, t)$

$\tilde{X} \coloneqq \mathsf{TwPK}(X, t)$
$\mathcal{T} \coloneqq \mathcal{T} \cup \{(t, \tilde{X})\}$
$s \leftarrow\!\!\$\ \mathbb{Z}_p$
$c \leftarrow\!\!\$\ \mathbb{Z}_p$
$R \coloneqq g^s \cdot \tilde{X}^{-c}$
assert $T[R, \tilde{X}, m] = \bot$
$T[R, \tilde{X}, m] \coloneqq c$
$\mathcal{Q} \coloneqq \mathcal{Q} \cup \{(m, t)\}$
return (R, s)

Oracle $\text{H}_\mathcal{C}(y)$

if $T[y] = \bot$ **then**
$\quad T[y] \coloneqq \text{H}(y)$
return $T[y]$

Fig. 11.3. Security reduction for SchnorrSigKP with SchnorrTS: EUF-CMA-TK game (left), algorithm $\mathcal{B}$ (middle), and algorithm $\mathcal{C}$ (right). Changes from the previous game are marked with ·.

Exercise 11.7 (Optional). Read about signatures with re-randomizable keys in Section 3 of Fleischhacker et al. [Fle+16], which presents a notion closely related to key tweaking. Compare their security model with the EUF-CMA-TK model discussed in this section and discuss the practical impact. How can they prove security for Schnorr signatures with re-randomized keys without requiring public key prefixing?

12 Interactive Aggregate Signatures

An interactive aggregate signature (IAS) scheme allows n signers, each with their own key pair (sk_i, pk_i) and message m_i, to jointly produce a single short signature that proves m_i was signed under pk_i for every $i \in \{1, \ldots, n\}$.

The following exercises are based on the DahLIAS paper [NRS25], which presents a discrete logarithm-based IAS scheme with constant-size signatures.

12.1 Exercises

Exercise 12.1. Read the abstract and the main security theorem (Theorem 2) of DahLIAS. What does the security theorem tell us about the relationship between breaking DahLIAS and solving the underlying computational problems?

Exercise 12.2. Read the technical introduction (Section 1.3). Why does the straightforward adaptation of the MuSig2 two-round technique described in the text not yield a secure IAS scheme?

Exercise 12.3. Read about IAS syntax and security (Sections 3.1 and 3.3). What is the difference between EUF-CMA and co-EUF-CMA security for IAS?

Exercise 12.4. What are the applications where strong binding (Section 6) matters? Explain why this property is important in these contexts.

Exercise 12.5 (Optional). Read the proof sketch (Section 5.2). Outline the main proof technique and explain how the security reduction works.

A Notation

A.1 Sets and Numbers

- $\mathbb{N}$ denotes the set of natural numbers $\{0, 1, 2, \ldots\}$.
- $\mathbb{Z}$ denotes the set of integers.
- $\mathbb{Z}_p$ denotes the set of integers modulo p, i.e., $\{0, 1, \ldots, p-1\}$.
- $\mathbb{Z}_p^*$ denotes the set of integers modulo p excluding zero, i.e., $\{1, 2, \ldots, p-1\}$.
- $[n]$ denotes the set $\{1, 2, \ldots, n\}$ for a positive integer n.
- $[a, b]$ denotes the set $\{a, a+1, \ldots, b\}$ for integers $a \leq b$.
- $\{0,1\}^n$ denotes the set of all binary strings of length n.
- $\{0,1\}^*$ denotes the set of all binary strings of finite length.

A.2 Probability and Sampling

- $x \leftarrow\!\!\$\ S$ denotes sampling x uniformly at random from the finite set S.
- $x := \mathsf{A}(y)$ denotes running algorithm A on input y and assigning the output to x.
- $\Pr[E]$ denotes the probability of event E.
- $:=$ denotes a definition, i.e., the left side is defined to equal the right side.

A.3 Cryptographic Notation

- λ denotes the security parameter.
- 1^λ denotes the security parameter in unary representation.
- $\mathit{negl}(\lambda)$ denotes a negligible function in the security parameter.
- p.p.t. stands for probabilistic polynomial-time.

A.4 Groups and Fields

- $(\mathbb{G}, p, g)$ denotes group parameters, where $\mathbb{G}$ denotes a cyclic group, g denotes the generator and p denotes the order of a group.
- $1_{\mathbb{G}}$ denotes the identity element of group $\mathbb{G}$.

A.5 Logical Operators

- $\wedge$ denotes logical AND.
- $\vee$ denotes logical OR.
- $\neg$ denotes logical NOT.
- $\oplus$ denotes bitwise XOR.

A.6 Other Notation

- $\|$ denotes concatenation (e.g., $x\|y$ is the concatenation of x and y).
- $|S|$ denotes the cardinality (size) of set S.
- $\perp$ denotes an error or undefined value.
- $\emptyset$ denotes the empty set.
- $\log_g(h)$ denotes the discrete logarithm of h with respect to base g.
- $O(\cdot)$ denotes asymptotic upper bound (big-O notation).

B Appendix: Proofs of Negligible Function Properties

This appendix contains the proofs of properties (2)-(4) from Lemma 1.1.

Proof of Lemma 1.1(2). Let f_1, f_2 be negligible functions. We need to show that $f_1 + f_2$ is negligible.
Let $c > 0$ be arbitrary. We must find N such that $(f_1 + f_2)(\lambda) < \lambda^{-c}$ for all $\lambda > N$.
Since f_1 and f_2 are negligible, for the constant $c + 1 > 0$, there exist $N_1, N_2 \in \mathbb{N}$ such that:

$$\forall \lambda > N_1 : f_1(\lambda) < \lambda^{-(c+1)} \quad \text{and} \quad \forall \lambda > N_2 : f_2(\lambda) < \lambda^{-(c+1)}$$

Let $N = \max(N_1, N_2, 2)$. Then for all $\lambda > N$:

$$f_1(\lambda) + f_2(\lambda) < \lambda^{-(c+1)} + \lambda^{-(c+1)} = 2 \cdot \lambda^{-(c+1)} = \frac{2}{\lambda} \cdot \lambda^{-c} \leq \lambda^{-c}$$

where the last inequality holds because $\frac{2}{\lambda} \leq 1$ when $\lambda \geq 2$, which is guaranteed by our choice of N. □

Proof of Lemma 1.1(3). Let f be a negligible function with $f(\lambda) \geq 0$ and let $k > 0$ be a constant. We need to show that $f + k$ is not negligible.

Assume for contradiction that $f + k$ is negligible. Then for the constant $c = 1$, there exists $N \in \mathbb{N}$ such that for all $\lambda > N$:

$$f(\lambda) + k < \lambda^{-1}$$

Since $f(\lambda) \geq 0$, we have $f(\lambda) + k \geq k$ for all λ.
Choose $\lambda_0 = \max(N + 1, \lceil \frac{2}{k} \rceil)$. Then $\lambda_0 > N$ and $\lambda_0^{-1} \leq \frac{k}{2}$.
By our assumption, we must have:

$$k \leq f(\lambda_0) + k < \lambda_0^{-1} \leq \frac{k}{2}$$

This gives us $k < \frac{k}{2}$, which is a contradiction.
Therefore, $f + k$ is not negligible. □

Proof of Lemma 1.1(4). Let f be a non-negligible function and g be a negligible function. Assume for contradiction that $f - g$ is negligible. Then by property (2), $(f - g) + g = f$ would be negligible (as the sum of two negligible functions). This contradicts the assumption that f is non-negligible. □

C Appendix: Notation

Acknowledgements

We thank Nadav Kohen and Pieter Wuille for valuable feedback and discussions. We also thank Claude for assistance with revisions and exercises.

Changelog

2025-08-24
- First public version.

References

[BCY20] Foteini Baldimtsi, Ran Canetti, and Sophia Yakoubov. "Universally Composable Accumulators". In: *CT-RSA 2020*. DOI: 10.1007/978-3-030-40186-3_27.

[BDL19] Mihir Bellare, Wei Dai, and Lucy Li. "The Local Forking Lemma and Its Application to Deterministic Encryption". In: *ASIACRYPT 2019, Part III*. DOI: 10.1007/978-3-030-34618-8_21.

[BIP32] Pieter Wuille. *Hierarchical Deterministic Wallets*. Bitcoin Improvement Proposal 32. See https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki. 2012.

[BIP341] Pieter Wuille, Jonas Nick, and Anthony Towns. *Taproot: SegWit version 1 spending rules*. Bitcoin Improvement Proposal 341. See https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki. 2020.

[BIP352] josibake and Ruben Somsen. *Silent Payments*. See https://github.com/bitcoin/bips/blob/master/bip-0352.mediawiki. 2023.

[BL13] Daniel J. Bernstein and Tanja Lange. "Non-uniform Cracks in the Concrete: The Power of Free Precomputation". In: *ASIACRYPT 2013, Part II*. DOI: 10.1007/978-3-642-42045-0_17.

[BN06] Mihir Bellare and Gregory Neven. "Multi-signatures in the plain public-Key model and a general forking lemma". In: *ACM CCS 2006*. DOI: 10.1145/1180405.1180453.

[BR93] Mihir Bellare and Phillip Rogaway. "Random Oracles are Practical: A Paradigm for Designing Efficient Protocols". In: *ACM CCS 93*. DOI: 10.1145/168588.168596.

[CGH98] Ran Canetti, Oded Goldreich, and Shai Halevi. "The Random Oracle Methodology, Revisited (Preliminary Version)". In: *30th ACM STOC*. DOI: 10.1145/276698.276741.

[CLW24] Cas Cremers, Julian Loss, and Benedikt Wagner. "A Holistic Security Analysis of Monero Transactions". In: *EUROCRYPT 2024, Part III*. DOI: 10.1007/978-3-031-58734-4_5.

[Dri+19] Manu Drijvers, Kasra Edalatnejad, Bryan Ford, Eike Kiltz, Julian Loss, Gregory Neven, and Igors Stepanovs. "On the Security of Two-Round Multi-Signatures". In: *2019 IEEE Symposium on Security and Privacy*. DOI: 10.1109/SP.2019.00050.

[FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. "The Algebraic Group Model and its Applications". In: *CRYPTO 2018, Part II*. DOI: 10.1007/978-3-319-96881-0_2.

[Fle+16] Nils Fleischhacker, Johannes Krupp, Giulio Malavolta, Jonas Schneider, Dominique Schröder, and Mark Simkin. "Efficient Unlinkable Sanitizable Signatures from Signatures with Re-randomizable Keys". In: *PKC 2016, Part I*. DOI: 10.1007/978-3-662-49384-7_12.

[Ger+24] Paul Gerhart, Dominique Schröder, Pratik Soni, and Sri Aravinda Krishnan Thyagarajan. "Foundations of Adaptor Signatures". In: *EUROCRYPT 2024, Part II*. DOI: 10.1007/978-3-031-58723-8_6.

[HRS16] Andreas Hülsing, Joost Rijneveld, and Fang Song. "Mitigating Multi-target Attacks in Hash-Based Signatures". In: *PKC 2016, Part I*. DOI: 10.1007/978-3-662-49384-7_15.

[Hül13] Andreas Hülsing. "W-OTS+ - Shorter Signatures for Hash-Based Signature Schemes". In: *AFRICACRYPT 13*. DOI: 10.1007/978-3-642-38553-7_10.

[KM15] Neal Koblitz and Alfred J. Menezes. "The random oracle model: a twenty-year retrospective". In: *DCC* 77.2-3 (2015). DOI: 10.1007/s10623-015-0094-2.

[Mau05] Ueli M. Maurer. "Abstract Models of Computation in Cryptography (Invited Paper)". In: *10th IMA International Conference on Cryptography and Coding*. DOI: 10.1007/11586821_1.

[Max+19] Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. "Simple Schnorr multi-signatures with applications to Bitcoin". In: *DCC* 87.9 (2019). DOI: 10.1007/s10623-019-00608-x.

[Nao03] Moni Naor. "On Cryptographic Assumptions and Challenges (Invited Talk)". In: *CRYPTO 2003*. DOI: 10.1007/978-3-540-45146-4_6.

[NEL25] Jonas Nick, Liam Eagen, and Robin Linus. "Shielded CSV: Private and Efficient Client-Side Validation". In: *Cryptology ePrint Archive* (2025).

[NRS25] Jonas Nick, Tim Ruffing, and Yannick Seurin. "DahLIAS: Discrete Logarithm-Based Interactive Aggregate Signatures". In: *Cryptology ePrint Archive* (2025).

[Rog06] Phillip Rogaway. "Formalizing Human Ignorance". In: *Progress in Cryptology - VIETCRYPT 06*. DOI: 10.1007/11958239_14.

[Sho97] Victor Shoup. "Lower Bounds for Discrete Logarithms and Related Problems". In: *EUROCRYPT'97*. DOI: 10.1007/3-540-69053-0_18.

[Wag02] David Wagner. "A Generalized Birthday Problem". In: *CRYPTO 2002*. DOI: 10.1007/3-540-45708-9_19.